

小白学视觉微信公众号出品

2021年7月

Pytorch

常用函数手册



小白学视觉团队

小白学视觉团队介绍

小白学视觉团队由哈尔滨工业大学在读博士创建，主要成员来自哈尔滨工业大学、华中科技大学、香港理工大学、苏黎世联邦理工等。团队成员研究方向聚焦计算机视觉、图像处理、深度学习等领域，热衷于分享相关领域入门知识、前沿进展等。

团队每个月定期举行论文解读讲座，不定期与大家直播聊天，所设计话题包含招聘、学术、科研、工作、思维等。

创始人出版《OpenCV4 快速入门》和《OpenCV 4 详解：基于 Python 版》，累计销售近 2 两万册。

团队维护 2 个主要微信公众号，扫码关注作者微信公众号，领取更多资料~



小白学视觉

面向初学者介绍计算机视觉基础知识、OpenCV 使用、SLAM 技术，深度学习等内容。



深度学习爱好者

分享机器学习、深度学习和 Python 等知识与原理，每天分享深度学习与计算机视觉领域的经典和最新的论文总结，带读者一起跟踪前沿科技！



作者微信

添加作者微信，与作者思维沟通

目录

目录.....	2
第一章 torch.....	3
张量 Tensors	3
创建操作 Creation Ops.....	4
索引,切片,连接,换位 Indexing, Slicing, Joining, Mutating Ops	9
随机抽样 Random sampling	15
序列化 Serialization	19
并行化 Parallelism	20
数学操作 Math operations.....	20
Reduction Ops	42
比较操作 Comparison Ops	52
其它操作 Other Operations.....	60
BLAS and LAPACK Operations.....	66
第二章 torch.Tensor	78
第三章 torch.Storage.....	95
第四章 torch.nn	97
第五章 torch.nn.functional.....	139
第六章 torch.nn.init.....	144
第七章 torch.optim.....	148
第八章 Automatic differentiation package - torch.autograd	154
第九章 torch.multiprocessing.....	159
第十章 torch.legacy	161
第十一章 torch.cuda.....	162
第十二章 torch.utils.ffi	165
第十三章 torch.utils.data	166
第十四章 torch.utils.model_zoo	168

第一章 torch

包 `torch` 包含了多维张量的数据结构以及基于其上的多种数学操作。另外，它也提供了多种工具，

其中一些可以更有效地对张量和任意类型进行序列化。

它有 CUDA 的对应实现，可以在 NVIDIA GPU 上进行张量运算(计算能力 ≥ 2.0)。

张量 Tensors

`torch.is_tensor`

```
torch.is_tensor(obj)
```

如果 `obj` 是一个 pytorch 张量，则返回 True

- 参数: `obj` (Object) – 判断对象

`torch.is_storage`

```
torch.is_storage(obj)
```

如何 `obj` 是一个 pytorch storage 对象，则返回 True

- 参数: `input` (Object) – 判断对象

`torch.__set_default_tensor_type__`

```
torch.set_default_tensor_type(t)
```

`torch.numel`

```
torch.numel(input) -> int
```

返回 `input` 张量中的元素个数

- 参数: `input` (*Tensor*) – 输入张量

例子:

```
>>> a = torch.randn(1,2,3,4,5)
```

```
>>> torch.numel(a)
```

```
120
```

```
>>> a = torch.zeros(4,4)
```

```
>>> torch.numel(a)
```

```
16
```

`torch.set_printoptions`

```
torch.set_printoptions(precision=None, threshold=None, edgeitems=None, linewidth=None, profile=None)
```

设置打印选项。 完全参考自 *Numpy*。

参数:

- `precision` – 浮点数输出的精度位数 (默认为 8)
- `threshold` – 阈值，触发汇总显示而不是完全显示(repr)的数组元素的总数 (默认为 1000)
- `edgeitems` – 汇总显示中，每维(轴)两端显示的项数(默认值为 3)
- `linewidth` – 用于插入行间隔的每行字符数(默认为 80)。Thresholded matrices will ignore

this parameter.

- profile – pretty 打印的完全默认值。 可以覆盖上述所有选项 (默认为 short, full)

创建操作 Creation Ops

torch.eye

```
torch.eye(n, m=None, out=None)
```

返回一个 2 维张量, 对角线位置全 1, 其它位置全 0

参数:

- n (**int**) – 行数
- m (**int**, optional) – 列数. 如果为 None, 则默认为 n
- out (**Tensor**, optional) - Output tensor

返回值: 对角线位置全 1, 其它位置全 0 的 2 维张量

返回值类型: **Tensor**

例子:

```
>>> torch.eye(3)
1  0  0
0  1  0
0  0  1
[torch.FloatTensor of size 3x3]
```

from_numpy

```
torch.from_numpy(ndarray) → Tensor
```

Numpy 桥, 将 `numpy.ndarray` 转换为 `pytorch` 的 `Tensor`。返回的张量 `tensor` 和 `numpy` 的 `ndarray` 共享同一内存空间。修改一个会导致另外一个也被修改。返回的张量不能改变大小。

例子:

```
>>> a = numpy.array([1, 2, 3])
>>> t = torch.from_numpy(a)
>>> t
torch.LongTensor([1, 2, 3])
>>> t[0] = -1
>>> a
array([-1,  2,  3])
```

torch.linspace

```
torch.linspace(start, end, steps=100, out=None) → Tensor
```

返回一个 1 维张量, 包含在区间 `start` 和 `end` 上均匀间隔的 `step` 个点。 输出 1 维张量的长度为

`steps`。

参数:

- start (float) – 序列的起始点
- end (float) – 序列的最终值
- steps (int) – 在 `start` 和 `end` 间生成的样本数
- out (Tensor, optional) – 结果张量

例子:

```
>>> torch.linspace(3, 10, steps=5)
```

```
3.0000
```

```
4.7500
```

```
6.5000
```

```
8.2500
```

```
10.0000
```

```
[torch.FloatTensor of size 5]
```

```
>>> torch.linspace(-10, 10, steps=5)
```

```
-10
```

```
-5
```

```
0
```

```
5
```

```
10
```

```
[torch.FloatTensor of size 5]
```

```
>>> torch.linspace(start=-10, end=10, steps=5)
```

```
-10
```

```
-5
```

```
0
```

```
5
```

```
10
```

```
[torch.FloatTensor of size 5]
```

torch.logspace

```
torch.logspace(start, end, steps=100, out=None) → Tensor
```

返回一个 1 维张量, 包含在区间 10^{start} 和 10^{end} 上以对数刻度均匀间隔的 `steps` 个点。输出 1 维张量的长度为 `steps`。

参数:

- `start` (float) – 序列的起始点
- `end` (float) – 序列的最终值
- `steps` (int) – 在 `start` 和 `end` 间生成的样本数
- `out` (Tensor, optional) – 结果张量

例子:

```
>>> torch.logspace(start=-10, end=10, steps=5)
```

```
1.0000e-10
```

```
1.0000e-05
```

```
1.0000e+00
```

```
1.0000e+05
```

```
1.0000e+10
```

```
[torch.FloatTensor of size 5]
```

```
>>> torch.logspace(start=0.1, end=1.0, steps=5)
```

```
1.2589
```

```
2.1135
```

```
3.5481
```

```
5.9566
```

```
10.0000
```

```
[torch.FloatTensor of size 5]
```

torch.ones

```
torch.ones(*sizes, out=None) → Tensor
```

返回一个全为 1 的张量，形状由可变参数 `sizes` 定义。

参数:

- `sizes (int...)` – 整数序列，定义了输出形状
- `out (Tensor, optional)` – 结果张量 例子:

```
>>> torch.ones(2, 3)
```

```
1 1 1
```

```
1 1 1
```

```
[torch.FloatTensor of size 2x3]
```

```
>>> torch.ones(5)
```

```
1
```

```
1
```

```
1
```

```
1
```

```
1
```

```
[torch.FloatTensor of size 5]
```

torch.rand

```
torch.rand(*sizes, out=None) → Tensor
```

返回一个张量，包含了从区间[0,1)的均匀分布中抽取的一组随机数，形状由可变参数 `sizes` 定义。

参数:

- `sizes (int...)` – 整数序列，定义了输出形状
- `out (Tensor, optional)` - 结果张量 例子:

```
>>> torch.rand(4)
```

```
0.9193
```

```
0.3347
```

```
0.3232
```

```
0.7715
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.rand(2, 3)
```

```
0.5010  0.5140  0.0719
```

```
0.1435  0.5636  0.0538
```

```
[torch.FloatTensor of size 2x3]
```

torch.randn

```
torch.randn(*sizes, out=None) → Tensor
```

返回一个张量，包含了从标准正态分布(均值为 0，方差为 1，即高斯白噪声)中抽取一组随机数，形

状由可变参数 `sizes` 定义。 参数:

- `sizes (int...)` – 整数序列，定义了输出形状
- `out (Tensor, optional)` - 结果张量

例子: :

```
>>> torch.randn(4)
```

```
-0.1145
```

```
0.0094
```

```
-1.1717
```

```
0.9846
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.randn(2, 3)
```

```
1.4339  0.3351 -1.0999
```

```
1.5458 -0.9643 -0.3558
```

```
[torch.FloatTensor of size 2x3]
```

torch.randperm

```
torch.randperm(n, out=None) → LongTensor
```

给定参数 `n`，返回一个从 0 到 `n-1` 的随机整数排列。

参数:

- `n (int)` – 上边界(不包含)

例子:

```
>>> torch.randperm(4)
```

```
2
```

```
1
```

```
3
```

```
0
```

```
[torch.LongTensor of size 4]
```

torch.arange

```
torch.arange(start, end, step=1, out=None) → Tensor
```

返回一个 1 维张量，长度为 $\text{floor}((\text{end}-\text{start})/\text{step})$ 。包含从 `start` 到 `end`，以 `step` 为步长的一组

序列值(默认步长为 1)。

参数:

- `start (float)` – 序列的起始点
- `end (float)` – 序列的终止点
- `step (float)` – 相邻点的间隔大小
- `out (Tensor, optional)` – 结果张量

例子:

```
>>> torch.arange(1, 4)
```

```
1
```

```
2
```

```
3
```

```
[torch.FloatTensor of size 3]
```

```
>>> torch.arange(1, 2.5, 0.5)
```

```
1.0000
```

```
1.5000
```

```
2.0000
```

```
[torch.FloatTensor of size 3]
```

torch.range

```
torch.range(start, end, step=1, out=None) → Tensor
```

返回一个 1 维张量，有 $\text{floor}((\text{end}-\text{start})/\text{step})+1$ 个元素。包含在半开区间 `[start, end)` 从 `start`

开始，以 `step` 为步长的一组值。`step` 是两个值之间的间隔，即 $x_{i+1}=x_i+\text{step}$

警告: 建议使用函数 `torch.arange()`

参数:

- `start (float)` – 序列的起始点
- `end (float)` – 序列的最终值
- `step (int)` – 相邻点的间隔大小
- `out (Tensor, optional)` – 结果张量

例子:

```
>>> torch.range(1, 4)
```

```
1
```

```
2
```

```
3
```

```
4
[torch.FloatTensor of size 4]

>>> torch.range(1, 4, 0.5)

1.0000
1.5000
2.0000
2.5000
3.0000
3.5000
4.0000
[torch.FloatTensor of size 7]
```

torch.zeros

`torch.zeros(*sizes, out=None) → Tensor`

返回一个全为标量 0 的张量，形状由可变参数 `sizes` 定义。

参数:

- `sizes (int...)` – 整数序列，定义了输出形状
- `out (Tensor, optional)` – 结果张量

例子:

```
>>> torch.zeros(2, 3)

0 0 0
0 0 0
[torch.FloatTensor of size 2x3]
```

```
>>> torch.zeros(5)

0
0
0
0
0
[torch.FloatTensor of size 5]
```

索引, 切片, 连接, 换位 Indexing, Slicing, Joining, Mutating Ops

torch.cat

`torch.cat(inputs, dimension=0) → Tensor`

在给定维度上对输入的张量序列 `seq` 进行连接操作。

`torch.cat()` 可以看做 `torch.split()` 和 `torch.chunk()` 的反操作。`cat()` 函数可以通过下

面例子更好的理解。

参数:

- **inputs (sequence of Tensors)** – 可以是任意相同 Tensor 类型的 python 序列
- **dimension (int, optional)** – 沿着此维连接张量序列。

例子:

```
>>> x = torch.randn(2, 3)
```

```
>>> x
```

```
0.5983 -0.0341 2.4918
```

```
1.5981 -0.5265 -0.8735
```

```
[torch.FloatTensor of size 2x3]
```

```
>>> torch.cat((x, x, x), 0)
```

```
0.5983 -0.0341 2.4918
```

```
1.5981 -0.5265 -0.8735
```

```
0.5983 -0.0341 2.4918
```

```
1.5981 -0.5265 -0.8735
```

```
0.5983 -0.0341 2.4918
```

```
1.5981 -0.5265 -0.8735
```

```
[torch.FloatTensor of size 6x3]
```

```
>>> torch.cat((x, x, x), 1)
```

```
0.5983 -0.0341 2.4918 0.5983 -0.0341 2.4918 0.5983 -0.0341
2.4918
```

```
1.5981 -0.5265 -0.8735 1.5981 -0.5265 -0.8735 1.5981 -0.5265
-0.8735
```

```
[torch.FloatTensor of size 2x9]
```

torch.chunk

```
torch.chunk(tensor, chunks, dim=0)
```

在给定维度(轴)上将输入张量进行分块儿。

参数:

- **tensor (Tensor)** – 待分块的输入张量
- **chunks (int)** – 分块的个数
- **dim (int)** – 沿着此维度进行分块

torch.gather

```
torch.gather(input, dim, index, out=None) → Tensor
```

沿给定轴 **dim**, 将输入索引张量 **index** 指定位置的值进行聚合。

对一个 3 维张量, 输出可以定义为:

```
out[i][j][k] = tensor[index[i][j][k]][j][k] # dim=0
```

```
out[i][j][k] = tensor[i][index[i][j][k]][k] # dim=1
```

```
out[i][j][k] = tensor[i][j][index[i][j][k]] # dim=3
```

例子:

```
>>> t = torch.Tensor([[1,2],[3,4]])
>>> torch.gather(t, 1, torch.LongTensor([[0,0],[1,0]]))
1 1
4 3
[torch.FloatTensor of size 2x2]
```

参数:

- input (Tensor) – 源张量
- dim (int) – 索引的轴
- index (LongTensor) – 聚合元素的下标
- out (Tensor, optional) – 目标张量

torch.index_select

torch.index_select(input, dim, index, out=None) → Tensor

沿着指定维度对输入进行切片, 取 `index` 中指定的相应项(`index` 为一个 LongTensor), 然后返回到

一个新的张量, 返回的张量与原始张量_Tensor_有相同的维度(在指定轴上)。

注意: 返回的张量不与原始张量共享内存空间。

参数:

- input (Tensor) – 输入张量
- dim (int) – 索引的轴
- index (LongTensor) – 包含索引下标的一维张量
- out (Tensor, optional) – 目标张量

例子:

```
>>> x = torch.randn(3, 4)
>>> x
```

```
1.2045  2.4084  0.4001  1.1372
0.5596  1.5677  0.6219 -0.7954
1.3635 -1.2313 -0.5414 -1.8478
[torch.FloatTensor of size 3x4]
```

```
>>> indices = torch.LongTensor([0, 2])
>>> torch.index_select(x, 0, indices)
```

```
1.2045  2.4084  0.4001  1.1372
1.3635 -1.2313 -0.5414 -1.8478
[torch.FloatTensor of size 2x4]
```

```
>>> torch.index_select(x, 1, indices)
```

```
1.2045  0.4001
0.5596  0.6219
1.3635 -0.5414
[torch.FloatTensor of size 3x2]
```

torch.masked_select

```
torch.masked_select(input, mask, out=None) → Tensor
```

根据掩码张量 `mask` 中的二元值，取输入张量中的指定项(`mask` 为一个 `ByteTensor`)，将取值返回到

一个新的 1D 张量，

张量 `mask` 须跟 `input` 张量有相同数量的元素数目，但形状或维度不需要相同。 注意：返回的张量

不与原始张量共享内存空间。

参数:

- `input (Tensor)` – 输入张量
- `mask (ByteTensor)` – 掩码张量，包含了二元索引值
- `out (Tensor, optional)` – 目标张量

例子:

```
>>> x = torch.randn(3, 4)
```

```
>>> x
```

```
1.2045  2.4084  0.4001  1.1372
0.5596  1.5677  0.6219 -0.7954
1.3635 -1.2313 -0.5414 -1.8478
[torch.FloatTensor of size 3x4]
```

```
>>> indices = torch.LongTensor([0, 2])
```

```
>>> torch.index_select(x, 0, indices)
```

```
1.2045  2.4084  0.4001  1.1372
1.3635 -1.2313 -0.5414 -1.8478
[torch.FloatTensor of size 2x4]
```

```
>>> torch.index_select(x, 1, indices)
```

```
1.2045  0.4001
0.5596  0.6219
1.3635 -0.5414
[torch.FloatTensor of size 3x2]
```

`torch.nonzero`

```
torch.nonzero(input, out=None) → LongTensor
```

返回一个包含输入 `input` 中非零元素索引的张量。输出张量中的每行包含输入中非零元素的索引。

如果输入 `input` 有 `n` 维，则输出的索引张量 `output` 的形状为 `z x n`，这里 `z` 是输入张量 `input` 中

所有非零元素的个数。

参数:

- `input (Tensor)` – 源张量
- `out (LongTensor, optional)` – 包含索引值的结果张量

例子:

```
>>> torch.nonzero(torch.Tensor([1, 1, 1, 0, 1]))
```

```
0
```

```
1
```

```
2
```

```
4
```

```
[torch.LongTensor of size 4x1]
```

```
>>> torch.nonzero(torch.Tensor([[0.6, 0.0, 0.0, 0.0],  
...                             [0.0, 0.4, 0.0, 0.0],  
...                             [0.0, 0.0, 1.2, 0.0],  
...                             [0.0, 0.0, 0.0, -0.4]]))
```

```
0 0
```

```
1 1
```

```
2 2
```

```
3 3
```

```
[torch.LongTensor of size 4x2]
```

torch.split

```
torch.split(tensor, split_size, dim=0)
```

将输入张量分割成相等形状的 **chunks**（如果可分）。如果沿指定维的张量形状大小不能被 **split_size** 整分，则最后一个分块会小于其它分块。

参数:

- **tensor (Tensor)** – 待分割张量
- **split_size (int)** – 单个分块的形状大小
- **dim (int)** – 沿着此维进行分割

torch.squeeze

```
torch.squeeze(input, dim=None, out=None)
```

将输入张量形状中的 1 去除并返回。如果输入是形如(A×1×B×1×C×1×D)，那么输出形状就为:

(A×B×C×D)

当给定 **dim** 时，那么挤压操作只在给定维度上。例如，输入形状为: (A×1×B), **squeeze(input, 0)** 将

会保持张量不变，只有用 **squeeze(input, 1)**，形状会变成 (A×B)。

注意: 返回张量与输入张量共享内存，所以改变其中一个的内容会改变另一个。

参数:

- **input (Tensor)** – 输入张量
- **dim (int, optional)** – 如果给定，则 **input** 只会在给定维度挤压
- **out (Tensor, optional)** – 输出张量

例子:

```
>>> x = torch.zeros(2,1,2,1,2)
```

```
>>> x.size()
```

```
(2L, 1L, 2L, 1L, 2L)
```

```
>>> y = torch.squeeze(x)
```

```
>>> y.size()
(2L, 2L, 2L)
>>> y = torch.squeeze(x, 0)
>>> y.size()
(2L, 1L, 2L, 1L, 2L)
>>> y = torch.squeeze(x, 1)
>>> y.size()
(2L, 2L, 1L, 2L)
```

torch.stack[source]

```
torch.stack(sequence, dim=0)
```

沿着一个新维度对输入张量序列进行连接。序列中所有的张量都应该为相同形状。

参数:

- **sequence (Sequence)** – 待连接的张量序列
- **dim (int)** – 插入的维度。必须介于 0 与 待连接的张量序列数之间。

torch.t

```
torch.t(input, out=None) → Tensor
```

输入一个矩阵（2 维张量），并转置 0,1 维。可以被视为函数 `transpose(input, 0, 1)` 的简写

函数。

参数:

- **input (Tensor)** – 输入张量
- **out (Tensor, optional)** – 结果张量

```
>>> x = torch.randn(2, 3)
>>> x
```

```
0.4834  0.6907  1.3417
-0.1300  0.5295  0.2321
[torch.FloatTensor of size 2x3]
```

```
>>> torch.t(x)
```

```
0.4834 -0.1300
0.6907  0.5295
1.3417  0.2321
[torch.FloatTensor of size 3x2]
```

torch.transpose

```
torch.transpose(input, dim0, dim1, out=None) → Tensor
```

返回输入矩阵 `input` 的转置。交换维度 `dim0` 和 `dim1`。输出张量与输入张量共享内存，所以改变

其中一个会导致另外一个也被修改。

参数:

- **input (Tensor)** – 输入张量
- **dim0 (int)** – 转置的第一维
- **dim1 (int)** – 转置的第二维

```
>>> x = torch.randn(2, 3)
```

```
>>> x
```

```
0.5983 -0.0341 2.4918
```

```
1.5981 -0.5265 -0.8735
```

```
[torch.FloatTensor of size 2x3]
```

```
>>> torch.transpose(x, 0, 1)
```

```
0.5983 1.5981
```

```
-0.0341 -0.5265
```

```
2.4918 -0.8735
```

```
[torch.FloatTensor of size 3x2]
```

torch.unbind

```
torch.unbind(tensor, dim=0) [source]
```

移除指定维后，返回一个元组，包含了沿着指定维切片后的各个切片

参数:

- **tensor (Tensor)** – 输入张量
- **dim (int)** – 删除的维度

torch.unsqueeze

```
torch.unsqueeze(input, dim, out=None)
```

返回一个新的张量，对输入的制定位置插入维度 1

注意： 返回张量与输入张量共享内存，所以改变其中一个的内容会改变另一个。

如果 **dim** 为负，则将会被转化 $\text{dim} + \text{input.dim()} + 1$

参数:

- **tensor (Tensor)** – 输入张量
- **dim (int)** – 插入维度的索引
- **out (Tensor, optional)** – 结果张量

```
>>> x = torch.Tensor([1, 2, 3, 4])
```

```
>>> torch.unsqueeze(x, 0)
```

```
1 2 3 4
```

```
[torch.FloatTensor of size 1x4]
```

```
>>> torch.unsqueeze(x, 1)
```

```
1
```

```
2
```

```
3
```

```
4
```

```
[torch.FloatTensor of size 4x1]
```

随机抽样 Random sampling

torch.manual_seed

```
torch.manual_seed(seed)
```

设定生成随机数的种子，并返回一个 `torch._C.Generator` 对象。

参数: `seed` (int or long) – 种子。

`torch.initial_seed`

`torch.initial_seed()`

返回生成随机数的原始种子值 (python long)。

`torch.get_rng_state`

`torch.get_rng_state()[source]`

返回随机生成器状态(`ByteTensor`)

`torch.set_rng_state`

`torch.set_rng_state(new_state)[source]`

设定随机生成器状态 参数: `new_state` (`torch.ByteTensor`) – 期望的状态

`torch.default_generator`

`torch.default_generator = <torch._C.Generator object>`

`torch.bernoulli`

`torch.bernoulli(input, out=None) → Tensor`

从伯努利分布中抽取二元随机数(0 或者 1)。

输入张量须包含用于抽取上述二元随机值的概率。 因此，输入中的所有值都必须在 `[0,1]` 区间，

即 $0 \leq \text{input}_i \leq 1$

输出张量的第 i 个元素值， 将会以输入张量的第 i 个概率值等于 1。

返回值将会是与输入相同大小的张量，每个值为 0 或者 1 参数:

- `input` (`Tensor`) – 输入为伯努利分布的概率值
- `out` (`Tensor`, optional) – 输出张量(可选)

例子:

```
>>> a = torch.Tensor(3, 3).uniform_(0, 1) # generate a uniform random matrix
```

```
with range [0, 1]
```

```
>>> a
```

```
0.7544 0.8140 0.9842
```

```
0.5282 0.0595 0.6445
```

```
0.1925 0.9553 0.9732
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> torch.bernoulli(a)
```

```
1 1 1
```

```
0 0 1
```

```
0 1 1
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> a = torch.ones(3, 3) # probability of drawing "1" is 1
```

```
>>> torch.bernoulli(a)
```

```
1 1 1
```

```
1 1 1
```

```
1 1 1
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> a = torch.zeros(3, 3) # probability of drawing "1" is 0
```

```
>>> torch.bernoulli(a)
```

```
0 0 0
```

```
0 0 0
```

```
0 0 0
```

```
[torch.FloatTensor of size 3x3]
```

```
torch.multinomial
```

```
torch.multinomial(input, num_samples, replacement=False, out=None) →
```

```
LongTensor
```

返回一个张量，每行包含从 `input` 相应行中定义的多项分布中抽取的 `num_samples` 个样本。

[注意]:输入 `input` 每行的值不需要总和为 1 (这里我们用来做权重)，但是必须非负且总和不能为 0。

当抽取样本时，依次从左到右排列(第一个样本对应第一列)。

如果输入 `input` 是一个向量，输出 `out` 也是一个相同长度 `num_samples` 的向量。如果输入 `input` 是有 `m` 行的矩阵，输出 `out` 是形如 `m×n` 的矩阵。

如果参数 `replacement` 为 `True`，则样本抽取可以重复。否则，一个样本在每行不能被重复抽取。

参数 `num_samples` 必须小于 `input` 长度(即，`input` 的列数，如果是 `input` 是一个矩阵)。

参数:

- `input (Tensor)` – 包含概率值的张量
- `num_samples (int)` – 抽取的样本数
- `replacement (bool, optional)` – 布尔值，决定是否能重复抽取
- `out (Tensor, optional)` – 结果张量

例子:

```
>>> weights = torch.Tensor([0, 10, 3, 0]) # create a Tensor of weights
```

```
>>> torch.multinomial(weights, 4)
```

```
1
```

```
2
```

```
0
```

```
0
```

```
[torch.LongTensor of size 4]
```

```
>>> torch.multinomial(weights, 4, replacement=True)
```

```
1
```

```
2
```

```
1
```

2

[torch.LongTensor of size 4]

torch.normal()

torch.normal(means, std, out=None)

返回一个张量，包含从给定参数 `means`, `std` 的离散正态分布中抽取随机数。均值 `means` 是一个张

量，包含每个输出元素相关的正态分布的均值。 `std` 是一个张量，包含每个输出元素相关的正态分

布的标准差。 均值和标准差的形状不须匹配，但每个张量的元素个数须相同。

参数:

- `means` (Tensor) – 均值
- `std` (Tensor) – 标准差
- `out` (Tensor) – 可选的输出张量

torch.normal(means=torch.arange(1, 11), std=torch.arange(1, 0, -0.1))

1.5104

1.6955

2.4895

4.9185

4.9895

6.9155

7.3683

8.1836

8.7164

9.8916

[torch.FloatTensor of size 10]

torch.normal(mean=0.0, std, out=None)

与上面函数类似，所有抽取的样本共享均值。

参数:

- `means` (Tensor, optional) – 所有分布均值
- `std` (Tensor) – 每个元素的标准差
- `out` (Tensor) – 可选的输出张量

例子:

>>> torch.normal(mean=0.5, std=torch.arange(1, 6))

0.5723

0.0871

-0.3783

-2.5689

10.7893

[torch.FloatTensor of size 5]

torch.normal(means, std=1.0, out=None)

与上面函数类似，所有抽取的样本共享标准差。

参数:

- means (Tensor) – 每个元素的均值
- std (float, optional) – 所有分布的标准差
- out (Tensor) – 可选的输出张量

例子:

```
>>> torch.normal(means=torch.arange(1, 6))
```

```
1.1681
```

```
2.8884
```

```
3.7718
```

```
2.5616
```

```
4.2500
```

```
[torch.FloatTensor of size 5]
```

序列化 Serialization

[torch.save](#)[\[source\]](#)

```
torch.save(obj, f, pickle_module=<module 'pickle' from  
/home/jenkins/miniconda/lib/python3.5/pickle.py>, pickle_protocol=2)
```

保存一个对象到一个硬盘文件上 参考: [Recommended approach for saving a model](#) 参数:

- obj – 保存对象
- f – 类文件对象 (返回文件描述符)或一个保存文件名的字符串
- pickle_module – 用于 pickling 元数据和对象的模块
- pickle_protocol – 指定 pickle protocol 可以覆盖默认参数

[torch.load](#)[\[source\]](#)

```
torch.load(f, map_location=None, pickle_module=<module 'pickle' from  
/home/jenkins/miniconda/lib/python3.5/pickle.py>)
```

从磁盘文件中读取一个通过 `torch.save()` 保存的对象。 `torch.load()` 可通过参数

`map_location` 动态地进行内存重映射, 使其能从不动设备中读取文件。一般调用时, 需两个参数: `storage` 和 `location tag`. 返回不同地址中的 `storage`, 或者返回 `None` (此时地址可以通过默认方法进行解析). 如果这个参数是字典的话, 意味着其是从文件的地址标记到当前系统的地址标记的映射。默认情况下, `location tags` 中 `"cpu"` 对应 host tensors, `'cuda:device_id'` (e.g. `'cuda:2'`) 对应 cuda tensors。用户可以通过 `register_package` 进行扩展, 使用自己定义的标记和反序列化方法。

参数:

- f – 类文件对象 (返回文件描述符)或一个保存文件名的字符串
- map_location – 一个函数或字典规定如何 remap 存储位置
- pickle_module – 用于 unpickling 元数据和对象的模块 (必须匹配序列化文件时的 `pickle_module`)

例子:

```
>>> torch.load('tensors.pt')
```

```
# Load all tensors onto the CPU
```

```
>>> torch.load('tensors.pt', map_location=lambda storage, loc: storage)
```

```
# Map tensors from GPU 1 to GPU 0
```

```
>>> torch.load('tensors.pt', map_location={'cuda:1': 'cuda:0'})
```

并行化 Parallelism

`torch.get_num_threads`

`torch.get_num_threads() → int`

获得用于并行化 CPU 操作的 OpenMP 线程数

`torch.set_num_threads`

`torch.set_num_threads(int)`

设定用于并行化 CPU 操作的 OpenMP 线程数

数学操作 Math operations

Pointwise Ops

`torch.abs`

`torch.abs(input, out=None) → Tensor`

计算输入张量的每个元素绝对值

例子:

```
>>> torch.abs(torch.FloatTensor([-1, -2, 3]))
```

```
FloatTensor([1, 2, 3])
```

`torch.acos(input, out=None) → Tensor`

`torch.acos(input, out=None) → Tensor`

返回一个新张量，包含输入张量每个元素的反余弦。 参数:

- `input` (Tensor) – 输入张量
- `out` (Tensor, optional) – 结果张量

例子:

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
-0.6366
```

```
0.2718
```

```
0.4469
```

```
1.3122
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.acos(a)
```

```
2.2608
```

```
1.2956
```

```
1.1075
```

```
nan
```

```
[torch.FloatTensor of size 4]
```

`torch.add()`

`torch.add(input, value, out=None)`

对输入张量 `input` 逐元素加上标量值 `value`，并返回结果到一个新的张量 `out`，

即 $\text{out}=\text{tensor}+\text{value}$ 。

如果输入 `input` 是 `FloatTensor` or `DoubleTensor` 类型，则 `value` 必须为实数，否则须为整数。【译注：似乎并非如此，无关输入类型，`value` 取整数、实数皆可。】

- `input (Tensor)` – 输入张量
- `value (Number)` – 添加到输入每个元素的数
- `out (Tensor, optional)` – 结果张量

```
>>> a = torch.randn(4)
>>> a

0.4050
-1.2227
1.8688
-0.4185
[torch.FloatTensor of size 4]
```

```
>>> torch.add(a, 20)

20.4050
18.7773
21.8688
19.5815
[torch.FloatTensor of size 4]
```

```
torch.add(input, value=1, other, out=None)
```

`other` 张量的每个元素乘以一个标量值 `value`，并加到 `input` 张量上。返回结果到输出张量 `out`。

即， $\text{out}=\text{input}+(\text{other}*\text{value})$

两个张量 `input` and `other` 的尺寸不需要匹配，但元素总数必须一样。

注意 :当两个张量形状不匹配时，输入张量的形状会作为输出张量的尺寸。

如果 `other` 是 `FloatTensor` or `DoubleTensor` 类型，则 `value` 必须为实数，否则须为整数。【译注：似乎并非如此，无关输入类型，`value` 取整数、实数皆可。】

参数:

- `input (Tensor)` – 第一个输入张量
- `value (Number)` – 用于第二个张量的尺寸因子
- `other (Tensor)` – 第二个输入张量
- `out (Tensor, optional)` – 结果张量

例子:

```
>>> import torch
>>> a = torch.randn(4)
>>> a

-0.9310
2.0330
0.0852
-0.2941
[torch.FloatTensor of size 4]
```

```
>>> b = torch.randn(2, 2)
```

```
>>> b
```

```
1.0663  0.2544
```

```
-0.1513  0.0749
```

```
[torch.FloatTensor of size 2x2]
```

```
>>> torch.add(a, 10, b)
```

```
9.7322
```

```
4.5770
```

```
-1.4279
```

```
0.4552
```

```
[torch.FloatTensor of size 4]
```

torch.addcddiv

`torch.addcddiv(tensor, value=1, tensor1, tensor2, out=None) → Tensor`

用 `tensor2` 对 `tensor1` 逐元素相除，然后乘以标量值 `value` 并加到 `tensor`。

张量的形状不需要匹配，但元素数量必须一致。

如果输入是 `FloatTensor` or `DoubleTensor` 类型，则 `value` 必须为实数，否则须为整数。

参数：

- `tensor` (`Tensor`) – 张量，对 `tensor1 ./ tensor` 进行相加
- `value` (`Number`, optional) – 标量，对 `tensor1 ./ tensor2` 进行相乘
- `tensor1` (`Tensor`) – 张量，作为被除数(分子)
- `tensor2` (`Tensor`) – 张量，作为除数(分母)
- `out` (`Tensor`, optional) – 输出张量

例子：

```
>>> t = torch.randn(2, 3)
```

```
>>> t1 = torch.randn(1, 6)
```

```
>>> t2 = torch.randn(6, 1)
```

```
>>> torch.addcddiv(t, 0.1, t1, t2)
```

```
0.0122 -0.0188 -0.2354
```

```
0.7396 -1.5721  1.2878
```

```
[torch.FloatTensor of size 2x3]
```

torch.addcmul

`torch.addcmul(tensor, value=1, tensor1, tensor2, out=None) → Tensor`

用 `tensor2` 对 `tensor1` 逐元素相乘，并对结果乘以标量值 `value` 然后加到 `tensor`。张量的形

状不需要匹配，但元素数量必须一致。如果输入是 `FloatTensor` or `DoubleTensor` 类型，则 `value` 必须为实数，否则须为整数。

参数：

- `tensor` (`Tensor`) – 张量，对 `tensor1 . tensor` 进行相加
- `value` (`Number`, optional) – 标量，对 `tensor1 . tensor2` 进行相乘
- `tensor1` (`Tensor`) – 张量，作为乘子 1
- `tensor2` (`Tensor`) – 张量，作为乘子 2

- out (Tensor, optional) – 输出张量

例子:

```
>>> t = torch.randn(2, 3)
>>> t1 = torch.randn(1, 6)
>>> t2 = torch.randn(6, 1)
>>> torch.addcmul(t, 0.1, t1, t2)
```

```
0.0122 -0.0188 -0.2354
```

```
0.7396 -1.5721 1.2878
```

```
[torch.FloatTensor of size 2x3]
```

torch.asin

torch.asin(input, out=None) → Tensor

返回一个新张量，包含输入 input 张量每个元素的反正弦函数

参数:

- tensor (Tensor) – 输入张量
- out (Tensor, optional) – 输出张量

例子:

```
>>> a = torch.randn(4)
>>> a
-0.6366
0.2718
0.4469
1.3122
[torch.FloatTensor of size 4]
```

```
>>> torch.asin(a)
```

```
-0.6900
```

```
0.2752
```

```
0.4633
```

```
nan
```

```
[torch.FloatTensor of size 4]
```

torch.atan

torch.atan(input, out=None) → Tensor

返回一个新张量，包含输入 input 张量每个元素的反正切函数

参数:

- tensor (Tensor) – 输入张量
- out (Tensor, optional) – 输出张量

例子:

```
>>> a = torch.randn(4)
>>> a
-0.6366
0.2718
0.4469
1.3122
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.atan(a)
```

```
-0.5669
```

```
0.2653
```

```
0.4203
```

```
0.9196
```

```
[torch.FloatTensor of size 4]
```

torch.atan2

```
torch.atan2(input1, input2, out=None) → Tensor
```

返回一个新张量，包含两个输入张量 `input1` 和 `input2` 的反正切函数

参数：

- `input1` (Tensor) – 第一个输入张量
- `input2` (Tensor) – 第二个输入张量
- `out` (Tensor, optional) – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
-0.6366
```

```
0.2718
```

```
0.4469
```

```
1.3122
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.atan2(a, torch.randn(4))
```

```
-2.4167
```

```
2.9755
```

```
0.9363
```

```
1.6613
```

```
[torch.FloatTensor of size 4]
```

torch.ceil

```
torch.ceil(input, out=None) → Tensor
```

天井函数，对输入 `input` 张量每个元素向上取整，即取不小于每个元素的最小整数，并返回结果到输出。

参数：

- `input` (Tensor) – 输入张量
- `out` (Tensor, optional) – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
1.3869
```

```
0.3912
```

```
-0.8634
```

```
-0.5468
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.ceil(a)
```

```
2
```

```
1
```

```
-0
```

```
-0
```

```
[torch.FloatTensor of size 4]
```

torch.clamp

torch.clamp(input, min, max, out=None) → Tensor

将输入 **input** 张量每个元素的夹紧到区间 **[min,max]**，并返回结果到一个新张量。

操作定义如下：

```
| min, if x_i < min
```

```
y_i = | x_i, if min <= x_i <= max
```

```
| max, if x_i > max
```

如果输入是 **FloatTensor** or **DoubleTensor** 类型，则参数 **min max** 必须为实数，否则须为整数。【译注：似乎并非如此，无关输入类型，**min**，**max** 取整数、实数皆可。】

参数：

- **input (Tensor)** – 输入张量
- **min (Number)** – 限制范围下限
- **max (Number)** – 限制范围上限
- **out (Tensor, optional)** – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
1.3869
```

```
0.3912
```

```
-0.8634
```

```
-0.5468
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.clamp(a, min=-0.5, max=0.5)
```

```
0.5000
```

```
0.3912
```

```
-0.5000
```

```
-0.5000
```

```
[torch.FloatTensor of size 4]
```

torch.clamp(input, *, min, out=None) → Tensor

将输入 **input** 张量每个元素的限制到不小于 **min**，并返回结果到一个新张量。

如果输入是 **FloatTensor** or **DoubleTensor** 类型，则参数 **min** 必须为实数，否则须为整数。【译注：

似乎并非如此，无关输入类型，`min` 取整数、实数皆可。】

参数：

- `input (Tensor)` – 输入张量
- `value (Number)` – 限制范围下限
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(4)
>>> a

1.3869
0.3912
-0.8634
-0.5468
[torch.FloatTensor of size 4]
```

```
>>> torch.clamp(a, min=0.5)
```

```
1.3869
0.5000
0.5000
0.5000
[torch.FloatTensor of size 4]
```

```
torch.clamp(input, *, max, out=None) → Tensor
```

将输入 `input` 张量每个元素的限制到不大于 `max`，并返回结果到一个新张量。

如果输入是 `FloatTensor` or `DoubleTensor` 类型，则参数 `max` 必须为实数，否则须为整数。【译注：

似乎并非如此，无关输入类型，`max` 取整数、实数皆可。】

参数：

- `input (Tensor)` – 输入张量
- `value (Number)` – 限制范围上限
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(4)
>>> a

1.3869
0.3912
-0.8634
-0.5468
[torch.FloatTensor of size 4]
```

```
>>> torch.clamp(a, max=0.5)
```

```
0.5000
0.3912
```

```
-0.8634
```

```
-0.5468
```

```
[torch.FloatTensor of size 4]
```

torch.cos

`torch.cos(input, out=None) → Tensor`

返回一个新张量，包含输入 `input` 张量每个元素的余弦。

参数：

- `input` (Tensor) – 输入张量
- `out` (Tensor, optional) – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
-0.6366
```

```
0.2718
```

```
0.4469
```

```
1.3122
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.cos(a)
```

```
0.8041
```

```
0.9633
```

```
0.9018
```

```
0.2557
```

```
[torch.FloatTensor of size 4]
```

torch.cosh

`torch.cosh(input, out=None) → Tensor`

返回一个新张量，包含输入 `input` 张量每个元素的双曲余弦。

参数：

- `input` (Tensor) – 输入张量
- `out` (Tensor, optional) – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
-0.6366
```

```
0.2718
```

```
0.4469
```

```
1.3122
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.cosh(a)
```

```
1.2095
```

```
1.0372
```

```
1.1015
```

```
1.9917
```

```
[torch.FloatTensor of size 4]
```

```
torch.div()
```

```
torch.div(input, value, out=None)
```

将 `input` 逐元素除以标量值 `value`，并返回结果到输出张量 `out`。

即 $\text{out} = \text{tensor} / \text{value}$

如果输入是 `FloatTensor` or `DoubleTensor` 类型，则参数 `value` 必须为实数，否则须为整数。【译注：似乎并非如此，无关输入类型，`value` 取整数、实数皆可。】

参数：

- `input (Tensor)` – 输入张量
- `value (Number)` – 除数
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(5)
```

```
>>> a
```

```
-0.6147
```

```
-1.1237
```

```
-0.1604
```

```
-0.6853
```

```
0.1063
```

```
[torch.FloatTensor of size 5]
```

```
>>> torch.div(a, 0.5)
```

```
-1.2294
```

```
-2.2474
```

```
-0.3208
```

```
-1.3706
```

```
0.2126
```

```
[torch.FloatTensor of size 5]
```

```
torch.div(input, other, out=None)
```

两张量 `input` 和 `other` 逐元素相除，并将结果返回到输出。

即， $\text{out}_i = \text{input}_i / \text{other}_i$

两张量形状不须匹配，但元素数须一致。

注意：当形状不匹配时，`input` 的形状作为输出张量的形状。

参数：

- `input (Tensor)` – 张量(分子)
- `other (Tensor)` – 张量(分母)
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(4,4)
```

```
>>> a
```

```
-0.1810  0.4017  0.2863 -0.1013
```

```
0.6183  2.0696  0.9012 -1.5933
0.5679  0.4743 -0.0117 -0.1266
-0.1213  0.9629  0.2682  1.5968
[torch.FloatTensor of size 4x4]
```

```
>>> b = torch.randn(8, 2)
>>> b
```

```
0.8774  0.7650
0.8866  1.4805
-0.6490  1.1172
1.4259 -0.8146
1.4633 -0.1228
0.4643 -0.6029
0.3492  1.5270
1.6103 -0.6291
[torch.FloatTensor of size 8x2]
```

```
>>> torch.div(a, b)
```

```
-0.2062  0.5251  0.3229 -0.0684
-0.9528  1.8525  0.6320  1.9559
0.3881 -3.8625 -0.0253  0.2099
-0.3473  0.6306  0.1666 -2.5381
[torch.FloatTensor of size 4x4]
```

torch.exp

`torch.exp(tensor, out=None) → Tensor`

返回一个新张量，包含输入 `input` 张量每个元素的指数。

参数：

- `input (Tensor)` – 输入张量
- `out (Tensor, optional)` – 输出张量

```
>>> torch.exp(torch.Tensor([0, math.log(2)]))
torch.FloatTensor([1, 2])
```

torch.floor

`torch.floor(input, out=None) → Tensor`

床函数：返回一个新张量，包含输入 `input` 张量每个元素的 `floor`，即不小于元素的最大整数。

参数：

- `input (Tensor)` – 输入张量
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(4)
>>> a
```

```
1.3869
```

```
0.3912
```

```
-0.8634
```

```
-0.5468
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.floor(a)
```

```
1
```

```
0
```

```
-1
```

```
-1
```

```
[torch.FloatTensor of size 4]
```

torch.fmod

```
torch.fmod(input, divisor, out=None) → Tensor
```

计算除法余数。除数与被除数可能同时含有整数和浮点数。此时，余数的正负与被除数相同。

参数： - input (Tensor) – 被除数 - divisor (Tensor or float) – 除数，一个数或与被除数相同类型的张量 - out (Tensor, optional) – 输出张量

例子：

```
>>> torch.fmod(torch.Tensor([-3, -2, -1, 1, 2, 3]), 2)
```

```
torch.FloatTensor([-1, -0, -1, 1, 0, 1])
```

```
>>> torch.fmod(torch.Tensor([1, 2, 3, 4, 5]), 1.5)
```

```
torch.FloatTensor([1.0, 0.5, 0.0, 1.0, 0.5])
```

参考: [torch.remainder\(\)](#), 计算逐元素余数，相当于 python 中的 % 操作符。

torch.frac

```
torch.frac(tensor, out=None) → Tensor
```

返回每个元素的分数部分。

例子：

```
>>> torch.frac(torch.Tensor([1, 2.5, -3.2]))
```

```
torch.FloatTensor([0, 0.5, -0.2])
```

torch.lerp

```
torch.lerp(start, end, weight, out=None)
```

对两个张量以 start, end 做线性插值，将结果返回到输出张量。

即， $out_i = start_i + weight * (end_i - start_i)$

参数：

- start (Tensor) – 起始点张量
- end (Tensor) – 终止点张量
- weight (float) – 插值公式的 weight
- out (Tensor, optional) – 结果张量

例子：

```
>>> start = torch.arange(1, 5)
```

```
>>> end = torch.Tensor(4).fill_(10)
```

```
>>> start
```

```
1
```

```
2
3
4
[torch.FloatTensor of size 4]
```

```
>>> end
```

```
10
10
10
10
[torch.FloatTensor of size 4]
```

```
>>> torch.lerp(start, end, 0.5)
```

```
5.5000
6.0000
6.5000
7.0000
[torch.FloatTensor of size 4]
```

torch.log

torch.log(input, out=None) → Tensor

计算 input 的自然对数

参数:

- input (Tensor) – 输入张量
- out (Tensor, optional) – 输出张量

例子:

```
>>> a = torch.randn(5)
>>> a

-0.4183
 0.3722
-0.3091
 0.4149
 0.5857
[torch.FloatTensor of size 5]
```

```
>>> torch.log(a)
```

```
nan
-0.9883
nan
-0.8797
```

```
-0.5349
```

```
[torch.FloatTensor of size 5]
```

torch.log1p

```
torch.log1p(input, out=None) → Tensor
```

计算 $\text{input}+1$ 的自然对数 $y_i=\log(x_i+1)$

注意：对值比较小的输入，此函数比 `torch.log()` 更准确。

如果输入是 `FloatTensor` or `DoubleTensor` 类型，则 `value` 必须为实数，否则须为整数。

参数：

- `input (Tensor)` – 输入张量
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(5)
```

```
>>> a
```

```
-0.4183
```

```
0.3722
```

```
-0.3091
```

```
0.4149
```

```
0.5857
```

```
[torch.FloatTensor of size 5]
```

```
>>> torch.log1p(a)
```

```
-0.5418
```

```
0.3164
```

```
-0.3697
```

```
0.3471
```

```
0.4611
```

```
[torch.FloatTensor of size 5]
```

torch.mul

```
torch.mul(input, value, out=None)
```

用标量值 `value` 乘以输入 `input` 的每个元素，并返回一个新的结果张量。 `out=tensor*value`

如果输入是 `FloatTensor` or `DoubleTensor` 类型，则 `value` 必须为实数，否则须为整数。【译注：似乎并非如此，无关输入类型，`value` 取整数、实数皆可。】

参数：

- `input (Tensor)` – 输入张量
- `value (Number)` – 乘到每个元素的数
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(3)
```

```
>>> a
```

```
-0.9374
```

```
-0.5254
```

```
-0.6069
```

```
[torch.FloatTensor of size 3]
```

```
>>> torch.mul(a, 100)
```

```
-93.7411
```

```
-52.5374
```

```
-60.6908
```

```
[torch.FloatTensor of size 3]
```

```
torch.mul(input, other, out=None)
```

两个张量 `input`, `other` 按元素进行相乘，并返回到输出张量。即计算 $out_i = input_i * other_i$ ，两计算张量形状不须匹配，但总元素数须一致。**注意：**当形状不匹配时，`input` 的形状作为输入张量的形状。

参数：

- `input` (Tensor) – 第一个相乘张量
- `other` (Tensor) – 第二个相乘张量
- `out` (Tensor, optional) – 结果张量

例子：

```
>>> a = torch.randn(4,4)
```

```
>>> a
```

```
-0.7280  0.0598 -1.4327 -0.5825
```

```
-0.1427 -0.0690  0.0821 -0.3270
```

```
-0.9241  0.5110  0.4070 -1.1188
```

```
-0.8308  0.7426 -0.6240 -1.1582
```

```
[torch.FloatTensor of size 4x4]
```

```
>>> b = torch.randn(2, 8)
```

```
>>> b
```

```
 0.0430 -1.0775  0.6015  1.1647 -0.6549  0.0308 -0.1670  1.0742
```

```
-1.2593  0.0292 -0.0849  0.4530  1.2404 -0.4659 -0.1840  0.5974
```

```
[torch.FloatTensor of size 2x8]
```

```
>>> torch.mul(a, b)
```

```
-0.0313 -0.0645 -0.8618 -0.6784
```

```
 0.0934 -0.0021 -0.0137 -0.3513
```

```
 1.1638  0.0149 -0.0346 -0.5068
```

```
-1.0304 -0.3460  0.1148 -0.6919
```

```
[torch.FloatTensor of size 4x4]
```

`torch.neg`

```
torch.neg(input, out=None) → Tensor
```

返回一个新张量，包含输入 `input` 张量按元素取负。即， $out = -1 * input$

参数:

- input (Tensor) – 输入张量
- out (Tensor, optional) – 输出张量

例子:

```
>>> a = torch.randn(5)
```

```
>>> a
```

```
-0.4430
```

```
1.1690
```

```
-0.8836
```

```
-0.4565
```

```
0.2968
```

```
[torch.FloatTensor of size 5]
```

```
>>> torch.neg(a)
```

```
0.4430
```

```
-1.1690
```

```
0.8836
```

```
0.4565
```

```
-0.2968
```

```
[torch.FloatTensor of size 5]
```

torch.pow

torch.pow(input, exponent, out=None)

对输入 **input** 的按元素求 **exponent** 次幂值，并返回结果张量。 幂值 **exponent** 可以为单一 **float** 数或者与 **input** 相同元素数的张量。

当幂值为标量时，执行操作：

$$\text{out}_i = x^{\text{exponent}}$$

当幂值为张量时，执行操作：

$$\text{out}_i = x^{\text{exponent}_i}$$

参数:

- input (Tensor) – 输入张量
- exponent (float or Tensor) – 幂值
- out (Tensor, optional) – 输出张量

例子:

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
-0.5274
```

```
-0.8232
```

```
-2.1128
```

```
1.7558
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.pow(a, 2)
```

```
0.2781
```

```
0.6776
```

```
4.4640
```

```
3.0829
```

```
[torch.FloatTensor of size 4]
```

```
>>> exp = torch.arange(1, 5)
```

```
>>> a = torch.arange(1, 5)
```

```
>>> a
```

```
1
```

```
2
```

```
3
```

```
4
```

```
[torch.FloatTensor of size 4]
```

```
>>> exp
```

```
1
```

```
2
```

```
3
```

```
4
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.pow(a, exp)
```

```
1
```

```
4
```

```
27
```

```
256
```

```
[torch.FloatTensor of size 4]
```

```
torch.pow(base, input, out=None)
```

base 为标量浮点值, **input** 为张量, 返回的输出张量 **out** 与输入张量相同形状。

执行操作为:

$$\text{out}_i = \text{base}^{\text{input}_i}$$

参数:

- **base** (float) – 标量值, 指数的底
- **input** (Tensor) – 幂值
- **out** (Tensor, optional) – 输出张量

例子:

```
>>> exp = torch.arange(1, 5)
```

```
>>> base = 2
```

```
>>> torch.pow(base, exp)
```

```
2
```

```
4
```

```
8
```

```
16
```

```
[torch.FloatTensor of size 4]
```

torch.reciprocal

```
torch.reciprocal(input, out=None) → Tensor
```

返回一个新张量，包含输入 `input` 张量每个元素的倒数，即 $1.0/x$ 。

参数：

- `input` (Tensor) – 输入张量
- `out` (Tensor, optional) – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
1.3869
```

```
0.3912
```

```
-0.8634
```

```
-0.5468
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.reciprocal(a)
```

```
0.7210
```

```
2.5565
```

```
-1.1583
```

```
-1.8289
```

```
[torch.FloatTensor of size 4]
```

torch.remainder

```
torch.remainder(input, divisor, out=None) → Tensor
```

返回一个新张量，包含输入 `input` 张量每个元素的除法余数。除数与被除数可能同时包含整数或浮点数。余数与除数有相同的符号。

参数：

- `input` (Tensor) – 被除数
- `divisor` (Tensor or float) – 除数，一个数或者与除数相同大小的张量
- `out` (Tensor, optional) – 输出张量

例子：

```
>>> torch.remainder(torch.Tensor([-3, -2, -1, 1, 2, 3]), 2)
```

```
torch.FloatTensor([1, 0, 1, 1, 0, 1])
```

```
>>> torch.remainder(torch.Tensor([1, 2, 3, 4, 5]), 1.5)
```

```
torch.FloatTensor([1.0, 0.5, 0.0, 1.0, 0.5])
```

参考：函数 `torch.fmod()` 同样可以计算除法余数，相当于 C 的 库函数 `fmod()`

torch.round

`torch.round(input, out=None) → Tensor`

返回一个新张量，将输入 `input` 张量每个元素舍入到最近的整数。

参数：

- `input (Tensor)` – 输入张量
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
1.2290
```

```
1.3409
```

```
-0.5662
```

```
-0.0899
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.round(a)
```

```
1
```

```
1
```

```
-1
```

```
-0
```

```
[torch.FloatTensor of size 4]
```

torch.rsqrt

`torch.rsqrt(input, out=None) → Tensor`

返回一个新张量，包含输入 `input` 张量每个元素的平方根倒数。

参数：

- `input (Tensor)` – 输入张量
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
1.2290
```

```
1.3409
```

```
-0.5662
```

```
-0.0899
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.rsqrt(a)
```

```
0.9020
```

```
0.8636
```

```
nan
```

```
nan
[torch.FloatTensor of size 4]
```

torch.sigmoid

`torch.sigmoid(input, out=None) → Tensor`

返回一个新张量，包含输入 `input` 张量每个元素的 sigmoid 值。

参数：

- `input (Tensor)` – 输入张量
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(4)
>>> a

-0.4972
 1.3512
 0.1056
-0.2650
[torch.FloatTensor of size 4]
```

```
>>> torch.sigmoid(a)
```

```
0.3782
0.7943
0.5264
0.4341
[torch.FloatTensor of size 4]
```

torch.sign

`torch.sign(input, out=None) → Tensor`

符号函数：返回一个新张量，包含输入 `input` 张量每个元素的正负。

参数：

- `input (Tensor)` – 输入张量
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(4)
>>> a

-0.6366
 0.2718
 0.4469
 1.3122
[torch.FloatTensor of size 4]
```

```
>>> torch.sign(a)
```

```
-1
 1
```

```
1
```

```
1
```

```
[torch.FloatTensor of size 4]
```

torch.sin

torch.sin(input, out=None) → Tensor

返回一个新张量，包含输入 **input** 张量每个元素的正弦。

参数：

- input (Tensor) – 输入张量
- out (Tensor, optional) – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
-0.6366
```

```
0.2718
```

```
0.4469
```

```
1.3122
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.sin(a)
```

```
-0.5944
```

```
0.2684
```

```
0.4322
```

```
0.9667
```

```
[torch.FloatTensor of size 4]
```

torch.sinh

torch.sinh(input, out=None) → Tensor

返回一个新张量，包含输入 **input** 张量每个元素的双曲正弦。

参数：

- input (Tensor) – 输入张量
- out (Tensor, optional) – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
-0.6366
```

```
0.2718
```

```
0.4469
```

```
1.3122
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.sinh(a)
```

```
-0.6804
```

```
0.2751
```

```
0.4619
```

```
1.7225
```

```
[torch.FloatTensor of size 4]
```

torch.sqrt

```
torch.sqrt(input, out=None) → Tensor
```

返回一个新张量，包含输入 `input` 张量每个元素的平方根。

参数：

- `input` (Tensor) – 输入张量
- `out` (Tensor, optional) – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
1.2290
```

```
1.3409
```

```
-0.5662
```

```
-0.0899
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.sqrt(a)
```

```
1.1086
```

```
1.1580
```

```
nan
```

```
nan
```

```
[torch.FloatTensor of size 4]
```

torch.tan

```
torch.tan(input, out=None) → Tensor
```

返回一个新张量，包含输入 `input` 张量每个元素的正切。

参数：

- `input` (Tensor) – 输入张量
- `out` (Tensor, optional) – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
-0.6366
```

```
0.2718
```

```
0.4469
```

```
1.3122
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.tan(a)
```

```
-0.7392
```

```
0.2786
```

```
0.4792
```

```
3.7801
```

```
[torch.FloatTensor of size 4]
```

torch.tanh

torch.tanh(input, out=None) → Tensor

返回一个新张量，包含输入 `input` 张量每个元素的双曲正切。

参数：

- `input (Tensor)` – 输入张量
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
-0.6366
```

```
0.2718
```

```
0.4469
```

```
1.3122
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.tanh(a)
```

```
-0.5625
```

```
0.2653
```

```
0.4193
```

```
0.8648
```

```
[torch.FloatTensor of size 4]
```

torch.trunc

torch.trunc(input, out=None) → Tensor

返回一个新张量，包含输入 `input` 张量每个元素的截断值(标量 `x` 的截断值是最接近其的整数，其比 `x` 更接近零。简而言之，有符号数的小数部分被舍弃)。

参数：

- `input (Tensor)` – 输入张量
- `out (Tensor, optional)` – 输出张量

例子：

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
-0.4972
```

```
1.3512
```

```
0.1056
```

```
-0.2650
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.trunc(a)
```

```
-0
```

```
1
```

```
0
```

```
-0
```

```
[torch.FloatTensor of size 4]
```

Reduction Ops

`torch.cumprod`

```
torch.cumprod(input, dim, out=None) → Tensor
```

返回输入沿指定维度的累积积。例如，如果输入是一个 N 元向量，则结果也是一个 N 元向量，第 i 个输出元素值为 $y_i = x_1 * x_2 * x_3 * \dots * x_i$

参数：

- `input` (Tensor) – 输入张量
- `dim` (int) – 累积积操作的维度
- `out` (Tensor, optional) – 结果张量

例子：

```
>>> a = torch.randn(10)
```

```
>>> a
```

```
1.1148
```

```
1.8423
```

```
1.4143
```

```
-0.4403
```

```
1.2859
```

```
-1.2514
```

```
-0.4748
```

```
1.1735
```

```
-1.6332
```

```
-0.4272
```

```
[torch.FloatTensor of size 10]
```

```
>>> torch.cumprod(a, dim=0)
```

```
1.1148
```

```
2.0537
```

```
2.9045
```

```
-1.2788
```

```
-1.6444
```

```
2.0578
```

```
-0.9770
```

```
-1.1466
```

```
1.8726
```

```
-0.8000
```

```
[torch.FloatTensor of size 10]
```

```
>>> a[5] = 0.0
```

```
>>> torch.cumprod(a, dim=0)
```

```
1.1148
2.0537
2.9045
-1.2788
-1.6444
-0.0000
0.0000
0.0000
-0.0000
0.0000
```

```
[torch.FloatTensor of size 10]
```

torch.cumsum

torch.cumsum(input, dim, out=None) → Tensor

返回输入沿指定维度的累积和。例如，如果输入是一个 N 元向量，则结果也是一个 N 元向量，第 i 个输出元素值为 $y_i = x_1 + x_2 + x_3 + \dots + x_i$

参数：

- input (Tensor) – 输入张量
- dim (int) – 累积和操作的维度
- out (Tensor, optional) – 结果张量

例子：

```
>>> a = torch.randn(10)
```

```
>>> a
```

```
-0.6039
-0.2214
-0.3705
-0.0169
1.3415
-0.1230
0.9719
0.6081
-0.1286
1.0947
```

```
[torch.FloatTensor of size 10]
```

```
>>> torch.cumsum(a, dim=0)
```

```
-0.6039
-0.8253
-1.1958
-1.2127
0.1288
```

```
0.0058
```

```
0.9777
```

```
1.5858
```

```
1.4572
```

```
2.5519
```

```
[torch.FloatTensor of size 10]
```

torch.dist

`torch.dist(input, other, p=2, out=None) → Tensor`

返回 $(input - other)$ 的 p 范数。

参数：

- `input` (Tensor) – 输入张量
- `other` (Tensor) – 右侧输入张量
- `p` (float, optional) – 所计算的范数
- `out` (Tensor, optional) – 结果张量

例子：

```
>>> x = torch.randn(4)
```

```
>>> x
```

```
0.2505
```

```
-0.4571
```

```
-0.3733
```

```
0.7807
```

```
[torch.FloatTensor of size 4]
```

```
>>> y = torch.randn(4)
```

```
>>> y
```

```
0.7782
```

```
-0.5185
```

```
1.4106
```

```
-2.4063
```

```
[torch.FloatTensor of size 4]
```

```
>>> torch.dist(x, y, 3.5)
```

```
3.302832063224223
```

```
>>> torch.dist(x, y, 3)
```

```
3.3677282206393286
```

```
>>> torch.dist(x, y, 0)
```

```
inf
```

```
>>> torch.dist(x, y, 1)
```

```
5.560028076171875
```

torch.mean

`torch.mean(input) → float`

返回输入张量所有元素的均值。

参数: input (Tensor) – 输入张量

例子:

```
>>> a = torch.randn(1, 3)
```

```
>>> a
```

```
-0.2946 -0.9143  2.1809
```

```
[torch.FloatTensor of size 1x3]
```

```
>>> torch.mean(a)
```

```
0.32398951053619385
```

```
torch.mean(input, dim, out=None) → Tensor
```

返回输入张量给定维度 `dim` 上每行的均值。

输出形状与输入相同，除了给定维度上为 1。

参数:

- input (Tensor) – 输入张量
- dim (int) – the dimension to reduce
- out (Tensor, optional) – 结果张量

例子:

```
>>> a = torch.randn(4, 4)
```

```
>>> a
```

```
-1.2738 -0.3058  0.1230 -1.9615
```

```
0.8771 -0.5430 -0.9233  0.9879
```

```
1.4107  0.0317 -0.6823  0.2255
```

```
-1.3854  0.4953 -0.2160  0.2435
```

```
[torch.FloatTensor of size 4x4]
```

```
>>> torch.mean(a, 1)
```

```
-0.8545
```

```
0.0997
```

```
0.2464
```

```
-0.2157
```

```
[torch.FloatTensor of size 4x1]
```

torch.median

```
torch.median(input, dim=-1, values=None, indices=None) -> (Tensor,
```

```
LongTensor)
```

返回输入张量给定维度每行的中位数，同时返回一个包含中位数的索引的 `LongTensor`。

`dim` 值默认为输入张量的最后一维。输出形状与输入相同，除了给定维度上为 1。

注意: 这个函数还没有在 `torch.cuda.Tensor` 中定义

参数:

- input (Tensor) – 输入张量
- dim (int) – 缩减的维度

- values (Tensor, optional) – 结果张量
- indices (Tensor, optional) – 返回的索引结果张量

```
>>> a
```

```
-0.6891 -0.6662
```

```
0.2697 0.7412
```

```
0.5254 -0.7402
```

```
0.5528 -0.2399
```

```
[torch.FloatTensor of size 4x2]
```

```
>>> a = torch.randn(4, 5)
```

```
>>> a
```

```
0.4056 -0.3372 1.0973 -2.4884 0.4334
```

```
2.1336 0.3841 0.1404 -0.1821 -0.7646
```

```
-0.2403 1.3975 -2.0068 0.1298 0.0212
```

```
-1.5371 -0.7257 -0.4871 -0.2359 -1.1724
```

```
[torch.FloatTensor of size 4x5]
```

```
>>> torch.median(a, 1)
```

```
(
```

```
0.4056
```

```
0.1404
```

```
0.0212
```

```
-0.7257
```

```
[torch.FloatTensor of size 4x1]
```

```
,
```

```
0
```

```
2
```

```
4
```

```
1
```

```
[torch.LongTensor of size 4x1]
```

```
)
```

torch.mode

```
torch.mode(input, dim=-1, values=None, indices=None) -> (Tensor, LongTensor)
```

返回给定维 `dim` 上，每行的众数值。 同时返回一个 `LongTensor`，包含众数值的索引。 `dim` 值默认为输入张量的最后一维。

输出形状与输入相同，除了给定维度上为 1。

注意：这个函数还没有在 `torch.cuda.Tensor` 中定义

参数：

- input (Tensor) – 输入张量
- dim (int) – 缩减的维度
- values (Tensor, optional) – 结果张量
- indices (Tensor, optional) – 返回的索引张量

例子:

```
>>> a
```

```
-0.6891 -0.6662
```

```
0.2697  0.7412
```

```
0.5254 -0.7402
```

```
0.5528 -0.2399
```

```
[torch.FloatTensor of size 4x2]
```

```
>>> a = torch.randn(4, 5)
```

```
>>> a
```

```
0.4056 -0.3372  1.0973 -2.4884  0.4334
```

```
2.1336  0.3841  0.1404 -0.1821 -0.7646
```

```
-0.2403  1.3975 -2.0068  0.1298  0.0212
```

```
-1.5371 -0.7257 -0.4871 -0.2359 -1.1724
```

```
[torch.FloatTensor of size 4x5]
```

```
>>> torch.mode(a, 1)
```

```
(
```

```
-2.4884
```

```
-0.7646
```

```
-2.0068
```

```
-1.5371
```

```
[torch.FloatTensor of size 4x1]
```

```
,
```

```
3
```

```
4
```

```
2
```

```
0
```

```
[torch.LongTensor of size 4x1]
```

```
)
```

torch.norm

`torch.norm(input, p=2) → float`

返回输入张量 `input` 的 `p` 范数。

参数:

- `input (Tensor)` – 输入张量
- `p (float, optional)` – 范数计算中的幂指数值

例子:

```
>>> a = torch.randn(1, 3)
```

```
>>> a
```

```
-0.4376 -0.5328  0.9547
```

```
[torch.FloatTensor of size 1x3]
```



```
>>> torch.norm(a, 3)
```

```
1.0338925067372466
```

```
torch.norm(input, p, dim, out=None) → Tensor
```

返回输入张量给定维 `dim` 上每行的 `p` 范数。输出形状与输入相同，除了给定维度上为 1。

参数：

- `input (Tensor)` – 输入张量
- `p (float)` – 范数计算中的幂指数值
- `dim (int)` – 缩减的维度
- `out (Tensor, optional)` – 结果张量

例子：

```
>>> a = torch.randn(4, 2)
```

```
>>> a
```

```
-0.6891 -0.6662
```

```
0.2697 0.7412
```

```
0.5254 -0.7402
```

```
0.5528 -0.2399
```

```
[torch.FloatTensor of size 4x2]
```

```
>>> torch.norm(a, 2, 1)
```

```
0.9585
```

```
0.7888
```

```
0.9077
```

```
0.6026
```

```
[torch.FloatTensor of size 4x1]
```

```
>>> torch.norm(a, 0, 1)
```

```
2
```

```
2
```

```
2
```

```
2
```

```
[torch.FloatTensor of size 4x1]
```

`torch.prod`

```
torch.prod(input) → float
```

返回输入张量 `input` 所有元素的积。

参数： `input (Tensor)` – 输入张量

例子：

```
>>> a = torch.randn(1, 3)
```

```
>>> a
```

```
0.6170 0.3546 0.0253
```

```
[torch.FloatTensor of size 1x3]
```

```
>>> torch.prod(a)
```

```
0.005537458061418483
```

```
torch.prod(input, dim, out=None) → Tensor
```

返回输入张量给定维度上每行的积。 输出形状与输入相同，除了给定维度上为 1。

参数：

- input (Tensor) – 输入张量
- dim (int) – 缩减的维度
- out (Tensor, optional) – 结果张量

例子：

```
>>> a = torch.randn(4, 2)
```

```
>>> a
```

```
0.1598 -0.6884
```

```
-0.1831 -0.4412
```

```
-0.9925 -0.6244
```

```
-0.2416 -0.8080
```

```
[torch.FloatTensor of size 4x2]
```

```
>>> torch.prod(a, 1)
```

```
-0.1100
```

```
0.0808
```

```
0.6197
```

```
0.1952
```

```
[torch.FloatTensor of size 4x1]
```

torch.std

```
torch.std(input) → float
```

返回输入张量 `input` 所有元素的标准差。

参数： - input (Tensor) – 输入张量

例子：

```
>>> a = torch.randn(1, 3)
```

```
>>> a
```

```
-1.3063 1.4182 -0.3061
```

```
[torch.FloatTensor of size 1x3]
```

```
>>> torch.std(a)
```

```
1.3782334731508061
```

```
torch.std(input, dim, out=None) → Tensor
```

返回输入张量给定维度上每行的标准差。 输出形状与输入相同，除了给定维度上为 1。

参数：

- input (Tensor) – 输入张量

- `dim (int)` – 缩减的维度
- `out (Tensor, optional)` – 结果张量

例子:

```
>>> a = torch.randn(4, 4)
```

```
>>> a
```

```
0.1889 -2.4856 0.0043 1.8169
-0.7701 -0.4682 -2.2410 0.4098
0.1919 -1.1856 -1.0361 0.9085
0.0173 1.0662 0.2143 -0.5576
[torch.FloatTensor of size 4x4]
```

```
>>> torch.std(a, dim=1)
```

```
1.7756
1.1025
1.0045
0.6725
[torch.FloatTensor of size 4x1]
```

torch.sum

`torch.sum(input) → float`

返回输入张量 `input` 所有元素的和。

输出形状与输入相同，除了给定维度上为 1。

参数:

- `input (Tensor)` – 输入张量

例子:

```
>>> a = torch.randn(1, 3)
```

```
>>> a
```

```
0.6170 0.3546 0.0253
[torch.FloatTensor of size 1x3]
```

```
>>> torch.sum(a)
```

```
0.9969287421554327
```

`torch.sum(input, dim, out=None) → Tensor`

返回输入张量给定维度上每行的和。 输出形状与输入相同，除了给定维度上为 1。

参数:

- `input (Tensor)` – 输入张量
- `dim (int)` – 缩减的维度
- `out (Tensor, optional)` – 结果张量

例子:

```
>>> a = torch.randn(4, 4)
```

```
>>> a
```

```
-0.4640  0.0609  0.1122  0.4784
-1.3063  1.6443  0.4714 -0.7396
-1.3561 -0.1959  1.0609 -1.9855
 2.6833  0.5746 -0.5709 -0.4430
[torch.FloatTensor of size 4x4]
```

```
>>> torch.sum(a, 1)
```

```
0.1874
0.0698
-2.4767
2.2440
[torch.FloatTensor of size 4x1]
```

torch.var

torch.var(input) → float

返回输入张量所有元素的方差

输出形状与输入相同，除了给定维度上为 1.

参数:

- input (Tensor) – 输入张量

例子:

```
>>> a = torch.randn(1, 3)
>>> a
```

```
-1.3063  1.4182 -0.3061
[torch.FloatTensor of size 1x3]
```

```
>>> torch.var(a)
```

```
1.899527506513334
```

torch.var(input, dim, out=None) → Tensor

返回输入张量给定维度上每行的方差。 输出形状与输入相同，除了给定维度上为 1.

参数:

- input (Tensor) – 输入张量
- dim (int) – the dimension to reduce
- out (Tensor, optional) – 结果张量 例子:

```
>>> a = torch.randn(4, 4)
>>> a
```

```
-1.2738 -0.3058  0.1230 -1.9615
 0.8771 -0.5430 -0.9233  0.9879
 1.4107  0.0317 -0.6823  0.2255
-1.3854  0.4953 -0.2160  0.2435
[torch.FloatTensor of size 4x4]
```

```
>>> torch.var(a, 1)
```

```
0.8859
```

```
0.9509
```

```
0.7548
```

```
0.6949
```

```
[torch.FloatTensor of size 4x1]
```

比较操作 Comparison Ops

torch.eq

```
torch.eq(input, other, out=None) → Tensor
```

比较元素相等性。第二个参数可为一个数或与第一个参数同类型形状的张量。

参数:

- input (Tensor) – 待比较张量
- other (Tensor or float) – 比较张量或数
- out (Tensor, optional) – 输出张量, 须为 ByteTensor 类型 or 与 input 同类型

返回值: 一个 torch.ByteTensor 张量, 包含了每个位置的比较结果(相等为 1, 不等为 0)

返回类型: Tensor

例子:

```
>>> torch.eq(torch.Tensor([[1, 2], [3, 4]]), torch.Tensor([[1, 1], [4, 4]]))
```

```
1 0
```

```
0 1
```

```
[torch.ByteTensor of size 2x2]
```

torch.equal

```
torch.equal(tensor1, tensor2) → bool
```

如果两个张量有相同的形状和元素值, 则返回 True, 否则 False。

例子:

```
>>> torch.equal(torch.Tensor([1, 2]), torch.Tensor([1, 2]))
```

```
True
```

torch.ge

```
torch.ge(input, other, out=None) → Tensor
```

逐元素比较 input 和 other, 即是否 input >= other。

如果两个张量有相同的形状和元素值, 则返回 True, 否则 False。第二个参数可以为一个数或与第一个参数相同形状和类型的张量

参数:

- input (Tensor) – 待对比的张量
- other (Tensor or float) – 对比的张量或 float 值
- out (Tensor, optional) – 输出张量。必须为 ByteTensor 或者与第一个参数 tensor 相同类型。

返回值: 一个 torch.ByteTensor 张量, 包含了每个位置的比较结果(是否 input >= other)。

返回类型: Tensor

例子:

```
>>> torch.ge(torch.Tensor([[1, 2], [3, 4]]), torch.Tensor([[1, 1], [4, 4]]))
```

```
1 1
```

```
0 1
```

[torch.ByteTensor of size 2x2]

torch.gt

torch.gt(input, other, out=None) → Tensor

逐元素比较 `input` 和 `other`，即是否 `input > other`。如果两个张量有相同的形状和元素值，则返回 `True`，否则 `False`。第二个参数可以为一个数或与第一个参数相同形状和类型的张量。

参数:

- `input` (Tensor) – 要对比的张量
- `other` (Tensor or float) – 要对比的张量或 `float` 值
- `out` (Tensor, optional) – 输出张量。必须为 `ByteTensor` 或者与第一个参数 `tensor` 相同类型。

返回值: 一个 `torch.ByteTensor` 张量，包含了每个位置的比较结果(是否 `input >= other`)。返

回类型: Tensor

例子:

```
>>> torch.gt(torch.Tensor([[1, 2], [3, 4]]), torch.Tensor([[1, 1], [4, 4]]))
```

```
0 1
```

```
0 0
```

[torch.ByteTensor of size 2x2]

torch.kthvalue

torch.kthvalue(input, k, dim=None, out=None) -> (Tensor, LongTensor)

取输入张量 `input` 指定维上第 `k` 个最小值。如果不指定 `dim`，则默认为 `input` 的最后一维。

返回一个元组 (values, indices)，其中 `indices` 是原始输入张量 `input` 中沿 `dim` 维的第 `k` 个最小值下标。

参数:

- `input` (Tensor) – 要对比的张量
- `k` (int) – 第 `k` 个最小值
- `dim` (int, optional) – 沿着此维进行排序
- `out` (tuple, optional) – 输出元组 (Tensor, LongTensor) 可选地给定作为 输出 buffers

例子:

```
>>> x = torch.arange(1, 6)
```

```
>>> x
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

[torch.FloatTensor of size 5]

```
>>> torch.kthvalue(x, 4)
```

```
(
```

```
4
```

[torch.FloatTensor of size 1]

```
,
```

```
3
```

[torch.LongTensor of size 1]

)

torch.le

`torch.le(input, other, out=None) → Tensor`

逐元素比较 `input` 和 `other`，即是否 `input <= other` 第二个参数可以为一个数或与第一个参数相同形状和类型的张量

参数:

- `input (Tensor)` – 要对比的张量
- `other (Tensor or float)` – 对比的张量或 `float` 值
- `out (Tensor, optional)` – 输出张量。必须为 `ByteTensor` 或者与第一个参数 `tensor` 相同类型。

返回值: 一个 `torch.ByteTensor` 张量, 包含了每个位置的比较结果(是否 `input >= other`)。返

回类型: `Tensor`

例子:

```
>>> torch.le(torch.Tensor([[1, 2], [3, 4]]), torch.Tensor([[1, 1], [4, 4]]))
```

```
1 0
```

```
1 1
```

```
[torch.ByteTensor of size 2x2]
```

torch.lt

`torch.lt(input, other, out=None) → Tensor`

逐元素比较 `input` 和 `other`，即是否 `input < other` 第二个参数可以为一个数或与第一个参数相同形状和类型的张量。

参数:

- `input (Tensor)` – 要对比的张量
- `other (Tensor or float)` – 对比的张量或 `float` 值
- `out (Tensor, optional)` – 输出张量。必须为 `ByteTensor` 或者与第一个参数 `tensor` 相同类型。

input: 一个 `torch.ByteTensor` 张量, 包含了每个位置的比较结果(是否 `tensor >= other`)。返

回类型: `Tensor`

例子:

```
>>> torch.lt(torch.Tensor([[1, 2], [3, 4]]), torch.Tensor([[1, 1], [4, 4]]))
```

```
0 0
```

```
1 0
```

```
[torch.ByteTensor of size 2x2]
```

torch.max

`torch.max()`

返回输入张量所有元素的最大值。

参数:

- `input (Tensor)` – 输入张量

例子:

```
>>> a = torch.randn(1, 3)
```

```
>>> a
```

```
0.4729 -0.2266 -0.2085
```

```
[torch.FloatTensor of size 1x3]
```

```
>>> torch.max(a)
```

0.4729

`torch.max(input, dim, max=None, max_indices=None) -> (Tensor, LongTensor)`

返回输入张量给定维度上每行的最大值，并同时返回每个最大值的位置索引。

输出形状中，将 `dim` 维设定为 1，其它与输入形状保持一致。

参数:

- `input (Tensor)` – 输入张量
- `dim (int)` – 指定的维度
- `max (Tensor, optional)` – 结果张量，包含给定维度上的最大值
- `max_indices (LongTensor, optional)` – 结果张量，包含给定维度上每个最大值的位置索引

例子:

```
>> a = torch.randn(4, 4)
```

```
>> a
```

```
0.0692  0.3142  1.2513 -0.5428
```

```
0.9288  0.8552 -0.2073  0.6409
```

```
1.0695 -0.0101 -2.4507 -1.2230
```

```
0.7426 -0.7666  0.4862 -0.6628
```

```
torch.FloatTensor of size 4x4]
```

```
>>> torch.max(a, 1)
```

```
(
```

```
 1.2513
```

```
 0.9288
```

```
 1.0695
```

```
 0.7426
```

```
[torch.FloatTensor of size 4x1]
```

```
,
```

```
 2
```

```
 0
```

```
 0
```

```
 0
```

```
[torch.LongTensor of size 4x1]
```

```
)
```

`torch.max(input, other, out=None) -> Tensor`

返回输入张量给定维度上每行的最大值，并同时返回每个最大值的位置索引。即，

`outi = max(inputi, otheri)`

输出形状中，将 `dim` 维设定为 1，其它与输入形状保持一致。

参数:

- `input (Tensor)` – 输入张量
- `other (Tensor)` – 输出张量
- `out (Tensor, optional)` – 结果张量

例子:

```
>>> a = torch.randn(4)
```

```
>>> a
```



```
1.3869
0.3912
-0.8634
-0.5468
[torch.FloatTensor of size 4]
```

```
>>> b = torch.randn(4)
>>> b
```

```
1.0067
-0.8010
0.6258
0.3627
[torch.FloatTensor of size 4]
```

```
>>> torch.max(a, b)
```

```
1.3869
0.3912
0.6258
0.3627
[torch.FloatTensor of size 4]
```

torch.min

torch.min(input) → float

返回输入张量所有元素的最小值。

参数: input (Tensor) – 输入张量

例子:

```
>>> a = torch.randn(1, 3)
>>> a
```

```
0.4729 -0.2266 -0.2085
[torch.FloatTensor of size 1x3]
```

```
>>> torch.min(a)
-0.22663167119026184
```

torch.min(input, dim, min=None, min_indices=None) -> (Tensor, LongTensor)

返回输入张量给定维度上每行的最小值，并同时返回每个最小值的位置索引。

输出形状中，将 **dim** 维设定为 1，其它与输入形状保持一致。

参数:

- input (Tensor) – 输入张量
- dim (int) – 指定的维度
- min (Tensor, optional) – 结果张量，包含给定维度上的最小值
- min_indices (LongTensor, optional) – 结果张量，包含给定维度上每个最小值的位置索引

例子:

```
>> a = torch.randn(4, 4)
```

```
>> a
```

```
0.0692  0.3142  1.2513 -0.5428
```

```
0.9288  0.8552 -0.2073  0.6409
```

```
1.0695 -0.0101 -2.4507 -1.2230
```

```
0.7426 -0.7666  0.4862 -0.6628
```

```
torch.FloatTensor of size 4x4]
```

```
>> torch.min(a, 1)
```

```
0.5428
```

```
0.2073
```

```
2.4507
```

```
0.7666
```

```
torch.FloatTensor of size 4x1]
```

```
3
```

```
2
```

```
2
```

```
1
```

```
torch.LongTensor of size 4x1]
```

```
torch.min(input, other, out=None) → Tensor
```

input 中逐元素与 **other** 相应位置的元素对比，返回最小值到输出张量。即，

out=min(tensor_i,other_i)两张量形状不需匹配，但元素数须相同。

注意：当形状不匹配时，**input** 的形状作为返回张量的形状。

参数:

- **input** (Tensor) – 输入张量
- **other** (Tensor) – 第二个输入张量
- **out** (Tensor, optional) – 结果张量

例子:

```
>>> a = torch.randn(4)
```

```
>>> a
```

```
1.3869
```

```
0.3912
```

```
-0.8634
```

```
-0.5468
```

```
[torch.FloatTensor of size 4]
```

```
>>> b = torch.randn(4)
```

```
>>> b
```

```
1.0067
-0.8010
0.6258
0.3627
[torch.FloatTensor of size 4]
```

```
>>> torch.min(a, b)
```

```
1.0067
-0.8010
-0.8634
-0.5468
[torch.FloatTensor of size 4]
```

torch.ne

torch.ne(input, other, out=None) → Tensor

逐元素比较 `input` 和 `other`，即是否 `input!=other`。第二个参数可以为一个数或与第一个参数相同形状和类型的张量

参数:

- input (Tensor) – 待对比的张量
- other (Tensor or float) – 对比的张量或 float 值
- out (Tensor, optional) – 输出张量。必须为 ByteTensor 或者与 input 相同类型。

返回值： 一个 torch.ByteTensor 张量，包含了每个位置的比较结果（如果 `tensor != other` 为 True，返回 1）。

返回类型： Tensor

例子:

```
>>> torch.ne(torch.Tensor([[1, 2], [3, 4]]), torch.Tensor([[1, 1], [4, 4]]))
0 1
1 0
[torch.ByteTensor of size 2x2]
```

torch.sort

torch.sort(input, dim=None, descending=False, out=None) -> (Tensor, LongTensor)

对输入张量 `input` 沿着指定维按升序排序。如果不给定 `dim`，则默认为输入的最后二维。如果指定参数 `descending` 为 True，则按降序排序

返回元组 (sorted_tensor, sorted_indices)，sorted_indices 为原始输入中的下标。

参数:

- input (Tensor) – 要对比的张量
- dim (int, optional) – 沿着此维排序
- descending (bool, optional) – 布尔值，控制升降排序
- out (tuple, optional) – 输出张量。必须为 ByteTensor 或者与第一个参数 tensor 相同类型。

例子:

```
>>> x = torch.randn(3, 4)
>>> sorted, indices = torch.sort(x)
>>> sorted
```

```
-1.6747  0.0610  0.1190  1.4137
-1.4782  0.7159  1.0341  1.3678
-0.3324 -0.0782  0.3518  0.4763
[torch.FloatTensor of size 3x4]
```

```
>>> indices
```

```
0  1  3  2
2  1  0  3
3  1  0  2
[torch.LongTensor of size 3x4]
```

```
>>> sorted, indices = torch.sort(x, 0)
>>> sorted
```

```
-1.6747 -0.0782 -1.4782 -0.3324
0.3518  0.0610  0.4763  0.1190
1.0341  0.7159  1.4137  1.3678
[torch.FloatTensor of size 3x4]
```

```
>>> indices
```

```
0  2  1  2
2  0  2  0
1  1  0  1
[torch.LongTensor of size 3x4]
```

torch.topk

```
torch.topk(input, k, dim=None, largest=True, sorted=True, out=None) ->
(Tensor, LongTensor)
```

沿给定 `dim` 维度返回输入张量 `input` 中 `k` 个最大值。如果不指定 `dim`，则默认为 `input` 的最后一维。如果 `largest` 为 `False`，则返回最小的 `k` 个值。

返回一个元组 (values, indices)，其中 `indices` 是原始输入张量 `input` 中元素下标。如果设定布尔值 `sorted` 为 `_True_`，将会确保返回的 `k` 个值被排序。

参数:

- `input (Tensor)` – 输入张量
- `k (int)` – “top-k”中的 `k`
- `dim (int, optional)` – 排序的维
- `largest (bool, optional)` – 布尔值，控制返回最大或最小值
- `sorted (bool, optional)` – 布尔值，控制返回值是否排序
- `out (tuple, optional)` – 可选输出张量 (Tensor, LongTensor) output buffers

```
>>> x = torch.arange(1, 6)
>>> x
```

```
1
2
3
4
5
[torch.FloatTensor of size 5]

>>> torch.topk(x, 3)
(
  5
  4
  3
  [torch.FloatTensor of size 3]
,
  4
  3
  2
  [torch.LongTensor of size 3]
)
>>> torch.topk(x, 3, 0, largest=False)
(
  1
  2
  3
  [torch.FloatTensor of size 3]
,
  0
  1
  2
  [torch.LongTensor of size 3]
)
```

其它操作 Other Operations

`torch.cross`

`torch.cross(input, other, dim=-1, out=None) → Tensor`

返回沿着维度 `dim` 上，两个张量 `input` 和 `other` 的向量积（叉积）。`input` 和 `other` 必须有相同的形状，且指定的 `dim` 维上 `size` 必须为 3。

如果不指定 `dim`，则默认为第一个尺度为 3 的维。

参数：

- `input` (Tensor) – 输入张量
- `other` (Tensor) – 第二个输入张量
- `dim` (int, optional) – 沿着此维进行叉积操作
- `out` (Tensor, optional) – 结果张量

例子:

```
>>> a = torch.randn(4, 3)
```

```
>>> a
```

```
-0.6652 -1.0116 -0.6857
```

```
0.2286 0.4446 -0.5272
```

```
0.0476 0.2321 1.9991
```

```
0.6199 1.1924 -0.9397
```

```
[torch.FloatTensor of size 4x3]
```

```
>>> b = torch.randn(4, 3)
```

```
>>> b
```

```
-0.1042 -1.1156 0.1947
```

```
0.9947 0.1149 0.4701
```

```
-1.0108 0.8319 -0.0750
```

```
0.9045 -1.3754 1.0976
```

```
[torch.FloatTensor of size 4x3]
```

```
>>> torch.cross(a, b, dim=1)
```

```
-0.9619 0.2009 0.6367
```

```
0.2696 -0.6318 -0.4160
```

```
-1.6805 -2.0171 0.2741
```

```
0.0163 -1.5304 -1.9311
```

```
[torch.FloatTensor of size 4x3]
```

```
>>> torch.cross(a, b)
```

```
-0.9619 0.2009 0.6367
```

```
0.2696 -0.6318 -0.4160
```

```
-1.6805 -2.0171 0.2741
```

```
0.0163 -1.5304 -1.9311
```

```
[torch.FloatTensor of size 4x3]
```

torch.diag

`torch.diag(input, diagonal=0, out=None) → Tensor`

- 如果输入是一个向量(1D 张量), 则返回一个以 `input` 为对角线元素的 2D 方阵
- 如果输入是一个矩阵(2D 张量), 则返回一个包含 `input` 对角线元素的 1D 张量

参数 `diagonal` 指定对角线:

- `diagonal = 0`, 主对角线
- `diagonal > 0`, 主对角线之上
- `diagonal < 0`, 主对角线之下

参数:

- `input (Tensor)` – 输入张量

-
- `diagonal (int, optional)` – 指定对角线
 - `out (Tensor, optional)` – 输出张量

例子：

- 取得以 `input` 为对角线的方阵：

```
>>> a = torch.randn(3)
```

```
>>> a
```

```
1.0480
```

```
-2.3405
```

```
-1.1138
```

```
[torch.FloatTensor of size 3]
```

```
>>> torch.diag(a)
```

```
1.0480  0.0000  0.0000
```

```
0.0000 -2.3405  0.0000
```

```
0.0000  0.0000 -1.1138
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> torch.diag(a, 1)
```

```
0.0000  1.0480  0.0000  0.0000
```

```
0.0000  0.0000 -2.3405  0.0000
```

```
0.0000  0.0000  0.0000 -1.1138
```

```
0.0000  0.0000  0.0000  0.0000
```

```
[torch.FloatTensor of size 4x4]
```

- 取得给定矩阵第 `k` 个对角线：

```
>>> a = torch.randn(3, 3)
```

```
>>> a
```

```
-1.5328 -1.3210 -1.5204
```

```
0.8596  0.0471 -0.2239
```

```
-0.6617  0.0146 -1.0817
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> torch.diag(a, 0)
```

```
-1.5328
```

```
0.0471
```

```
-1.0817
```

```
[torch.FloatTensor of size 3]
```

```
>>> torch.diag(a, 1)
```

```
-1.3210
```

```
-0.2239
```

```
[torch.FloatTensor of size 2]
```

torch.histc

```
torch.histc(input, bins=100, min=0, max=0, out=None) → Tensor
```

计算输入张量的直方图。以 `min` 和 `max` 为 `range` 边界，将其均分成 `bins` 个直条，然后将排序好的数据划分到各个直条(bins)中。如果 `min` 和 `max` 都为 0，则利用数据中的最大最小值作为边界。

参数：

- `input` (Tensor) – 输入张量
- `bins` (int) – 直方图 bins(直条)的个数(默认 100 个)
- `min` (int) – `range` 的下边界(包含)
- `max` (int) – `range` 的上边界(包含)
- `out` (Tensor, optional) – 结果张量

返回： 直方图 返回类型： 张量

例子：

```
>>> torch.histc(torch.FloatTensor([1, 2, 1]), bins=4, min=0, max=3)
```

```
FloatTensor([0, 2, 1, 0])
```

torch.renorm

```
torch.renorm(input, p, dim, maxnorm, out=None) → Tensor
```

返回一个张量，包含规范化后的各个子张量，使得沿着 `dim` 维划分的各子张量的 `p` 范数小于 `maxnorm`。

注意 如果 `p` 范数的值小于 `maxnorm`，则当前子张量不需要修改。

注意： 更详细解释参考 [torch7](#) 以及 [Hinton et al. 2012, p. 2](#)

参数：

- `input` (Tensor) – 输入张量
- `p` (float) – 范数的 `p`
- `dim` (int) – 沿着此维切片，得到张量子集
- `maxnorm` (float) – 每个子张量的范数的最大值
- `out` (Tensor, optional) – 结果张量

例子：

```
>>> x = torch.ones(3, 3)
```

```
>>> x[1].fill_(2)
```

```
>>> x[2].fill_(3)
```

```
>>> x
```

```
1 1 1
```

```
2 2 2
```

```
3 3 3
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> torch.renorm(x, 1, 0, 5)
```

```
1.0000 1.0000 1.0000
```

```
1.6667 1.6667 1.6667
```

```
1.6667 1.6667 1.6667
```

```
[torch.FloatTensor of size 3x3]
```

torch.trace

```
torch.trace(input) → float
```

返回输入 2 维矩阵对角线元素的和(迹)

例子:

```
>>> x = torch.arange(1, 10).view(3, 3)
```

```
>>> x
```

```
1  2  3
```

```
4  5  6
```

```
7  8  9
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> torch.trace(x)
```

```
15.0
```

torch.tril

```
torch.tril(input, k=0, out=None) → Tensor
```

返回一个张量 **out**, 包含输入矩阵(2D 张量)的下三角部分, **out** 其余部分被设为 0。这里所说的下三角部分为矩阵指定对角线 **diagonal** 之上的元素。

参数 **k** 控制对角线: - **k** = 0, 主对角线 - **k** > 0, 主对角线之上 - **k** < 0, 主对角线之下

参数:

- input (Tensor) – 输入张量
- k (int, optional) – 指定对角线
- out (Tensor, optional) – 输出张量

例子:

```
>>> a = torch.randn(3,3)
```

```
>>> a
```

```
1.3225  1.7304  1.4573
```

```
-0.3052 -0.3111 -0.1809
```

```
1.2469  0.0064 -1.6250
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> torch.tril(a)
```

```
1.3225  0.0000  0.0000
```

```
-0.3052 -0.3111  0.0000
```

```
1.2469  0.0064 -1.6250
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> torch.tril(a, k=1)
```

```
1.3225  1.7304  0.0000
```

```
-0.3052 -0.3111 -0.1809
```

```
1.2469  0.0064 -1.6250
[torch.FloatTensor of size 3x3]
```

```
>>> torch.tril(a, k=-1)
```

```
0.0000  0.0000  0.0000
-0.3052  0.0000  0.0000
1.2469  0.0064  0.0000
[torch.FloatTensor of size 3x3]
```

torch.triu

torch.triu(input, k=0, out=None) → Tensor

返回一个张量，包含输入矩阵(2D 张量)的上三角部分，其余部分被设为 0。这里所说的上三角部分为矩阵指定对角线 **diagonal** 之上的元素。

参数 **k** 控制对角线: $-k = 0$, 主对角线 $-k > 0$, 主对角线之上 $-k < 0$, 主对角线之下

参数:

- input (Tensor) – 输入张量
- k (int, optional) – 指定对角线
- out (Tensor, optional) – 输出张量

例子:

```
>>> a = torch.randn(3,3)
>>> a
```

```
1.3225  1.7304  1.4573
-0.3052 -0.3111 -0.1809
1.2469  0.0064 -1.6250
[torch.FloatTensor of size 3x3]
```

```
>>> torch.triu(a)
```

```
1.3225  1.7304  1.4573
0.0000 -0.3111 -0.1809
0.0000  0.0000 -1.6250
[torch.FloatTensor of size 3x3]
```

```
>>> torch.triu(a, k=1)
```

```
0.0000  1.7304  1.4573
0.0000  0.0000 -0.1809
0.0000  0.0000  0.0000
[torch.FloatTensor of size 3x3]
```

```
>>> torch.triu(a, k=-1)
```

```
1.3225  1.7304  1.4573
```

```
-0.3052 -0.3111 -0.1809
0.0000 0.0064 -1.6250
[torch.FloatTensor of size 3x3]
```

BLAS and LAPACK Operations

torch.addbmm

`torch.addbmm(beta=1, mat, alpha=1, batch1, batch2, out=None) → Tensor`

对两个批 `batch1` 和 `batch2` 内存储的矩阵进行批矩阵乘操作，附带 `reduced add` 步骤(所有矩阵乘结果沿着第一维相加)。矩阵 `mat` 加到最终结果。`batch1` 和 `batch2` 都为包含相同数量矩阵的 3 维张量。如果 `batch1` 是形为 $b \times n \times m$ 的张量，`batch2` 是形为 $b \times m \times p$ 的张量，则 `out` 和 `mat` 的形状都是 $n \times p$ ，即 $res = (beta * M) + (alpha * \sum(batch1_i @ batch2_i, i=0, b))$
对类型为 `FloatTensor` 或 `DoubleTensor` 的输入，`alpha` 和 `beta` 必须为实数，否则两个参数须为整数。

参数：

- `beta` (Number, optional) – 用于 `mat` 的乘子
- `mat` (Tensor) – 相加矩阵
- `alpha` (Number, optional) – 用于 `batch1 @ batch2` 的乘子
- `batch1` (Tensor) – 第一批相乘矩阵
- `batch2` (Tensor) – 第二批相乘矩阵
- `out` (Tensor, optional) – 输出张量

例子：

```
>>> M = torch.randn(3, 5)
>>> batch1 = torch.randn(10, 3, 4)
>>> batch2 = torch.randn(10, 4, 5)
>>> torch.addbmm(M, batch1, batch2)
```

```
-3.1162 11.0071 7.3102 0.1824 -7.6892
1.8265 6.0739 0.4589 -0.5641 -5.4283
-9.3387 -0.1794 -1.2318 -6.8841 -4.7239
[torch.FloatTensor of size 3x5]
```

torch.addmm

`torch.addmm(beta=1, mat, alpha=1, mat1, mat2, out=None) → Tensor`

对矩阵 `mat1` 和 `mat2` 进行矩阵乘操作。矩阵 `mat` 加到最终结果。如果 `mat1` 是一个 $n \times m$ 张量，`mat2` 是一个 $m \times p$ 张量，那么 `out` 和 `mat` 的形状为 $n \times p$ 。`alpha` 和 `beta` 分别是两个矩阵 `mat1 @ mat2` 和 `mat` 的比例因子，

即， $out = (beta * M) + (alpha * mat1 @ mat2)$

对类型为 `FloatTensor` 或 `DoubleTensor` 的输入，`beta` 和 `alpha` 必须为实数，否则两个参数须为整数。

参数：

- `beta` (Number, optional) – 用于 `mat` 的乘子
- `mat` (Tensor) – 相加矩阵
- `alpha` (Number, optional) – 用于 `mat1 @ mat2` 的乘子
- `mat1` (Tensor) – 第一个相乘矩阵

- `mat2 (Tensor)` – 第二个相乘矩阵
- `out (Tensor, optional)` – 输出张量

```
>>> M = torch.randn(2, 3)
>>> mat1 = torch.randn(2, 3)
>>> mat2 = torch.randn(3, 3)
>>> torch.addmm(M, mat1, mat2)
```

```
-0.4095 -1.9703  1.3561
```

```
5.7674 -4.9760  2.7378
```

```
[torch.FloatTensor of size 2x3]
```

`torch.addmv`

`torch.addmv(beta=1, tensor, alpha=1, mat, vec, out=None) → Tensor`

对矩阵 `mat` 和向量 `vec` 对进行相乘操作。向量 `tensor` 加到最终结果。如果 `mat` 是一个 $n \times m$ 维矩阵，`vec` 是一个 m 维向量，那么 `out` 和 `mat` 的为 n 元向量。可选参数 `alpha` 和 `beta` 分别是 `mat*vec` 和 `mat` 的比例因子，即，`out=(beta*tensor)+(alpha*(mat@vec))`

对类型为 `_FloatTensor_` 或 `_DoubleTensor_` 的输入，`alpha` 和 `beta` 必须为实数，否则两个参数须为整数。

参数：

- `beta (Number, optional)` – 用于 `mat` 的乘子
- `mat (Tensor)` – 相加矩阵
- `alpha (Number, optional)` – 用于 `mat1@vec` 的乘子
- `mat (Tensor)` – 相乘矩阵
- `vec (Tensor)` – 相乘向量
- `out (Tensor, optional)` – 输出张量

```
>>> M = torch.randn(2)
>>> mat = torch.randn(2, 3)
>>> vec = torch.randn(3)
>>> torch.addmv(M, mat, vec)
```

```
-2.0939
```

```
-2.2950
```

```
[torch.FloatTensor of size 2]
```

`torch.addr`

`torch.addr(beta=1, mat, alpha=1, vec1, vec2, out=None) → Tensor`

对向量 `vec1` 和 `vec2` 对进行张量积操作。矩阵 `mat` 加到最终结果。如果 `vec1` 是一个 n 维向量，`vec2` 是一个 m 维向量，那么矩阵 `mat` 的形状须为 $n \times m$ 。可选参数 `beta` 和 `alpha` 分别是两个矩阵 `matmat` 和 `vec1@vec2` 的比例因子，即，

`resi=(beta*Mi)+(alpha*batch1i*batch2i)`

对类型为 `_FloatTensor_` 或 `_DoubleTensor_` 的输入，`alpha` 和 `beta` 必须为实数，否则两个参数须为整数。

参数：

- `beta (Number, optional)` – 用于 `mat` 的乘子
- `mat (Tensor)` – 相加矩阵
- `alpha (Number, optional)` – 用于两向量 `vec1`, `vec2` 外积的乘子

- `vec1 (Tensor)` – 第一个相乘向量
- `vec2 (Tensor)` – 第二个相乘向量
- `out (Tensor, optional)` – 输出张量

```
>>> vec1 = torch.arange(1, 4)
>>> vec2 = torch.arange(1, 3)
>>> M = torch.zeros(3, 2)
>>> torch.addr(M, vec1, vec2)

1 2
2 4
3 6
```

[torch.FloatTensor of size 3x2]

`torch.baddbmm`

`torch.baddbmm(beta=1, mat, alpha=1, batch1, batch2, out=None) → Tensor`

对两个批 `batch1` 和 `batch2` 内存储的矩阵进行批矩阵乘操作，矩阵 `mat` 加到最终结果。`batch1` 和 `batch2` 都为包含相同数量矩阵的 3 维张量。如果 `batch1` 是形为 $b \times n \times m$ 的张量，`batch1` 是形为 $b \times m \times p$ 的张量，则 `out` 和 `mat` 的形状都是 $n \times p$ ，

即 $\text{resi} = (\text{beta} * \text{Mi}) + (\text{alpha} * \text{batch1}_i \times \text{batch2}_i)$

对类型为 `_FloatTensor_` 或 `_DoubleTensor_` 的输入，`alpha` 和 `beta` 必须为实数，否则两个参数须为整数。

参数：

- `beta (Number, optional)` – 用于 `mat` 的乘子
- `mat (Tensor)` – 相加矩阵
- `alpha (Number, optional)` – 用于 `batch1@batch2` 的乘子
- `batch1 (Tensor)` – 第一批相乘矩阵
- `batch2 (Tensor)` – 第二批相乘矩阵
- `out (Tensor, optional)` – 输出张量

```
>>> M = torch.randn(10, 3, 5)
>>> batch1 = torch.randn(10, 3, 4)
>>> batch2 = torch.randn(10, 4, 5)
>>> torch.baddbmm(M, batch1, batch2).size()

torch.Size([10, 3, 5])
```

`torch.bmm`

`torch.bmm(batch1, batch2, out=None) → Tensor`

对存储在两个批 `batch1` 和 `batch2` 内的矩阵进行批矩阵乘操作。`batch1` 和 `batch2` 都为包含相同数量矩阵的 3 维张量。如果 `batch1` 是形为 $b \times n \times m$ 的张量，`batch1` 是形为 $b \times m \times p$ 的张量，则 `out` 和 `mat` 的形状都是 $n \times p$ ，

即 $\text{res} = (\text{beta} * \text{M}) + (\text{alpha} * \text{sum}(\text{batch1}_i @ \text{batch2}_i, i=0, b))$

对类型为 `FloatTensor` 或 `DoubleTensor` 的输入，`alpha` 和 `beta` 必须为实数，否则两个参数须为整数。

参数：

- `batch1 (Tensor)` – 第一批相乘矩阵
- `batch2 (Tensor)` – 第二批相乘矩阵
- `out (Tensor, optional)` – 输出张量

```
>>> batch1 = torch.randn(10, 3, 4)
```

```
>>> batch2 = torch.randn(10, 4, 5)
>>> res = torch.bmm(batch1, batch2)
>>> res.size()
torch.Size([10, 3, 5])
```

torch.btrifact

`torch.btrifact(A, info=None) → Tensor, IntTensor`

返回一个元组，包含 LU 分解和 `pivots`。可选参数 `info` 决定是否对每个 minibatch 样本进行分解。`info` are from dgetrf and a non-zero value indicates an error occurred. 如果用 CUDA 的话，这个值来自于 CUBLAS，否则来自 LAPACK。

参数：A (Tensor) – 待分解张量

```
>>> A = torch.randn(2, 3, 3)
>>> A_LU = A.btrifact()
```

torch.btrisolve

`torch.btrisolve(b, LU_data, LU_pivots) → Tensor`

返回线性方程组 $Ax=b$ 的 LU 解。

参数：

- b (Tensor) – RHS 张量。
- LU_data (Tensor) – Pivoted LU factorization of A from btrifact.
- LU_pivots (IntTensor) – LU 分解的 Pivots.

例子：

```
>>> A = torch.randn(2, 3, 3)
>>> b = torch.randn(2, 3)
>>> A_LU = torch.btrifact(A)
>>> x = b.btrisolve(*A_LU)
>>> torch.norm(A.bmm(x.unsqueeze(2)) - b)
6.664001874625056e-08
```

torch.dot

`torch.dot(tensor1, tensor2) → float`

计算两个张量的点乘(内乘),两个张量都为 1-D 向量。

例子：

```
>>> torch.dot(torch.Tensor([2, 3]), torch.Tensor([2, 1]))
7.0
```

torch.eig

`torch.eig(a, eigenvectors=False, out=None) -> (Tensor, Tensor)`

计算实方阵 a 的特征值和特征向量

参数：

- a (Tensor) – 方阵，待计算其特征值和特征向量
- eigenvectors (bool) – 布尔值，如果 `True`，则同时计算特征值和特征向量，否则只计算特征值。
- out (tuple, optional) – 输出元组

返回值：元组，包括：

- e (Tensor): a 的右特征向量
- v (Tensor): 如果 `eigenvectors` 为 `True`，则为包含特征向量的张量；否则为空张量

返回值类型：(Tensor, Tensor)

torch.gels

```
torch.gels(B, A, out=None) → Tensor
```

对形如 $m \times n$ 的满秩矩阵 A 计算其最小二乘和最小范数问题的解。如果 $m \geq n$, `gels` 对最小二乘问题进行求解, 即:

```
minimize || AX-B ||_F
```

如果 $m < n$, `gels` 求解最小范数问题, 即:

```
minimize || X ||_F subject to ABX=B
```

返回矩阵 XX^T 的前 nn 行包含解。余下的行包含以下残差信息: 相应列从第 n 行开始计算的每列的欧式距离。

注意: 返回矩阵总是被转置, 无论输入矩阵的原始布局如何, 总会被转置; 即, 总是有 `stride (1, m)` 而不是 `(m, 1)`。

参数:

- `B (Tensor)` – 矩阵 B
- `A (Tensor)` – $m \times n$ 矩阵
- `out (tuple, optional)` – 输出元组

返回值: 元组, 包括:

- `X (Tensor)`: 最小二乘解
- `qr (Tensor)`: QR 分解的细节

返回值类型: `(Tensor, Tensor)`

例子:

```
>>> A = torch.Tensor([[1, 1],
...                    [2, 3, 4],
...                    [3, 5, 2],
...                    [4, 2, 5],
...                    [5, 4, 3]])
>>> B = torch.Tensor([[ -10, -3],
...                    [ 12, 14],
...                    [ 14, 12],
...                    [ 16, 16],
...                    [ 18, 16]])
>>> X, _ = torch.gels(B, A)
>>> X
2.0000  1.0000
1.0000  1.0000
1.0000  2.0000
[torch.FloatTensor of size 3x2]
```

`torch.geqrf`

```
torch.geqrf(input, out=None) -> (Tensor, Tensor)
```

这是一个直接调用 LAPACK 的底层函数。一般使用 `torch.qr()`

计算输入的 QR 分解, 但是并不会分别创建 Q, R 两个矩阵, 而是直接调用 LAPACK 函数 `Rather, this directly calls the underlying LAPACK function 'geqrf' which produces a sequence of 'elementary reflectors'.`

参考 [LAPACK 文档](#) 获取更详细信息。

参数:

- `input (Tensor)` – 输入矩阵

- out (tuple, optional) – 元组，包含输出张量 (Tensor, Tensor)

torch.ger

torch.ger(vec1, vec2, out=None) → Tensor

计算两向量 `vec1`, `vec2` 的张量积。如果 `vec1` 的长度为 `n`, `vec2` 长度为 `m`，则输出 `out` 应为形如 `n x m` 的矩阵。

参数:

- vec1 (Tensor) – 1D 输入向量
- vec2 (Tensor) – 1D 输入向量
- out (tuple, optional) – 输出张量

例子:

```
>>> v1 = torch.arange(1, 5)
>>> v2 = torch.arange(1, 4)
>>> torch.ger(v1, v2)
```

```
 1  2  3
 2  4  6
 3  6  9
 4  8 12
```

[torch.FloatTensor of size 4x3]

torch.gesv

torch.gesv(B, A, out=None) -> (Tensor, Tensor)

`X, LU=torch.gesv(B,A)`，返回线性方程组 $AX=B$ 的解。

`LU` 包含两个矩阵 `L`, `U`。 `A` 须为非奇异方阵，如果 `A` 是一个 $m \times m$ 矩阵， `B` 是 $m \times k$ 矩阵，则 `LU` 是 $m \times m$ 矩阵， `X` 为 $m \times k$ 矩阵

参数:

- B (Tensor) – $m \times k$ 矩阵
- A (Tensor) – $m \times m$ 矩阵
- out (Tensor, optional) – 可选地输出矩阵 `XX`

例子:

```
>>> A = torch.Tensor([[6.80, -2.11, 5.66, 5.97, 8.23],
...                    [-6.05, -3.30, 5.36, -4.44, 1.08],
...                    [-0.45, 2.58, -2.70, 0.27, 9.04],
...                    [8.32, 2.71, 4.35, -7.17, 2.14],
...                    [-9.67, -5.14, -7.26, 6.08, -6.87]]).t()
>>> B = torch.Tensor([[4.02, 6.19, -8.22, -7.57, -3.03],
...                    [-1.56, 4.00, -8.67, 1.75, 2.86],
...                    [9.81, -4.09, -4.57, -8.61, 8.99]]).t()
>>> X, LU = torch.gesv(B, A)
>>> torch.dist(B, torch.mm(A, X))
9.250057093890353e-06
```

torch.inverse

torch.inverse(input, out=None) → Tensor

对方阵输入 `input` 取逆。

注意： Irrespective of the original strides, the returned matrix will be transposed, i.e. with strides (1,

m) instead of (m, 1)

参数：

- input (Tensor) – 输入 2 维张量
- out (Tensor, optional) – 输出张量

例子:

```
>>> x = torch.rand(10, 10)
```

```
>>> x
```

0.7800	0.2267	0.7855	0.9479	0.5914	0.7119	0.4437	0.9131	0.1289	0.1982
0.0045	0.0425	0.2229	0.4626	0.6210	0.0207	0.6338	0.7067	0.6381	0.8196
0.8350	0.7810	0.8526	0.9364	0.7504	0.2737	0.0694	0.5899	0.8516	0.3883
0.6280	0.6016	0.5357	0.2936	0.7827	0.2772	0.0744	0.2627	0.6326	0.9153
0.7897	0.0226	0.3102	0.0198	0.9415	0.9896	0.3528	0.9397	0.2074	0.6980
0.5235	0.6119	0.6522	0.3399	0.3205	0.5555	0.8454	0.3792	0.4927	0.6086
0.1048	0.0328	0.5734	0.6318	0.9802	0.4458	0.0979	0.3320	0.3701	0.0909
0.2616	0.3485	0.4370	0.5620	0.5291	0.8295	0.7693	0.1807	0.0650	0.8497
0.1655	0.2192	0.6913	0.0093	0.0178	0.3064	0.6715	0.5101	0.2561	0.3396
0.4370	0.4695	0.8333	0.1180	0.4266	0.4161	0.0699	0.4263	0.8865	0.2578

[torch.FloatTensor of size 10x10]

```
>>> x = torch.rand(10, 10)
```

```
>>> y = torch.inverse(x)
```

```
>>> z = torch.mm(x, y)
```

```
>>> z
```

1.0000	0.0000	0.0000	-0.0000	0.0000	0.0000	0.0000	0.0000	-0.0000	-0.0000
0.0000	1.0000	-0.0000	0.0000	0.0000	0.0000	-0.0000	-0.0000	-0.0000	-0.0000
0.0000	0.0000	1.0000	-0.0000	-0.0000	0.0000	0.0000	0.0000	-0.0000	-0.0000
0.0000	0.0000	0.0000	1.0000	0.0000	0.0000	0.0000	-0.0000	-0.0000	0.0000
0.0000	0.0000	-0.0000	-0.0000	1.0000	0.0000	0.0000	-0.0000	-0.0000	-0.0000
0.0000	0.0000	0.0000	-0.0000	0.0000	1.0000	-0.0000	-0.0000	-0.0000	-0.0000
0.0000	0.0000	0.0000	-0.0000	0.0000	0.0000	1.0000	0.0000	-0.0000	0.0000
0.0000	0.0000	-0.0000	-0.0000	0.0000	0.0000	-0.0000	1.0000	-0.0000	0.0000
-0.0000	0.0000	-0.0000	-0.0000	0.0000	0.0000	-0.0000	-0.0000	1.0000	-0.0000
-0.0000	0.0000	-0.0000	-0.0000	-0.0000	0.0000	-0.0000	-0.0000	0.0000	1.0000

[torch.FloatTensor of size 10x10]

```
>>> torch.max(torch.abs(z - torch.eye(10))) # Max nonzero
```

```
5.096662789583206e-07
```

torch.mm

`torch.mm(mat1, mat2, out=None) → Tensor`

对矩阵 `mat1` 和 `mat2` 进行相乘。如果 `mat1` 是一个 $n \times m$ 张量，`mat2` 是一个 $m \times p$ 张量，将会输出一个 $n \times p$ 张量 `out`。

参数：

- `mat1 (Tensor)` – 第一个相乘矩阵
- `mat2 (Tensor)` – 第二个相乘矩阵
- `out (Tensor, optional)` – 输出张量

例子:

```
>>> mat1 = torch.randn(2, 3)
>>> mat2 = torch.randn(3, 3)
>>> torch.mm(mat1, mat2)
0.0519 -0.3304  1.2232
4.3910 -5.1498  2.7571
[torch.FloatTensor of size 2x3]
```

`torch.mv`

`torch.mv(mat, vec, out=None) → Tensor`

对矩阵 `mat` 和向量 `vec` 进行相乘。 如果 `mat` 是一个 $n \times m$ 张量, `vec` 是一个 m 元 1 维张量, 将会输出一个 n 元 1 维张量。

参数 :

- `mat (Tensor)` – 相乘矩阵
- `vec (Tensor)` – 相乘向量
- `out (Tensor, optional)` – 输出张量

例子:

```
>>> mat = torch.randn(2, 3)
>>> vec = torch.randn(3)
>>> torch.mv(mat, vec)
-2.0939
-2.2950
[torch.FloatTensor of size 2]
```

`torch.orgqr`

`torch.orgqr()`

`torch.ormqr`

`torch.ormqr()`

`torch.potrf`

`torch.potrf()`

`torch.potri`

`torch.potri()`

`torch.potrs`

`torch.potrs()`

`torch.pstrf`

`torch.pstrf()`

`torch.qr`

`torch.qr(input, out=None) -> (Tensor, Tensor)`

计算输入矩阵的 QR 分解: 返回两个矩阵 `q, r`, 使得 $x = q * r$, 这里 `q` 是一个半正交矩阵与 `r` 是一个上三角矩阵

本函数返回一个 `thin(reduced)` QR 分解。

注意 如果输入很大, 可能可能会丢失精度。

注意 本函数依赖于你的 LAPACK 实现, 虽然总能返回一个合法的分解, 但不同平台可能得到不同的

结果。

Irrespective of the original strides, the returned matrix q will be transposed, i.e. with strides (1, m) instead of (m, 1).

参数:

- input (Tensor) – 输入的 2 维张量
- out (tuple, optional) – 输出元组 tuple, 包含 Q 和 R

例子:

```
>>> a = torch.Tensor([[12, -51, 4], [6, 167, -68], [-4, 24, -41]])
```

```
>>> q, r = torch.qr(a)
```

```
>>> q
```

```
-0.8571  0.3943  0.3314
```

```
-0.4286 -0.9029 -0.0343
```

```
0.2857 -0.1714  0.9429
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> r
```

```
-14.0000 -21.0000  14.0000
```

```
0.0000 -175.0000  70.0000
```

```
0.0000  0.0000 -35.0000
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> torch.mm(q, r).round()
```

```
12  -51   4
```

```
6   167  -68
```

```
-4   24  -41
```

```
[torch.FloatTensor of size 3x3]
```

```
>>> torch.mm(q.t(), q).round()
```

```
1  0  0
```

```
0  1  0
```

```
0  0  1
```

```
[torch.FloatTensor of size 3x3]
```

torch.svd

```
torch.svd(input, some=True, out=None) -> (Tensor, Tensor, Tensor)
```

$U, S, V = \text{torch.svd}(A)$ 。 返回对形如 $n \times m$ 的实矩阵 A 进行奇异值分解的结果，使得 $A = USV^*$ 。 U 形状为 $n \times n$ ， S 形状为 $n \times m$ ， V 形状为 $m \times m$

`some` 代表了需要计算的奇异值数目。如果 `some=True`, it computes some and `some=False` computes all.

Irrespective of the original strides, the returned matrix U will be transposed, i.e. with strides (1, n) instead of (n, 1).

参数:

- input (Tensor) – 输入的 2 维张量
- some (bool, optional) – 布尔值, 控制需计算的奇异值数目
- out (tuple, optional) – 结果 tuple

例子:

```
>>> a = torch.Tensor([[8.79, 6.11, -9.15, 9.57, -3.49, 9.84],
...                    [9.93, 6.91, -7.93, 1.64, 4.02, 0.15],
...                    [9.83, 5.04, 4.86, 8.83, 9.80, -8.99],
...                    [5.45, -0.27, 4.85, 0.74, 10.00, -6.02],
...                    [3.16, 7.98, 3.01, 5.80, 4.27, -5.31]]).t()
>>> a
```

```
8.7900  9.9300  9.8300  5.4500  3.1600
6.1100  6.9100  5.0400 -0.2700  7.9800
-9.1500 -7.9300  4.8600  4.8500  3.0100
9.5700  1.6400  8.8300  0.7400  5.8000
-3.4900 4.0200  9.8000 10.0000  4.2700
9.8400  0.1500 -8.9900 -6.0200 -5.3100
```

[torch.FloatTensor of size 6x5]

```
>>> u, s, v = torch.svd(a)
```

```
>>> u
```

```
-0.5911  0.2632  0.3554  0.3143  0.2299
-0.3976  0.2438 -0.2224 -0.7535 -0.3636
-0.0335 -0.6003 -0.4508  0.2334 -0.3055
-0.4297  0.2362 -0.6859  0.3319  0.1649
-0.4697 -0.3509  0.3874  0.1587 -0.5183
0.2934  0.5763 -0.0209  0.3791 -0.6526
```

[torch.FloatTensor of size 6x5]

```
>>> s
```

```
27.4687
22.6432
8.5584
5.9857
2.0149
```

[torch.FloatTensor of size 5]

```
>>> v
```

```
-0.2514  0.8148 -0.2606  0.3967 -0.2180
-0.3968  0.3587  0.7008 -0.4507  0.1402
```

```
-0.6922 -0.2489 -0.2208  0.2513  0.5891
-0.3662 -0.3686  0.3859  0.4342 -0.6265
-0.4076 -0.0980 -0.4932 -0.6227 -0.4396
[torch.FloatTensor of size 5x5]
```

```
>>> torch.dist(a, torch.mm(torch.mm(u, torch.diag(s)), v.t()))
8.934150226306685e-06
```

`torch.symeig`

```
torch.symeig(input, eigenvectors=False, upper=True, out=None) -> (Tensor,
Tensor)
```

`e, V=torch.symeig(input)` 返回实对称矩阵 `input` 的特征值和特征向量。

`input` 和 `V` 为 $m \times m$ 矩阵，`e` 是一个 m 维向量。此函数计算 `input` 的所有特征值(和特征向量)，使得 `input=Vdiag(e)V'`

布尔值参数 `eigenvectors` 规定是否只计算特征向量。如果为 `False`，则只计算特征值；若设为 `True`，则两者都会计算。因为输入矩阵 `input` 是对称的，所以默认只需要上三角矩阵。如果参数 `upper` 为 `False`，下三角矩阵部分也被利用。

注意：不管原来 Irrespective of the original strides, the returned matrix V will be transposed, i.e. with strides (1, m) instead of (m, 1)

参数：

- `input` (Tensor) – 输入对称矩阵
- `eigenvectors` (boolean, optional) – 布尔值（可选），控制是否计算特征向量
- `upper` (boolean, optional) – 布尔值（可选），控制是否考虑上三角或下三角区域
- `out` (tuple, optional) – 输出元组(Tensor, Tensor)

例子：

```
>>> a = torch.Tensor([[ 1.96,  0.00,  0.00,  0.00,  0.00],
...                    [-6.49,  3.80,  0.00,  0.00,  0.00],
...                    [-0.47, -6.39,  4.17,  0.00,  0.00],
...                    [-7.20,  1.50, -1.51,  5.70,  0.00],
...                    [-0.65, -6.34,  2.67,  1.80, -7.10]]).t()
```

```
>>> e, v = torch.symeig(a, eigenvectors=True)
```

```
>>> e
```

```
-11.0656
-6.2287
 0.8640
 8.8655
16.0948
```

```
[torch.FloatTensor of size 5]
```

```
>>> v
```

```
-0.2981 -0.6075  0.4026 -0.3745  0.4896
```

```
-0.5078 -0.2880 -0.4066 -0.3572 -0.6053  
-0.0816 -0.3843 -0.6600  0.5008  0.3991  
-0.0036 -0.4467  0.4553  0.6204 -0.4564  
-0.8041  0.4480  0.1725  0.3108  0.1622
```

```
[torch.FloatTensor of size 5x5]
```

```
torch.trtrs
```

```
torch.trtrs()
```



关注【深度学习爱好者】

更多学习资源免费获取

第二章 torch.Tensor

`torch.Tensor` 是一种包含单一数据类型元素的多维矩阵。

Torch 定义了七种 CPU tensor 类型和八种 GPU tensor 类型：

32-bit floating point	<code>torch.FloatTensor</code>	<code>torch.cuda.FloatTensor</code>
64-bit floating point	<code>torch.DoubleTensor</code>	<code>torch.cuda.DoubleTensor</code>
16-bit floating point	N/A	<code>torch.cuda.HalfTensor</code>
8-bit integer (unsigned)	<code>torch.ByteTensor</code>	<code>torch.cuda.ByteTensor</code>
8-bit integer (signed)	<code>torch.CharTensor</code>	<code>torch.cuda.CharTensor</code>
16-bit integer (signed)	<code>torch.ShortTensor</code>	<code>torch.cuda.ShortTensor</code>
32-bit integer (signed)	<code>torch.IntTensor</code>	<code>torch.cuda.IntTensor</code>
64-bit integer (signed)	<code>torch.LongTensor</code>	<code>torch.cuda.LongTensor</code>

`torch.Tensor` 是默认的 tensor 类型（`torch.FloatTensor`）的简称。

一个张量 tensor 可以从 Python 的 `list` 或序列构建：

```
>>> torch.FloatTensor([[1, 2, 3], [4, 5, 6]])
1 2 3
4 5 6
[torch.FloatTensor of size 2x3]
```

一个空张量 tensor 可以通过规定其大小来构建：

```
>>> torch.IntTensor(2, 4).zero_()
0 0 0 0
0 0 0 0
[torch.IntTensor of size 2x4]
```

可以用 python 的索引和切片来获取和修改一个张量 tensor 中的内容：

```
>>> x = torch.FloatTensor([[1, 2, 3], [4, 5, 6]])
>>> print(x[1][2])
6.0
>>> x[0][1] = 8
>>> print(x)
1 8 3
4 5 6
[torch.FloatTensor of size 2x3]
```

每一个张量 tensor 都有一个相应的 `torch.Storage` 用来保存其数据。类 tensor 提供了一个存储的多维的、横向视图，并且定义了数值运算。

！注意： 会改变 tensor 的函数操作会用一个下划线后缀来标示。比如，`torch.FloatTensor.abs_()` 会在原地计算绝对值，并返回改变后的 tensor，而 `torch.FloatTensor.abs()` 将会在一个新的 tensor 中计算结果。

```
class torch.Tensor
class torch.Tensor(*sizes)
class torch.Tensor(size)
class torch.Tensor(sequence)
```

```
class torch.Tensor(ndarray)
```

```
class torch.Tensor(tensor)
```

```
class torch.Tensor(storage)
```

根据可选择的大小和数据新建一个 `tensor`。如果没有提供参数，将会返回一个空的零维张量。如果提供了 `numpy.ndarray`, `torch.Tensor` 或 `torch.Storage`，将会返回一个有同样参数的 `tensor`。

如果提供了 python 序列，将会从序列的副本创建一个 `tensor`。

```
abs() → Tensor
```

请查看 `torch.abs()`

```
abs_() → Tensor
```

`abs()` 的 in-place 运算形式

```
acos() → Tensor
```

请查看 `torch.acos()`

```
acos_() → Tensor
```

`acos()` 的 in-place 运算形式

```
add(value)
```

请查看 `torch.add()`

```
add_(value)
```

`add()` 的 in-place 运算形式

```
addbmm(beta=1, mat, alpha=1, batch1, batch2) → Tensor
```

请查看 `torch.addbmm()`

```
addbmm_(beta=1, mat, alpha=1, batch1, batch2) → Tensor
```

`addbmm()` 的 in-place 运算形式

```
addcddiv(value=1, tensor1, tensor2) → Tensor
```

请查看 `torch.addcddiv()`

```
addcddiv_(value=1, tensor1, tensor2) → Tensor
```

`addcddiv()` 的 in-place 运算形式

```
addcmul(value=1, tensor1, tensor2) → Tensor
```

请查看 `torch.addcmul()`

```
addcmul_(value=1, tensor1, tensor2) → Tensor
```

`addcmul()` 的 in-place 运算形式

```
addmm(beta=1, mat, alpha=1, mat1, mat2) → Tensor
```

请查看 `torch.addmm()`

```
addmm_(beta=1, mat, alpha=1, mat1, mat2) → Tensor
```

`addmm()` 的 in-place 运算形式

```
addmv(beta=1, tensor, alpha=1, mat, vec) → Tensor
```

请查看 `torch.addmv()`

```
addmv_(beta=1, tensor, alpha=1, mat, vec) → Tensor
```

`addmv()` 的 in-place 运算形式

```
addr(beta=1, alpha=1, vec1, vec2) → Tensor
```

请查看 `torch.addr()`

```
addr_(beta=1, alpha=1, vec1, vec2) → Tensor
```

`addr()` 的 in-place 运算形式

```
apply_(callable) → Tensor
```

将函数 `callable` 作用于 `tensor` 中每一个元素，并将每个元素用 `callable` 函数返回值替代。

！注意：该函数只能在 CPU tensor 中使用，并且不应该用在有较高性能要求的代码块。

`asin()` → Tensor

请查看 `torch.asin()`

`asin_()` → Tensor

`asin_()` 的 in-place 运算形式

`atan()` → Tensor

请查看 `torch.atan()`

`atan2()` → Tensor

请查看 `torch.atan2()`

`atan2_()` → Tensor

`atan2_()` 的 in-place 运算形式

`atan_()` → Tensor

`atan_()` 的 in-place 运算形式

`baddbmm(beta=1, alpha=1, batch1, batch2)` → Tensor

请查看 `torch.baddbmm()`

`baddbmm_(beta=1, alpha=1, batch1, batch2)` → Tensor

`baddbmm_()` 的 in-place 运算形式

`bernoulli()` → Tensor

请查看 `torch.bernoulli()`

`bernoulli_()` → Tensor

`bernoulli_()` 的 in-place 运算形式

`bmm(batch2)` → Tensor

请查看 `torch.bmm()`

`byte()` → Tensor

将 tensor 改为 byte 类型

`bmm(median=0, sigma=1, *, generator=None)` → Tensor

将 tensor 中元素用柯西分布得到的数值填充：

$P(x) = \frac{1}{\sigma} \exp(-\frac{x - \text{median}}{\sigma})$

`ceil()` → Tensor

请查看 `torch.ceil()`

`ceil_()` → Tensor

`ceil_()` 的 in-place 运算形式

`char()`

将 tensor 元素改为 char 类型

`chunk(n_chunks, dim=0)` → Tensor

将 tensor 分割为 tensor 元组。请查看 `torch.chunk()`

`clamp(min, max)` → Tensor

请查看 `torch.clamp()`

`clamp_(min, max)` → Tensor

`clamp_()` 的 in-place 运算形式

`clone()` → Tensor

返回与原 tensor 有相同大小和数据类型的 tensor

`contiguous()` → Tensor

返回一个内存连续的有相同数据的 tensor，如果原 tensor 内存连续则返回原 tensor

`copy_(src, async=False) → Tensor`

将 `src` 中的元素复制到 tensor 中并返回这个 tensor。两个 tensor 应该有相同数目的元素，可以是不同的数据类型或存储在不同的设备上。**参数：** - `src` (Tensor)-复制的源 tensor - `async` (bool)-如果为 True 并且复制是在 CPU 和 GPU 之间进行的，则复制后的拷贝可能会与源信息异步，对于其他类型的复制操作则该参数不会发生作用。

`cos() → Tensor`

请查看 `torch.cos()`

`cos_() → Tensor`

`cos()` 的 in-place 运算形式

`cosh() → Tensor`

请查看 `torch.cosh()`

`cosh_() → Tensor`

`cosh()` 的 in-place 运算形式

`cpu() → Tensor`

如果在 CPU 上没有该 tensor，则会返回一个 CPU 的副本

`cross(other, dim=-1) → Tensor`

请查看 `torch.cross()`

`cuda(device=None, async=False)`

返回此对象在 CPU 内存中的一个副本 如果对象已近存在与 CUDA 存储中并且在正确的设备上，则不会进行复制并返回原始对象。

参数： - `device`(int)-目的 GPU 的 id，默认为当前的设备。 - `async`(bool)-如果为 True 并且资源在固定内存中，则复制的副本将会与原始数据异步。否则，该参数没有意义。

`cumprod(dim) → Tensor`

请查看 `torch.cumprod()`

`cumsum(dim) → Tensor`

请查看 `torch.cumsum()`

`data_ptr() → int`

返回 tensor 第一个元素的地址

`diag(diagonal=0) → Tensor`

请查看 `torch.diag()`

`dim() → int`

返回 tensor 的维数

`dist(other, p=2) → Tensor`

请查看 `torch.dist()`

`div(value)`

请查看 `torch.div()`

`div_(value)`

`div()` 的 in-place 运算形式

`dot(tensor2) → float`

请查看 `torch.dot()`

`double()`

将该 tensor 投射为 double 类型

`eig(eigenvectors=False) → (Tensor, Tensor)`

请查看 `torch.eig()`

`element_size()` → int

返回单个元素的字节大小。 例：

```
>>> torch.FloatTensor().element_size()
```

```
4
```

```
>>> torch.ByteTensor().element_size()
```

```
1
```

`eq(other)` → Tensor

请查看 `torch.eq()`

`eq_(other)` → Tensor

`eq()` 的 in-place 运算形式

`equal(other)` → bool

请查看 `torch.equal()`

`exp()` → Tensor

请查看 `torch.exp()`

`exp_()` → Tensor

`exp()` 的 in-place 运算形式

`expand(*sizes)`

返回 tensor 的一个新视图，单个维度扩大为更大的尺寸。`tensor` 也可以扩大为更高维，新增加的维度将附在前面。扩大 `tensor` 不需要分配新内存，只是仅仅新建一个 `tensor` 的视图，其中通过将 `stride` 设为 0，一维将会扩展位更高维。任何一个一维的在不分配新内存情况下可扩展为任意的数值。

参数： - `sizes`(`torch.Size` or `int...`)-需要扩展的大小

例：

```
>>> x = torch.Tensor([[[1], [2], [3]]])
```

```
>>> x.size()
```

```
torch.Size([3, 1])
```

```
>>> x.expand(3, 4)
```

```
1 1
```

```
1 1
```

```
2 2 2 2
```

```
3 3 3 3
```

```
[torch.FloatTensor of size 3x4]
```

`expand_as(tensor)`

将 `tensor` 扩展为参数 `tensor` 的大小。 该操作等效与：

```
self.expand(tensor.size())
```

`exponential_(lambda=1, *, generator=None)` → Tensor

将该 `tensor` 用指数分布得到的元素填充：

$$P(x) = \lambda e^{-\lambda x}$$

`fill_(value)` → Tensor

将该 `tensor` 用指定的数值填充

`float()`

将 `tensor` 投射为 `float` 类型

`floor()` → Tensor

请查看 `torch.floor()`

`floor_()` → Tensor

`floor()`的 in-place 运算形式

`fmod(divisor)` → Tensor

请查看 `torch.fmod()`

`fmod_(divisor)` → Tensor

`fmod()`的 in-place 运算形式

`frac()` → Tensor

请查看 `torch.frac()`

`frac_()` → Tensor

`frac()`的 in-place 运算形式

`gather(dim, index)` → Tensor

请查看 `torch.gather()`

`ge(other)` → Tensor

请查看 `torch.ge()`

`ge_(other)` → Tensor

`ge()`的 in-place 运算形式

`gels(A)` → Tensor

请查看 `torch.gels()`

`geometric_(p, *, generator=None)` → Tensor

将该 tensor 用几何分布得到的元素填充:

$P(X=k)=(1-p)^{k-1}p$

`geqrf()` → (Tensor, Tensor)

请查看 `torch.geqrf()`

`ger(vec2)` → Tensor

请查看 `torch.ger()`

`gesv(A)` → Tensor, Tensor

请查看 `torch.gesv()`

`gt(other)` → Tensor

请查看 `torch.gt()`

`gt_(other)` → Tensor

`gt()`的 in-place 运算形式

`half()`

将 tensor 投射为半精度浮点类型

`histc(bins=100, min=0, max=0)` → Tensor

请查看 `torch.histc()`

`index(m)` → Tensor

用一个二进制的掩码或沿着一个给定的维度从 tensor 中选取元素。`tensor.index(m)`与 `tensor[m]`完全相同。

参数: - **m**(int or Byte Tensor or slice)-用来选取元素的维度或掩码

`index_add_(dim, index, tensor)` → Tensor

按参数 index 中的索引数确定的顺序, 将参数 tensor 中的元素加到原来的 tensor 中。参数 tensor 的尺寸必须严格地与原 tensor 匹配, 否则会发生错误。

参数: - **dim**(int)-索引 index 所指向的维度 - **index**(LongTensor)-需要从 tensor 中选取的指数

- **tensor**(Tensor)-含有相加元素的 tensor

例:

```
>>> x = torch.Tensor([[[1, 1, 1], [1, 1, 1], [1, 1, 1]]])
>>> t = torch.Tensor([[[1, 2, 3], [4, 5, 6], [7, 8, 9]]])
>>> index = torch.LongTensor([0, 2, 1])
>>> x.index_add_(0, index, t)
>>> x
 2 3 4
 8 9 10
 5 6 7
[torch.FloatTensor of size 3x3]
```

`index_copy_(dim, index, tensor) → Tensor`

按参数 `index` 中的索引数确定的顺序, 将参数 `tensor` 中的元素复制到原来的 `tensor` 中。参数 `tensor` 的尺寸必须严格地与原 `tensor` 匹配, 否则会发生错误。

参数: - `dim` (int)-索引 `index` 所指向的维度 - `index` (LongTensor)-需要从 `tensor` 中选取的指数
- `tensor` (Tensor)-含有被复制元素的 `tensor`

例:

```
>>> x = torch.Tensor(3, 3)
>>> t = torch.Tensor([[[1, 2, 3], [4, 5, 6], [7, 8, 9]]])
>>> index = torch.LongTensor([0, 2, 1])
>>> x.index_copy_(0, index, t)
>>> x
 1 2 3
 7 8 9
 4 5 6
[torch.FloatTensor of size 3x3]
```

`index_fill_(dim, index, val) → Tensor`

按参数 `index` 中的索引数确定的顺序, 将原 `tensor` 用参数 `val` 值填充。

参数: - `dim` (int)-索引 `index` 所指向的维度 - `index` (LongTensor)-索引 - `val` (Tensor)-填充的值

例:

```
>>> x = torch.Tensor([[[1, 2, 3], [4, 5, 6], [7, 8, 9]]])
>>> index = torch.LongTensor([0, 2])
>>> x.index_fill_(0, index, -1)
>>> x
-1 2 -1
-1 5 -1
-1 8 -1
[torch.FloatTensor of size 3x3]
```

`index_select(dim, index) → Tensor`

请查看 `torch.index_select()`

`int()`

将该 `tensor` 投射为 `int` 类型

`inverse() → Tensor`

请查看 `torch.inverse()`

`is_contiguous() → bool`

如果该 tensor 在内存中是连续的则返回 True。

is_cuda

is_pinned()

如果该 tensor 在固定内存中则返回 True

is_set_to(tensor) → bool

如果此对象引用与 Torch C API 相同的 `THTensor` 对象作为给定的张量，则返回 True。

is_signed()

kthvalue(k, dim=None) -> (Tensor, LongTensor)

请查看 `torch.kthvalue()`

le(other) → Tensor

请查看 `torch.le()`

le_(other) → Tensor

`le()` 的 in-place 运算形式

lerp(start, end, weight)

请查看 `torch.lerp()`

lerp_(start, end, weight) → Tensor

`lerp()` 的 in-place 运算形式

log() → Tensor

请查看 `torch.log()`

loglp() → Tensor

请查看 `torch.loglp()`

loglp_() → Tensor

`loglp()` 的 in-place 运算形式

log_() → Tensor

`log()` 的 in-place 运算形式

log_normal_(mu=1, std=2, generator=None*)

将该 tensor 用均值为 μ ，标准差为 σ 的对数正态分布得到的元素填充。要注意 `mean` 和 `stdv` 是基本正态分布的均值和标准差，不是返回的分布：

$$P(X) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(\ln x - \mu)^2}{2\sigma^2}}$$

long()

将 tensor 投射为 long 类型

lt(other) → Tensor

请查看 `torch.lt()`

lt_(other) → Tensor

`lt()` 的 in-place 运算形式

map_(tensor, callable)

将 `callable` 作用于本 tensor 和参数 tensor 中的每一个元素，并将结果存放在本 tensor 中。

`callable` 应该有下列标志：

`def callable(a, b) -> number`

masked_copy_(mask, source)

将 `mask` 中值为 1 元素对应的 `source` 中位置的元素复制到本 tensor 中。`mask` 应该有和本 tensor 相同数目的元素。`source` 中元素的个数最少为 `mask` 中值为 1 的元素的个数。

参数： - `mask` (ByteTensor)-二进制掩码 - `source` (Tensor)-复制的源 tensor

注意： `mask` 作用于 `self` 自身的 tensor，而不是参数中的 `source`。

`masked_fill_(mask, value)`

在 `mask` 值为 1 的位置处用 `value` 填充。`mask` 的元素个数需和本 `tensor` 相同，但尺寸可以不同。

参数： - `mask` (ByteTensor)-二进制掩码 - `value` (Tensor)-用来填充的值

`masked_select(mask) → Tensor`

请查看 `torch.masked_select()`

`max(dim=None) -> float or(Tensor, Tensor)`

请查看 `torch.max()`

`mean(dim=None) -> float or(Tensor, Tensor)`

请查看 `torch.mean()`

`median(dim=-1, value=None, indices=None) -> (Tensor, LongTensor)`

请查看 `torch.median()`

`min(dim=None) -> float or(Tensor, Tensor)`

请查看 `torch.min()`

`mm(mat2) → Tensor`

请查看 `torch.mm()`

`mode(dim=-1, value=None, indices=None) -> (Tensor, LongTensor)`

请查看 `torch.mode()`

`mul(value) → Tensor`

请查看 `torch.mul()`

`mul_(value)`

`mul()` 的 in-place 运算形式

`multinomial(num_samples, replacement=False, , generator=None*) → Tensor`

请查看 `torch.multinomial()`

`mv(vec) → Tensor`

请查看 `torch.mv()`

`narrow(dimension, start, length) → Te`

返回一个本 `tensor` 经过缩小后的 `tensor`。维度 `dim` 缩小范围是 `start` 到 `start+length`。原 `tensor` 与返回的 `tensor` 共享相同的底层内存。

参数： - `dimension` (int)-需要缩小的维度 - `start` (int)-起始维度 - `length` (int)-

例：

```
>>> x = torch.Tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
>>> x.narrow(0, 0, 2)
```

```
 1  2  3
 4  5  6
```

```
[torch.FloatTensor of size 2x3]
```

```
>>> x.narrow(1, 1, 2)
```

```
 2  3
 5  6
 8  9
```

```
[torch.FloatTensor of size 3x2]
```

`ndimension() → int`

`dim()` 的另一种表示。

`ne(other) → Tensor`

请查看 `torch.ne()`

`ne_(other) → Tensor`

`ne_()` 的 in-place 运算形式

`neg() → Tensor`

请查看 `torch.neg()`

`neg_() → Tensor`

`neg_()` 的 in-place 运算形式

`nelement() → int`

`numel()` 的另一种表示

`new(args, *kwargs)`

构建一个有相同数据类型的 tensor

`nonezero() → LongTensor`

请查看 `torch.nonezero()`

`norm(p=2) → float`

请查看 `torch.norm()`

`normal_(mean=0, std=1, generator=None*)`

将 tensor 用均值为 `mean` 和标准差为 `std` 的正态分布填充。

`numel() → int`

请查看 `numel()`

`numpy() → ndarray`

将该 tensor 以 NumPy 的形式返回 `ndarray`，两者共享相同的底层内存。原 tensor 改变后会相应的在 `ndarray` 有反映，反之也一样。

`orgqr(input2) → Tensor`

请查看 `torch.orgqr()`

`ormqr(input2, input3, left=True, transpose=False) → Tensor`

请查看 `torch.ormqr()`

`permute(dims)`

将 tensor 的维度换位。

参数： - `*dims` (int..)-换位顺序

例：

```
>>> x = torch.randn(2, 3, 5)
```

```
>>> x.size()
```

```
torch.Size([2, 3, 5])
```

```
>>> x.permute(2, 0, 1).size()
```

```
torch.Size([5, 2, 3])
```

`pin_memory()`

如果原来没有在固定内存中，则将 tensor 复制到固定内存中。

`potrf(upper=True) → Tensor`

请查看 `torch.potrf()`

`potri(upper=True) → Tensor`

请查看 `torch.potri()`

`potrs(input2, upper=True) → Tensor`

请查看 `torch.potrs()`

`pow(exponent)`

请查看 `torch.pow()`

`pow_()`

`pow_()`的 in-place 运算形式

`prod()` → float

请查看 `torch.prod()`

`pstrf(upper=True, tol=-1) -> (Tensor, IntTensor)`

请查看 `torch.pstrf()`

`qr()`-> (Tensor, IntTensor)

请查看 `torch.qr()`

`random_(from=0, to=None, *, generator=None)`

将 `tensor` 用从在`[from, to-1]`上的正态分布或离散正态分布取样值进行填充。如果没有明确说明，则填充值仅由本 `tensor` 的数据类型限定。

`reciprocal()` → Tensor

请查看 `torch.reciprocal()`

`reciprocal_()` → Tensor

`reciprocal_()`的 in-place 运算形式

`remainder(divisor)` → Tensor

请查看 `torch.remainder()`

`remainder_(divisor)` → Tensor

`remainder_()`的 in-place 运算形式

`renorm(p, dim, maxnorm)` → Tensor

请查看 `torch.renorm()`

`renorm_(p, dim, maxnorm)` → Tensor

`renorm_()`的 in-place 运算形式

`repeat(*sizes)`

沿着指定的维度重复 `tensor`。不同于 `expand()`，本函数复制的是 `tensor` 中的数据。

参数： - `*sizes` (`torch.Size` or `int...`)-沿着每一维重复的次数

例：

```
>>> x = torch.Tensor([1, 2, 3])
```

```
>>> x.repeat(4, 2)
```

```
1  2  3  1  2  3
```

```
1  2  3  1  2  3
```

```
1  2  3  1  2  3
```

```
1  2  3  1  2  3
```

```
[torch.FloatTensor of size 4x6]
```

```
>>> x.repeat(4, 2, 1).size()
```

```
torch.Size([4, 2, 3])
```

`resize_(*sizes)`

将 `tensor` 的大小调整为指定的大小。如果元素个数比当前的内存大小大，就将底层存储大小调整为与新元素数目一致的大小。如果元素个数比当前内存小，则底层存储不会被改变。原来 `tensor` 中保存下来的元素将保持不变，但新内存将不会被初始化。

参数： - `*sizes` (`torch.Size` or `int...`)-需要调整的大小

例：

```
>>> x = torch.Tensor([[1, 2], [3, 4], [5, 6]])
```

```
>>> x.resize_(2, 2)
```

```
>>> x
```

```
1 2
```

```
3 4
```

```
[torch.FloatTensor of size 2x2]
```

`resize_as_(tensor)`

将本 `tensor` 的大小调整为与参数中的 `tensor` 相同的大小。等效于:

```
self.resize_(tensor.size())
```

`round()` → Tensor

请查看 `torch.round()`

`round_()` → Tensor

`round()` 的 in-place 运算形式

`rsqrt()` → Tensor

请查看 `torch.rsqrt()`

`rsqrt_()` → Tensor

`rsqrt()` 的 in-place 运算形式

`scatter_(input, dim, index, src) → Tensor`

将 `src` 中的所有值按照 `index` 确定的索引写入本 `tensor` 中。其中索引是根据给定的 `dimension`, `dim` 按照 `gather()` 描述的规则来确定。

注意, `index` 的值必须是在 `_0_到_(self.size(dim)-1)_` 之间,

参数: - `input` (Tensor)-源 `tensor` - `dim` (int)-索引的轴向 - `index` (LongTensor)-散射元素的索引指数 - `src` (Tensor or float)-散射的源元素

例:

```
>>> x = torch.rand(2, 5)
```

```
>>> x
```

```
0.4319 0.6500 0.4080 0.8760 0.2355
```

```
0.2609 0.4711 0.8486 0.8573 0.1029
```

```
[torch.FloatTensor of size 2x5]
```

```
>>> torch.zeros(3, 5).scatter_(0, torch.LongTensor([[0, 1, 2, 0, 0], [2, 0, 0, 1, 2]]), x)
```

```
0.4319 0.4711 0.8486 0.8760 0.2355
```

```
0.0000 0.6500 0.0000 0.8573 0.0000
```

```
0.2609 0.0000 0.4080 0.0000 0.1029
```

```
[torch.FloatTensor of size 3x5]
```

```
>>> z = torch.zeros(2, 4).scatter_(1, torch.LongTensor([[2], [3]]), 1.23)
```

```
>>> z
```

```
0.0000 0.0000 1.2300 0.0000
```

```
0.0000 0.0000 0.0000 1.2300
```

```
[torch.FloatTensor of size 2x4]
```

`select(dim, index) → Tensor or number`

按照 `index` 中选定的维度将 `tensor` 切片。如果 `tensor` 是一维的, 则返回一个数字。否则, 返回给定

维度已经被移除的 `tensor`。

参数: - `dim` (int)-切片的维度 - `index` (int)-用来选取的索引

!注意: `select()` 等效于切片。例如, `tensor.select(0, index)` 等效于 `tensor[index]`, `tensor.select(2, index)` 等效于 `tensor[:, :, index]`。

`set(source=None, storage_offset=0, size=None, stride=None)`

设置底层内存, 大小和步长。如果 `tensor` 是一个 `tensor`, 则将会与本 `tensor` 共享底层内存并且有相同的大小和步长。改变一个 `tensor` 中的元素将会反映在另一个 `tensor`。如果 `source` 是一个 `Storage`, 则将设置底层内存, 偏移量, 大小和步长。

参数: - `source` (Tensor or Storage)-用到的 `tensor` 或内存 - `storage_offset` (int)-内存的偏移量 - `size` (torch.Size)-需要的大小, 默认为源 `tensor` 的大小。 - `stride` (tuple)-需要的步长, 默认为 C 连续的步长。

`share_memory_()`

将底层内存移到共享内存中。如果底层内存已经在共享内存中是不进行任何操作。在共享内存中的 `tensor` 不能调整大小。

`short()`

将 `tensor` 投射为 `short` 类型。

`sigmoid()` → Tensor

请查看 `torch.sigmoid()`

`sigmoid_()` → Tensor

`sigmoid_()` 的 in-place 运算形式

`sign()` → Tensor

请查看 `torch.sign()`

`sign_()` → Tensor

`sign_()` 的 in-place 运算形式

`sin()` → Tensor

请查看 `torch.sin()`

`sin_()` → Tensor

`sin_()` 的 in-place 运算形式

`sinh()` → Tensor

请查看 `torch.sinh()`

`sinh_()` → Tensor

`sinh_()` 的 in-place 运算形式

`size()` → torch.Size

返回 `tensor` 的大小。返回的值是 `tuple` 的子类。

例:

```
>>> torch.Tensor(3, 4, 5).size()
```

```
torch.Size([3, 4, 5])
```

`sort(dim=None, descending=False) -> (Tensor, LongTensor)`

请查看 `torch.sort()`

`split(split_size, dim=0)`

将 `tensor` 分割成 `tensor` 数组。请查看 `torch.split()`

`sqrt()` → Tensor

请查看 `torch.sqrt()`

`sqrt_()` → Tensor

`sqrt()`的 in-place 运算形式

`squeeze(dim=None) → Tensor`

请查看 `torch.squeeze()`

`squeeze_(dim=None) → Tensor`

`squeeze()`的 in-place 运算形式

`std() → float`

请查看 `torch.std()`

`storage() → torch.Storage`

返回底层内存。

`storage_offset() → int`

以储存元素的个数的形式返回 tensor 在地域内存中的偏移量。 例：

```
>>> x = torch.Tensor([1, 2, 3, 4, 5])
```

```
>>> x.storage_offset()
```

```
0
```

```
>>> x[3:].storage_offset()
```

```
3
```

`classmethod() storage_type()`

`stride() → Tensor`

返回 tensor 的步长。

`sub(value, other) → Tensor`

从 tensor 中抽取一个标量或 tensor。如果 `value` 和 `other` 都是给定的，则在使用之前 `other` 的每一个元素都会被 `value` 缩放。

`sub_(x) → Tensor`

`sub()`的 in-place 运算形式

`sum(dim=None) → Tensor`

请查看 `torch.sum()`

`svd(some=True) -> (Tensor, Tensor, Tensor)`

请查看 `torch.svd()`

`symeig(_eigenvectors=False, upper=True) -> (Tensor, Tensor)`

请查看 `torch.symeig()`

`t() → Tensor`

请查看 `torch.t()`

`t() → Tensor`

`t()`的 in-place 运算形式

`tan() → Tensor`

请查看 `torch.tan()`

`tan_() → Tensor`

`tan()`的 in-place 运算形式

`tanh() → Tensor`

请查看 `torch.tanh()`

`tanh_() → Tensor`

`tanh()`的 in-place 运算形式

`tolist()`

返回一个 tensor 的嵌套列表表示。

`topk(k, dim=None, largest=True, sorted=True) -> (Tensor, LongTensor)`

请查看 `torch.topk()`

`trace()` → float

请查看 `torch.trace()`

`transpose(dim0, dim1) → Tensor`

请查看 `torch.transpose()`

`transpose(dim0, dim1) → Tensor`

`transpose()` 的 in-place 运算形式

`tril(k=0) → Tensor`

请查看 `torch.tril()`

`tril_(k=0) → Tensor`

`tril()` 的 in-place 运算形式

`triu(k=0) → Tensor`

请查看 `torch.triu()`

`triu(k=0) → Tensor`

`triu()` 的 in-place 运算形式

`trtrs(A, upper=True, transpose=False, unitriangular=False) -> (Tensor, Tensor)`

请查看 `torch.trtrs()`

`trunc() → Tensor`

请查看 `torch.trunc()`

`trunc() → Tensor`

`trunc()` 的 in-place 运算形式

`type(new_type=None, async=False)`

将对象投为指定的类型。 如果已经是正确的类型，则不会进行复制并返回原对象。

参数： - `new_type` (type or string)-需要的类型 - `async` (bool)-如果为 True，并且源地址在固定内存中，目的地址在 GPU 或者相反，则会相对于源异步执行复制。否则，该参数不发挥作用。

`type_as(tesnor)`

将 tensor 投射为参数给定 tensor 类型并返回。 如果 tensor 已经是正确的类型则不会执行操作。等效于：

`self.type(tensor.type())`

参数： - `tensor` (Tensor): 有所需要类型的 tensor

`unfold(dim, size, step) → Tensor`

返回一个 tensor，其中含有在 `dim` 维 tianchong 度上所有大小为 `size` 的分片。两个分片之间的步长为 `step`。 如果 `_sizedim` 是 `dim` 维度的原始大小，则在返回 tensor 中的维度 `dim` 大小是 `_(sizedim-size)/step+1` 维度大小的附加维度将附加在返回的 tensor 中。

参数： - `dim` (int)-需要展开的维度 - `size` (int)-每一个分片需要展开的大小 - `step` (int)-相邻分片之间的步长

例：

```
>>> x = torch.arange(1, 8)
```

```
>>> x
```

```
1
```

```
2
```

```
3
```

```
4
5
6
7
[torch.FloatTensor of size 7]
```

```
>>> x.unfold(0, 2, 1)
```

```
1 2
2 3
3 4
4 5
5 6
6 7
[torch.FloatTensor of size 6x2]
```

```
>>> x.unfold(0, 2, 2)
```

```
1 2
3 4
5 6
[torch.FloatTensor of size 3x2]
```

`uniform_(from=0, to=1) → Tensor`

将 `tensor` 用从均匀分布中抽样得到的值填充。

`unsqueeze(dim)`

请查看 `torch.unsqueeze()`

`unsqueeze_(dim) → Tensor`

`unsqueeze()` 的 in-place 运算形式。

`var()`

请查看 `torch.var()`

`view(*args) → Tensor`

返回一个有相同数据但大小不同的 `tensor`。返回的 `tensor` 必须有与原 `tensor` 相同的数据和相同数目的元素，但可以有不同的大小。一个 `tensor` 必须是连续的 `contiguous()` 才能被查看。

例：

```
>>> x = torch.randn(4, 4)
>>> x.size()
torch.Size([4, 4])
>>> y = x.view(16)
>>> y.size()
torch.Size([16])
>>> z = x.view(-1, 8) # the size -1 is inferred from other dimensions
>>> z.size()
torch.Size([2, 8])
view_as(tensor)
```

返回被视作与给定的 `tensor` 相同大小的原 `tensor`。 等效于：

```
self.view(tensor.size())
```

```
zero_()
```

用 0 填充该 `tensor`。



关注【深度学习爱好者】

更多学习资源免费获取

第三章 torch.Storage

一个 `torch.Storage` 是一个单一数据类型的连续一维数组。
每个 `torch.Tensor` 都有一个对应的、相同数据类型的存储。

`class torch.FloatTensor`

`byte()`

将此存储转为 `byte` 类型

`char()`

将此存储转为 `char` 类型

`clone()`

返回此存储的一个副本

`copy_()`

`cpu()`

如果当前此存储不在 CPU 上，则返回一个它的 CPU 副本

`cuda(device=None, async=False)`

返回此对象在 CUDA 内存中的一个副本。

如果此对象已在 CUDA 内存中且在正确的设备上，那么不会执行复制操作，直接返回原对象。

参数：

- **device** (`int`) - 目标 GPU 的 id。默认值是当前设备。
- **async** (`bool`) - 如果值为 `True`，且源在锁定内存中，则副本相对于宿主是异步的。否则此参数不起效果。

`data_ptr()`

`double()`

将此存储转为 `double` 类型

`element_size()`

`fill_()`

`float()`

将此存储转为 `float` 类型

`from_buffer()`

`half()`

将此存储转为 `half` 类型

`int()`

将此存储转为 `int` 类型

`is_cuda = False`

`is_pinned()`

`is_shared()`

`is_sparse = False`

`long()`

将此存储转为 `long` 类型

`new()`

`pin_memory()`

如果此存储当前未被锁定，则将它复制到锁定内存中。

`resize_()`

`share_memory_()`

将此存储移动到共享内存中。

对于已经在共享内存中的存储或者 CUDA 存储，这是一条空指令，它们不需要移动就能在进程间共享。共享内存中的存储不能改变大小。

返回：self

`short()`

将此存储转为 short 类型

`size()`

`tolist()`

返回一个包含此存储中元素的列表

`type(new_type=None, async=False)`

将此对象转为指定类型。

如果已经是正确类型，不会执行复制操作，直接返回原对象。

参数：

- **new_type** (**type** or **string**) -需要转成的类型
- **async** (**bool**) -如果值为 True，且源在锁定内存中而目标在 GPU 中——或正好相反，则复制操作相对于宿主异步执行。否则此参数不起效果。



关注【深度学习爱好者】

更多学习资源免费获取

第四章 torch.nn

Parameters

`class torch.nn.Parameter()`

`Variable` 的一种，常被用于模块参数(module parameter)。

`Parameters` 是 `Variable` 的子类。`Parameters` 和 `Modules` 一起使用的时候会有一些特殊的属性，即：当 `Parameters` 赋值给 `Module` 的属性的时候，他会自动的被加到 `Module` 的参数列表中(即：会出现在 `parameters()` 迭代器中)。将 `Variable` 赋值给 `Module` 属性则不会有这样的影响。这样做的原因是：我们有时候会需要缓存一些临时的状态(state)，比如：模型中 `RNN` 的最后一个隐状态。如果没有 `Parameter` 这个类的话，那么这些临时变量也会注册成为模型变量。

`Variable` 与 `Parameter` 的另一个不同之处在于，`Parameter` 不能被 `volatile`(即：无法设置 `volatile=True`) 而且默认 `requires_grad=True`。`Variable` 默认 `requires_grad=False`。

参数说明：

- data (Tensor) – parameter tensor.
- requires_grad (bool, optional) – 默认为 `True`，在 `BP` 的过程中会对其求微分。

Containers（容器）：

`class torch.nn.Module`

所有网络的基类。

你的模型也应该继承这个类。

`Modules` 也可以包含其它 `Modules`，允许使用树结构嵌入他们。你可以将子模块赋值给模型属性。

```
import torch.nn as nn
```

```
import torch.nn.functional as F
```

```
class Model(nn.Module):
```

```
    def __init__(self):
```

```
        super(Model, self).__init__()
```

```
        self.conv1 = nn.Conv2d(1, 20, 5)# submodule: Conv2d
```

```
        self.conv2 = nn.Conv2d(20, 20, 5)
```

```
    def forward(self, x):
```

```
        x = F.relu(self.conv1(x))
```

```
        return F.relu(self.conv2(x))
```

通过上面方式赋值的 `submodule` 会被注册。当调用 `.cuda()` 的时候，`submodule` 的参数也会转换为 `cuda Tensor`。

`add_module(name, module)`

将一个 `child module` 添加到当前 `module`。被添加的 `module` 可以通过 `name` 属性来获取。例：

```
import torch.nn as nn
```

```
class Model(nn.Module):
```

```
    def __init__(self):
```

```
        super(Model, self).__init__()
```

```
        self.add_module("conv", nn.Conv2d(10, 20, 4))
```

```
        #self.conv = nn.Conv2d(10, 20, 4) 和上面这个增加 module 的方式等价
```

```
model = Model()
```

```
print(model.conv)
```

输出:

```
Conv2d(10, 20, kernel_size=(4, 4), stride=(1, 1))
```

```
children()
```

Returns an iterator over immediate children modules. 返回当前模型 子模块的迭代器。

```
import torch.nn as nn
```

```
class Model(nn.Module):
```

```
    def __init__(self):
```

```
        super(Model, self).__init__()
```

```
        self.add_module("conv", nn.Conv2d(10, 20, 4))
```

```
        self.add_module("conv1", nn.Conv2d(20, 10, 4))
```

```
model = Model()
```

```
for sub_module in model.children():
```

```
    print(sub_module)
```

```
Conv2d(10, 20, kernel_size=(4, 4), stride=(1, 1))
```

```
Conv2d(20, 10, kernel_size=(4, 4), stride=(1, 1))
```

```
cpu(device_id=None)
```

将所有的模型参数(parameters)和 buffers 复制到 CPU

NOTE: 官方文档用的 move, 但我觉着 copy 更合理。

```
cuda(device_id=None)
```

将所有的模型参数(parameters)和 buffers 赋值 GPU

参数说明:

- device_id (int, optional) – 如果指定的话, 所有的模型参数都会复制到指定的设备上。

```
double()
```

将 parameters 和 buffers 的数据类型转换成 double。

```
eval()
```

将模型设置成 evaluation 模式

仅仅当模型中有 Dropout 和 BatchNorm 是才会有影响。

```
float()
```

将 parameters 和 buffers 的数据类型转换成 float。

```
forward(* input)
```

定义了每次执行的 计算步骤。 在所有的子类中都需要重写这个函数。

```
half()
```

将 parameters 和 buffers 的数据类型转换成 half。

```
load_state_dict(state_dict)
```

将 state_dict 中的 parameters 和 buffers 复制到此 module 和它的后代中。state_dict 中的 key 必须和 model.state_dict() 返回的 key 一致。NOTE: 用来加载模型参数。

参数说明:

- state_dict (dict) – 保存 parameters 和 persistent buffers 的字典。

```
modules()
```

返回一个包含 当前模型 所有模块的迭代器。

```
import torch.nn as nn
```

```
class Model(nn.Module):
```

```

def __init__(self):
    super(Model, self).__init__()
    self.add_module("conv", nn.Conv2d(10, 20, 4))
    self.add_module("conv1", nn.Conv2d(20, 10, 4))
model = Model()

```

```

for module in model.modules():
    print(module)
Model (
  (conv): Conv2d(10, 20, kernel_size=(4, 4), stride=(1, 1))
  (conv1): Conv2d(20, 10, kernel_size=(4, 4), stride=(1, 1))
)
Conv2d(10, 20, kernel_size=(4, 4), stride=(1, 1))
Conv2d(20, 10, kernel_size=(4, 4), stride=(1, 1))

```

可以看出，`modules()`返回的 `iterator` 不止包含 子模块。这是和 `children()`的不同。

NOTE: 重复的模块只被返回一次(`children()`也是)。 在下面的例子中，`submodule` 只会被返回一次：

```
import torch.nn as nn
```

```

class Model(nn.Module):
    def __init__(self):
        super(Model, self).__init__()
        submodule = nn.Conv2d(10, 20, 4)
        self.add_module("conv", submodule)
        self.add_module("conv1", submodule)
model = Model()

```

```

for module in model.modules():
    print(module)
Model (
  (conv): Conv2d(10, 20, kernel_size=(4, 4), stride=(1, 1))
  (conv1): Conv2d(10, 20, kernel_size=(4, 4), stride=(1, 1))
)
Conv2d(10, 20, kernel_size=(4, 4), stride=(1, 1))

```

`named_children()`

返回 包含 模型当前子模块 的迭代器，`yield` 模块名字和模块本身。

例子：

```

for name, module in model.named_children():
    if name in ['conv4', 'conv5']:
        print(module)

```

`named_modules(memo=None, prefix="")[source]`

返回包含网络中所有模块的迭代器，`yielding` 模块名和模块本身。

注意：

重复的模块只被返回一次(`children()`也是)。 在下面的例子中，`submodule` 只会被返回一次。

```
parameters(memo=None)
```

返回一个 包含模型所有参数 的迭代器。

一般用来当作 `optimizer` 的参数。

例子：

```
for param in model.parameters():
    print(type(param.data), param.size())
```

```
<class 'torch.FloatTensor'> (20L,)
```

```
<class 'torch.FloatTensor'> (20L, 1L, 5L, 5L)
```

`register_backward_hook(hook)`

在 `module` 上注册一个 `backward hook`。

每次计算 `module` 的 `inputs` 的梯度的时候，这个 `hook` 会被调用。`hook` 应该拥有下面的 `signature`。

`hook(module, grad_input, grad_output) -> Variable or None`

如果 `module` 有多个输入输出的话，那么 `grad_input` `grad_output` 将会是个 `tuple`。`hook` 不应该修改它的 `arguments`，但是它可以选择性的返回关于输入的梯度，这个返回的梯度在后续的计算中会替代 `grad_input`。

这个函数返回一个句柄(`handle`)。它有一个方法 `handle.remove()`，可以用这个方法将 `hook` 从 `module` 移除。

`register_buffer(name, tensor)`

给 `module` 添加一个 `persistent buffer`。

`persistent buffer` 通常被用在这么一种情况：我们需要保存一个状态，但是这个状态不能看作为模型参数。例如：，`BatchNorm's running_mean` 不是一个 `parameter`，但是它也是需要保存的状态之一。

`Buffers` 可以通过注册时候的 `name` 获取。

NOTE:我们可以用 `buffer` 保存 `moving average`

例子：

```
self.register_buffer('running_mean', torch.zeros(num_features))
```

```
self.running_mean
```

`register_forward_hook(hook)`

在 `module` 上注册一个 `forward hook`。每次调用 `forward()` 计算输出的时候，这个 `hook` 就会被调用。

它应该拥有以下签名：

`hook(module, input, output) -> None`

`hook` 不应该修改 `input` 和 `output` 的值。这个函数返回一个 句柄(`handle`)。它有一个方法 `handle.remove()`，可以用这个方法将 `hook` 从 `module` 移除。

`register_parameter(name, param)`

向 `module` 添加 `parameter`

`parameter` 可以通过注册时候的 `name` 获取。

`state_dict(destination=None, prefix="")[source]`

返回一个字典，保存着 `module` 的所有状态 (`state`)。

`parameters` 和 `persistent buffers` 都会包含在字典中，字典的 `key` 就是 `parameter` 和 `buffer` 的 `names`。

例子：

```
import torch
```

```
from torch.autograd import Variable
```

```
import torch.nn as nn
```

```
class Model(nn.Module):
    def __init__(self):
        super(Model, self).__init__()
        self.conv2 = nn.Linear(1, 2)
        self.vari = Variable(torch.rand([1]))
        self.par = nn.Parameter(torch.rand([1]))
        self.register_buffer("buffer", torch.randn([2,3]))
```

```
model = Model()
print(model.state_dict().keys())
```

```
odict_keys(['par', 'buffer', 'conv2.weight', 'conv2.bias'])
```

```
train(mode=True)
```

将 `module` 设置为 `training mode`。

仅仅当模型中有 `Dropout` 和 `BatchNorm` 是才会有影响。

```
zero_grad()
```

将 `module` 中的所有模型参数的梯度设置为 0。

```
class torch.nn.Sequential(* args)
```

一个时序容器。`Modules` 会以他们传入的顺序被添加到容器中。当然，也可以传入一个 `OrderedDict`。

为了更好的理解如何使用 `Sequential`，下面给出了一个例子：

```
# Example of using Sequential
```

```
model = nn.Sequential(
    nn.Conv2d(1,20,5),
    nn.ReLU(),
    nn.Conv2d(20,64,5),
    nn.ReLU()
)
```

```
# Example of using Sequential with OrderedDict
```

```
model = nn.Sequential(OrderedDict([
    ('conv1', nn.Conv2d(1,20,5)),
    ('relu1', nn.ReLU()),
    ('conv2', nn.Conv2d(20,64,5)),
    ('relu2', nn.ReLU())
]))
```

```
class torch.nn.ModuleList(modules=None)[source]
```

将 `submodules` 保存在一个 `list` 中。

`ModuleList` 可以像一般的 `Python list` 一样被索引。而且 `ModuleList` 中包含的 `modules` 已经被正确的注册，对所有的 `module method` 可见。

参数说明：

- `modules (list, optional)` – 将要被添加到 `ModuleList` 中的 `modules` 列表

例子：

```
class MyModule(nn.Module):
    def __init__(self):
```

```
super(MyModule, self).__init__()
self.linears = nn.ModuleList([nn.Linear(10, 10) for i in range(10)])
```

```
def forward(self, x):
    # ModuleList can act as an iterable, or be indexed using ints
    for i, l in enumerate(self.linears):
        x = self.linears[i // 2](x) + l(x)
    return x
```

`append(module)[source]`

等价于 `list` 的 `append()`

参数说明:

- `module (nn.Module)` – 要 `append` 的 `module`

`extend(modules)[source]`

等价于 `list` 的 `extend()` 方法

参数说明:

- `modules (list)` – list of modules to append

`class torch.nn.ParameterList(parameters=None)`

将 `submodules` 保存在一个 `list` 中。

`ParameterList` 可以像一般的 `Python list` 一样被索引。而且 `ParameterList` 中包含的 `parameters` 已经被正确的注册，对所有的 `module method` 可见。

参数说明:

- `modules (list, optional)` – a list of `nn.Parameter`

例子:

```
class MyModule(nn.Module):
    def __init__(self):
        super(MyModule, self).__init__()
        self.params = nn.ParameterList([nn.Parameter(torch.randn(10, 10)) for i in range(10)])
```

```
def forward(self, x):
    # ModuleList can act as an iterable, or be indexed using ints
    for i, p in enumerate(self.params):
        x = self.params[i // 2].mm(x) + p.mm(x)
    return x
```

`append(parameter)[source]`

等价于 `python list` 的 `append` 方法。

参数说明:

- `parameter (nn.Parameter)` – parameter to append

`extend(parameters)[source]`

等价于 `python list` 的 `extend` 方法。

参数说明:

- `parameters (list)` – list of parameters to append

卷积层

`class torch.nn.Conv1d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True)`

一维卷积层，输入的尺度是(N, C_in, L)，输出尺度 (N, C_out, L_out) 的计算方式：

$$out(N_i, C_{out_j}) = bias(C_{out_j}) + \sum_{k=0}^{C_{in}-1} weight(C_{out_j}, k) \cdot input(N_i, k)$$

说明

bigotimes: 表示相关系数计算

stride: 控制相关系数的计算步长

dilation: 用于控制内核点之间的距离，详细描述在[这里](#)

groups: 控制输入和输出之间的连接，**group=1**，输出是所有的输入的卷积；**group=2**，此时相当于有并排的两个卷积层，每个卷积层计算输入通道的一半，并且产生的输出是输出通道的一半，随后将这两个输出连接起来。

Parameters:

- **in_channels(int)** – 输入信号的通道
- **out_channels(int)** – 卷积产生的通道
- **kernel_size(int or tuple)** - 卷积核的尺寸
- **stride(int or tuple, optional)** - 卷积步长
- **padding (int or tuple, optional)**- 输入的每一条边补充 0 的层数
- **dilation(int or tuple, `optional`)** – 卷积核元素之间的间距
- **groups(int, optional)** – 从输入通道到输出通道的阻塞连接数
- **bias(bool, optional)** - 如果 **bias=True**，添加偏置

shape:

输入: (N, C_in, L_in)

输出: (N, C_out, L_out)

输入输出的计算方式：

$$L_{out} = \text{floor}((L_{in} + 2padding - dilation(kernel_size - 1) - 1) / stride + 1)$$

变量:

weight(tensor) - 卷积的权重，大小是(out_channels, in_channels, kernel_size)

bias(tensor) - 卷积的偏置系数，大小是 (out_channel)

example:

```
>>> m = nn.Conv1d(16, 33, 3, stride=2)
```

```
>>> input = autograd.Variable(torch.randn(20, 16, 50))
```

```
>>> output = m(input)
```

```
class torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True)
```

二维卷积层，输入的尺度是(N, C_in, H, W)，输出尺度 (N, C_out, H_out, W_out) 的计算方式：

$$out(N_i, C_{out_j}) = bias(C_{out_j}) + \sum_{k=0}^{C_{in}-1} weight(C_{out_j}, k) \cdot input(N_i, k)$$

说明

bigotimes: 表示二维的相关系数计算 **stride**: 控制相关系数的计算步长

dilation: 用于控制内核点之间的距离，详细描述在[这里](#)

groups: 控制输入和输出之间的连接：**group=1**，输出是所有的输入的卷积；**group=2**，此时相当于有并排的两个卷积层，每个卷积层计算输入通道的一半，并且产生的输出是输出通道的一半，随后将这两个输出连接起来。

参数 **kernel_size**, **stride**, **padding**, **dilation** 也可以是一个 **int** 的数据，此时卷积 **height** 和 **width** 值相同；也可以是一个 **tuple** 数组，**tuple** 的第一维度表示 **height** 的数值，**tuple** 的第二维度表示 **width** 的数值

Parameters:

- `in_channels(int)` - 输入信号的通道
- `out_channels(int)` - 卷积产生的通道
- `kernel_size(int or tuple)` - 卷积核的尺寸
- `stride(int or tuple, optional)` - 卷积步长
- `padding(int or tuple, optional)` - 输入的每一条边补充 0 的层数
- `dilation(int or tuple, optional)` - 卷积核元素之间的间距
- `groups(int, optional)` - 从输入通道到输出通道的阻塞连接数
- `bias(bool, optional)` - 如果 `bias=True`, 添加偏置

shape:

input: (N,C_in,H_in,W_in)

output: (N,C_out,H_out,W_out)

$H_{out} = \text{floor}((H_{in} + 2padding[0] - dilation[0](kernel_size[0] - 1) - 1) / stride[0] + 1)$

$W_{out} = \text{floor}((W_{in} + 2padding[1] - dilation[1](kernel_size[1] - 1) - 1) / stride[1] + 1)$

变量:

`weight(tensor)` - 卷积的权重, 大小是 `(out_channels, in_channels, kernel_size)`

`bias(tensor)` - 卷积的偏置系数, 大小是 `(out_channel)`

example:

```
>>> # With square kernels and equal stride
>>> m = nn.Conv2d(16, 33, 3, stride=2)
>>> # non-square kernels and unequal stride and with padding
>>> m = nn.Conv2d(16, 33, (3, 5), stride=(2, 1), padding=(4, 2))
>>> # non-square kernels and unequal stride and with padding and dilation
>>> m = nn.Conv2d(16, 33, (3, 5), stride=(2, 1), padding=(4, 2), dilation=(3, 1))
>>> input = autograd.Variable(torch.randn(20, 16, 50, 100))
>>> output = m(input)
```

`class torch.nn.Conv3d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True)`

三维卷积层, 输入的尺度是 `(N, C_in, D, H, W)`, 输出尺度 `(N, C_out, D_out, H_out, W_out)` 的计算方式:

$$out(N_i, C_{out_j}) = bias(C_{out_j}) + \sum^{C_{in}-1}_{k=0} weight(C_{out_j}, k) \cdot input(N_i, k)$$

说明

`bigotimes`: 表示二维的相关系数计算 `stride`: 控制相关系数的计算步长

`dilation`: 用于控制内核点之间的距离, 详细描述在[这里](#)

`groups`: 控制输入和输出之间的连接: `group=1`, 输出是所有的输入的卷积; `group=2`, 此时相当于有并排的两个卷积层, 每个卷积层计算输入通道的一半, 并且产生的输出是输出通道的一半, 随后将这两个输出连接起来。

参数 `kernel_size`, `stride`, `padding`, `dilation` 可以是一个 `int` 的数据 - 卷积 `height` 和 `width` 值相同, 也可以是一个有三个 `int` 数据的 `tuple` 数组, `tuple` 的第一维度表示 `depth` 的数值, `tuple` 的第二维度表示 `height` 的数值, `tuple` 的第三维度表示 `width` 的数值

Parameters:

- `in_channels(int)` - 输入信号的通道
- `out_channels(int)` - 卷积产生的通道
- `kernel_size(int or tuple)` - 卷积核的尺寸

- `stride(int or tuple, optional)` - 卷积步长
- `padding(int or tuple, optional)` - 输入的每一条边补充 0 的层数
- `dilation(int or tuple, optional)` - 卷积核元素之间的间距
- `groups(int, optional)` - 从输入通道到输出通道的阻塞连接数
- `bias(bool, optional)` - 如果 `bias=True`, 添加偏置

shape:

input: (N,C_in,D_in,H_in,W_in)

output: (N,C_out,D_out,H_out,W_out)

$D_{out} = \text{floor}((D_{in} + 2padding[0] - dilation[0](kernel_size[0] - 1) - 1) / stride[0] + 1)$

$H_{out} = \text{floor}((H_{in} + 2padding[1] - dilation[2](kernel_size[1] - 1) - 1) / stride[1] + 1)$

$W_{out} = \text{floor}((W_{in} + 2padding[2] - dilation[2](kernel_size[2] - 1) - 1) / stride[2] + 1)$

变量:

- `weight(tensor)` - 卷积的权重, shape 是 `(out_channels, in_channels, kernel_size)`
- `bias(tensor)` - 卷积的偏置系数, shape 是 `(out_channel)`

example:

```
>>> # With square kernels and equal stride
>>> m = nn.Conv3d(16, 33, 3, stride=2)
>>> # non-square kernels and unequal stride and with padding
>>> m = nn.Conv3d(16, 33, (3, 5, 2), stride=(2, 1, 1), padding=(4, 2, 0))
>>> input = autograd.Variable(torch.randn(20, 16, 10, 50, 100))
>>> output = m(input)
```

`class torch.nn.ConvTranspose1d(in_channels, out_channels, kernel_size, stride=1, padding=0, output_padding=0, groups=1, bias=True)`

1 维的解卷积操作 (`transposed convolution operator`, 注意改视作操作可视为解卷积操作, 但并不是真正的解卷积操作) 该模块可以看作是 `Conv1d` 相对于其输入的梯度, 有时 (但不正确地) 被称为解卷积操作。

注意

由于内核的大小, 输入的最后的一些列的数据可能会丢失。因为输入和输出是不是完全的互相关。因此, 用户可以进行适当的填充 (`padding` 操作)。

参数

- `in_channels(int)` - 输入信号的通道数
- `out_channels(int)` - 卷积产生的通道
- `kernel_size(int or tuple)` - 卷积核的大小
- `stride(int or tuple, optional)` - 卷积步长
- `padding(int or tuple, optional)` - 输入的每一条边补充 0 的层数
- `output_padding(int or tuple, optional)` - 输出的每一条边补充 0 的层数
- `dilation(int or tuple, optional)` - 卷积核元素之间的间距
- `groups(int, optional)` - 从输入通道到输出通道的阻塞连接数
- `bias(bool, optional)` - 如果 `bias=True`, 添加偏置

shape:

输入: (N,C_in,L_in)

输出: (N,C_out,L_out)

$L_{out} = (L_{in} - 1)stride - 2padding + kernel_size + output_padding$

变量:

- weight(tensor) - 卷积的权重，大小是(in_channels, in_channels, kernel_size)

- bias(tensor) - 卷积的偏置系数，大小是(out_channel)

class torch.nn.ConvTranspose2d(in_channels, out_channels, kernel_size, stride=1, padding=0, output_padding=0, groups=1, bias=True)

2 维的转置卷积操作（transposed convolution operator，注意改视作操作可视作解卷积操作，但并不是真正的解卷积操作）该模块可以看作是 Conv2d 相对于其输入的梯度，有时（但不正确地）被称为解卷积操作。

说明

stride: 控制相关系数的计算步长

dilation: 用于控制内核点之间的距离，详细描述在[这里](#)

groups: 控制输入和输出之间的连接：group=1，输出是所有的输入的卷积；group=2，此时相当于有并排的两个卷积层，每个卷积层计算输入通道的一半，并且产生的输出是输出通道的一半，随后将这两个输出连接起来。

参数 kernel_size, stride, padding, dilation 数据类型： 可以是一个 int 类型的数据，此时卷积 height 和 width 值相同；也可以是一个 tuple 数组（包含来两个 int 类型的数据），第一个 int 数据表示 height 的数值，第二个 int 类型的数据表示 width 的数值

注意

由于内核的大小，输入的最后的一些列的数据可能会丢失。因为输入和输出是不是完全的互相关。因此，用户可以进行适当的填充（padding 操作）。

参数：

- in_channels(int) - 输入信号的通道数
- out_channels(int) - 卷积产生的通道数
- kernel_size(int or tuple) - 卷积核的大小
- stride(int or tuple, optional) - 卷积步长
- padding(int or tuple, optional) - 输入的每一条边补充 0 的层数
- output_padding(int or tuple, optional) - 输出的每一条边补充 0 的层数
- dilation(int or tuple, optional) - 卷积核元素之间的间距
- groups(int, optional) - 从输入通道到输出通道的阻塞连接数
- bias(bool, optional) - 如果 bias=True，添加偏置

shape:

输入: (N,C_in,H_in, W_in)

输出: (N,C_out,H_out,W_out)

$H_{out} = (H_{in} - 1) \cdot stride[0] - 2 \cdot padding[0] + kernel_size[0] + output_padding[0]$

$W_{out} = (W_{in} - 1) \cdot stride[1] - 2 \cdot padding[1] + kernel_size[1] + output_padding[1]$

变量:

- weight(tensor) - 卷积的权重，大小是(in_channels, in_channels, kernel_size)

- bias(tensor) - 卷积的偏置系数，大小是 (out_channel)

Example

```
>>> # With square kernels and equal stride
```

```
>>> m = nn.ConvTranspose2d(16, 33, 3, stride=2)
```

```
>>> # non-square kernels and unequal stride and with padding
```

```
>>> m = nn.ConvTranspose2d(16, 33, (3, 5), stride=(2, 1), padding=(4, 2))
```

```
>>> input = autograd.Variable(torch.randn(20, 16, 50, 100))
```

```
>>> output = m(input)
```

```
>>> # exact output size can be also specified as an argument
>>> input = autograd.Variable(torch.randn(1, 16, 12, 12))
>>> downsample = nn.Conv2d(16, 16, 3, stride=2, padding=1)
>>> upsample = nn.ConvTranspose2d(16, 16, 3, stride=2, padding=1)
>>> h = downsample(input)
>>> h.size()
torch.Size([1, 16, 6, 6])
>>> output = upsample(h, output_size=input.size())
>>> output.size()
torch.Size([1, 16, 12, 12])
```

`torch.nn.ConvTranspose3d(in_channels, out_channels, kernel_size, stride=1, padding=0, output_padding=0, groups=1, bias=True)`

3 维的转置卷积操作（[transposed convolution operator](#)，注意改视作操作可视为解卷积操作，但并不是真正的解卷积操作）转置卷积操作将每个输入值和一个可学习权重的卷积核相乘，输出所有输入通道的求和

该模块可以看作是 `Conv3d` 相对于其输入的梯度，有时（但不正确地）被称为解卷积操作。

说明

stride: 控制相关系数的计算步长

dilation: 用于控制内核点之间的距离，详细描述在[这里](#)

groups: 控制输入和输出之间的连接：`group=1`，输出是所有的输入的卷积；`group=2`，此时相当于有并排的两个卷积层，每个卷积层计算输入通道的一半，并且产生的输出是输出通道的一半，随后将这两个输出连接起来。

参数 `kernel_size`, `stride`, `padding`, `dilation` 数据类型：一个 `int` 类型的数据，此时卷积 `height` 和 `width` 值相同；也可以是一个 `tuple` 数组（包含来两个 `int` 类型的数据），第一个 `int` 数据表示 `height` 的数值，`tuple` 的第二个 `int` 类型的数据表示 `width` 的数值

注意

由于内核的大小，输入的最后的一些列的数据可能会丢失。因为输入和输出是不是完全的互相关。因此，用户可以进行适当的填充（`padding` 操作）。

参数：

- `in_channels(int)` – 输入信号的通道数
- `out_channels(int)` – 卷积产生的通道数
- `kernel_size(int or tuple)` - 卷积核的大小
- `stride(int or tuple, optional)` - 卷积步长
- `padding(int or tuple, optional)` - 输入的每一条边补充 0 的层数
- `output_padding(int or tuple, optional)` - 输出的每一条边补充 0 的层数
- `dilation(int or tuple, optional)` – 卷积核元素之间的间距
- `groups(int, optional)` – 从输入通道到输出通道的阻塞连接数
- `bias(bool, optional)` - 如果 `bias=True`，添加偏置

shape:

输入: (N,C_in,H_in, W_in)

输出: (N,C_out,H_out,W_out)

$D_{out} = (D_{in} - 1) \cdot stride[0] - 2 \cdot padding[0] + kernel_size[0] + output_padding[0]$

$H_{out} = (H_{in} - 1) \cdot stride[1] - 2 \cdot padding[1] + kernel_size[1] + output_padding[0]$

$W_{out} = (W_{in} - 1) \cdot stride[2] - 2 \cdot padding[2] + kernel_size[2] + output_padding[2]$

变量:

- weight(tensor) - 卷积的权重, 大小是(in_channels, in_channels, kernel_size)

- bias(tensor) - 卷积的偏置系数, 大小是(out_channel)

Example

```
>>> # With square kernels and equal stride
>>> m = nn.ConvTranspose3d(16, 33, 3, stride=2)
>>> # non-square kernels and unequal stride and with padding
>>> m = nn.Conv3d(16, 33, (3, 5, 2), stride=(2, 1, 1), padding=(0, 4, 2))
>>> input = autograd.Variable(torch.randn(20, 16, 10, 50, 100))
>>> output = m(input)
```

池化层

class torch.nn.MaxPool1d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False, ceil_mode=False)

对于输入信号的输入通道, 提供 1 维最大池化(max pooling)操作

如果输入的大小是(N,C,L), 那么输出的大小是(N,C,L_out)的计算方式是:

$$out(N_i, C_j, k) = \max_{m=0}^{kernel_size-1} input(N_i, C_j, stride*k+m)$$

如果 padding 不是 0, 会在输入的每一边添加相应数目 0

dilation 用于控制内核点之间的距离, 详细描述在[这里](#)

参数:

- kernel_size(int or tuple) - max pooling 的窗口大小
- stride(int or tuple, optional) - max pooling 的窗口移动的步长。默认值是 kernel_size
- padding(int or tuple, optional) - 输入的每一条边补充 0 的层数
- dilation(int or tuple, optional) - 一个控制窗口中元素步幅的参数
- return_indices - 如果等于 True, 会返回输出最大值的序号, 对于上采样操作会有帮助
- ceil_mode - 如果等于 True, 计算输出信号大小的时候, 会使用向上取整, 代替默认的向下取整的操作

shape:

输入: (N,C_in,L_in)

输出: (N,C_out,L_out)

$$L_{out} = \lfloor (L_{in} + 2padding - dilation(kernel_size - 1) - 1) / stride + 1 \rfloor$$

example:

```
>>> # pool of size=3, stride=2
>>> m = nn.MaxPool1d(3, stride=2)
>>> input = autograd.Variable(torch.randn(20, 16, 50))
>>> output = m(input)
```

class torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False, ceil_mode=False)

对于输入信号的输入通道, 提供 2 维最大池化(max pooling)操作

如果输入的大小是(N,C,H,W), 那么输出的大小是(N,C,H_out,W_out)和池化窗口大小(kH,kW)的关系是:

$$out(N_i, C_j, k) = \max_{m=0}^{kH-1} \max_{n=0}^{kW-1} input(N_i, C_j, stride[0]h+m, stride[1]w+n)$$

如果 padding 不是 0, 会在输入的每一边添加相应数目 0

dilation 用于控制内核点之间的距离, 详细描述在[这里](#)

参数 kernel_size, stride, padding, dilation 数据类型: 可以是一个 int 类型的数据, 此时卷积 height

和 width 值相同; 也可以是一个 `tuple` 数组 (包含来两个 `int` 类型的数据), 第一个 `int` 数据表示 height 的数值, `tuple` 的第二个 `int` 类型的数据表示 width 的数值

参数:

- `kernel_size(int or tuple)` - max pooling 的窗口大小
- `stride(int or tuple, optional)` - max pooling 的窗口移动的步长。默认值是 `kernel_size`
- `padding(int or tuple, optional)` - 输入的每一条边补充 0 的层数
- `dilation(int or tuple, optional)` - 一个控制窗口中元素步幅的参数
- `return_indices` - 如果等于 `True`, 会返回输出最大值的序号, 对于上采样操作会有帮助
- `ceil_mode` - 如果等于 `True`, 计算输出信号大小的时候, 会使用向上取整, 代替默认的向下取整的操作

shape:

输入: (N,C,H_in,W_in)

输出: (N,C,H_out,W_out)

$H_{out} = \text{floor}((H_{in} + 2padding[0] - dilation[0](kernel_size[0] - 1) - 1) / stride[0] + 1)$

$W_{out} = \text{floor}((W_{in} + 2padding[1] - dilation[1](kernel_size[1] - 1) - 1) / stride[1] + 1)$

example:

```
>>> # pool of square window of size=3, stride=2
```

```
>>> m = nn.MaxPool2d(3, stride=2)
```

```
>>> # pool of non-square window
```

```
>>> m = nn.MaxPool2d((3, 2), stride=(2, 1))
```

```
>>> input = autograd.Variable(torch.randn(20, 16, 50, 32))
```

```
>>> output = m(input)
```

`class torch.nn.MaxPool3d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False, ceil_mode=False)`

对于输入信号的输入通道, 提供 3 维最大池化 (max pooling) 操作

如果输入的大小是 (N,C,D,H,W), 那么输出的大小是 (N,C,D,H_out,W_out) 和池化窗口大小 (kD,kH,kW) 的关系是:

$H_{out}(N_i, C_j, d, h, w) = \max^{kD-1}_{m=0} \max^{kH-1}_{m=0} \max^{kW-1}_{m=0} \{input(N_i, C_j, stride[0]k+d, stride[1]h+m, stride[2]w+n)\}$

如果 `padding` 不是 0, 会在输入的每一边添加相应数目 0

`dilation` 用于控制内核点之间的距离, 详细描述在[这里](#)

参数 `kernel_size, stride, padding, dilation` 数据类型: 可以是 `int` 类型的数据, 此时卷积 height 和 width 值相同; 也可以是一个 `tuple` 数组 (包含来两个 `int` 类型的数据), 第一个 `int` 数据表示 height 的数值, `tuple` 的第三个 `int` 类型的数据表示 width 的数值

参数:

- `kernel_size(int or tuple)` - max pooling 的窗口大小
- `stride(int or tuple, optional)` - max pooling 的窗口移动的步长。默认值是 `kernel_size`
- `padding(int or tuple, optional)` - 输入的每一条边补充 0 的层数
- `dilation(int or tuple, optional)` - 一个控制窗口中元素步幅的参数
- `return_indices` - 如果等于 `True`, 会返回输出最大值的序号, 对于上采样操作会有帮助
- `ceil_mode` - 如果等于 `True`, 计算输出信号大小的时候, 会使用向上取整, 代替默认的向下取整的操作

shape:

输入: (N,C,H_in,W_in)

输出: (N,C,H_out,W_out)

$$D_{out} = \text{floor}((D_{in} + 2padding[0] - dilation[0](kernel_size[0] - 1) - 1) / stride[0] + 1)$$
$$H_{out} = \text{floor}((H_{in} + 2padding[1] - dilation[1](kernel_size[0] - 1) - 1) / stride[1] + 1)$$
$$W_{out} = \text{floor}((W_{in} + 2padding[2] - dilation[2](kernel_size[2] - 1) - 1) / stride[2] + 1)$$

example:

```
>>> # pool of square window of size=3, stride=2
```

```
>>> m = nn.MaxPool3d(3, stride=2)
```

```
>>> # pool of non-square window
```

```
>>> m = nn.MaxPool3d((3, 2, 2), stride=(2, 1, 2))
```

```
>>> input = autograd.Variable(torch.randn(20, 16, 50, 44, 31))
```

```
>>> output = m(input)
```

```
class torch.nn.MaxUnpool1d(kernel_size, stride=None, padding=0)
```

Maxpool1d 的逆过程，不过并不是完全的逆过程，因为在 **maxpool1d** 的过程中，一些最大值的已经丢失。**MaxUnpool1d** 输入 **MaxPool1d** 的输出，包括最大值的索引，并计算所有 **maxpool1d** 过程中非最大值被设置为零的部分的反向。

注意:

MaxPool1d 可以将多个输入大小映射到相同的输出大小。因此，反演过程可能会变得模棱两可。为了适应这一点，可以在调用中将输出大小 (**output_size**) 作为额外的参数传入。具体用法，请参阅下面的输入和示例

参数:

- **kernel_size**(int or tuple) - max pooling 的窗口大小
- **stride**(int or tuple, optional) - max pooling 的窗口移动的步长。默认值是 **kernel_size**
- **padding**(int or tuple, optional) - 输入的每一条边补充 0 的层数

输入:

input: 需要转换的 **tensor** **indices:** **Maxpool1d** 的索引号 **output_size:** 一个指定输出大小的 **torch.Size**

shape:

input: (N,C,H_in)

output: (N,C,H_out)

$$H_{out} = (H_{in} - 1) \cdot stride[0] - 2padding[0] + kernel_size[0]$$

也可以使用 **output_size** 指定输出的大小

Example:

```
>>> pool = nn.MaxPool1d(2, stride=2, return_indices=True)
```

```
>>> unpool = nn.MaxUnpool1d(2, stride=2)
```

```
>>> input = Variable(torch.Tensor([[[[1, 2, 3, 4, 5, 6, 7, 8]]]]))
```

```
>>> output, indices = pool(input)
```

```
>>> unpool(output, indices)
```

```
Variable containing:
```

```
(0 ,...) =
```

```
0  2  0  4  0  6  0  8
```

```
[torch.FloatTensor of size 1x1x8]
```

```
>>> # Example showcasing the use of output_size
```

```
>>> input = Variable(torch.Tensor([[[[1, 2, 3, 4, 5, 6, 7, 8, 9]]]]))
```

```
>>> output, indices = pool(input)
```



```
>>> unpool(output, indices, output_size=input.size())
```

Variable containing:

(0 ,...) =

0 2 0 4 0 6 0 8 0

[torch.FloatTensor of size 1x1x9]

```
>>> unpool(output, indices)
```

Variable containing:

(0 ,...) =

0 2 0 4 0 6 0 8

[torch.FloatTensor of size 1x1x8]

```
class torch.nn.MaxUnpool2d(kernel_size, stride=None, padding=0)
```

`Maxpool2d` 的逆过程，不过并不是完全的逆过程，因为在 `maxpool2d` 的过程中，一些最大值的已经丢失。`MaxUnpool2d` 的输入是 `MaxPool2d` 的输出，包括最大值的索引，并计算所有 `maxpool2d` 过程中非最大值被设置为零的部分的反向。

注意：

`MaxPool2d` 可以将多个输入大小映射到相同的输出大小。因此，反演过程可能会变得模棱两可。为了适应这一点，可以在调用中将输出大小（`output_size`）作为额外的参数传入。具体用法，请参阅下面示例

参数：

- `kernel_size(int or tuple)` - max pooling 的窗口大小
- `stride(int or tuple, optional)` - max pooling 的窗口移动的步长。默认值是 `kernel_size`
- `padding(int or tuple, optional)` - 输入的每一条边补充 0 的层数

输入：

`input`: 需要转换的 `tensor`

`indices`: `Maxpool1d` 的索引号

`output_size`: 一个指定输出大小的 `torch.Size`

大小：

`input`: (N,C,H_{in},W_{in})

`output`: (N,C,H_{out},W_{out})

$H_{out} = (H_{in} - 1) \cdot stride[0] - 2 \cdot padding[0] + kernel_size[0]$

$W_{out} = (W_{in} - 1) \cdot stride[1] - 2 \cdot padding[1] + kernel_size[1]$

也可以使用 `output_size` 指定输出的大小

Example:

```
>>> pool = nn.MaxPool2d(2, stride=2, return_indices=True)
```

```
>>> unpool = nn.MaxUnpool2d(2, stride=2)
```

```
>>> input = Variable(torch.Tensor([[[[ 1, 2, 3, 4],
```

```
... [ 5, 6, 7, 8],
```

```
... [ 9, 10, 11, 12],
```

```
... [13, 14, 15, 16]]]]))
```

```
>>> output, indices = pool(input)
```

```
>>> unpool(output, indices)
```

Variable containing:

(0 ,0 ,...) =

0 0 0 0

0	6	0	8
0	0	0	0
0	14	0	16

[torch.FloatTensor of size 1x1x4x4]

```
>>> # specify a different output size than input size
>>> unpool(output, indices, output_size=torch.Size([1, 1, 5, 5]))
```

Variable containing:

(0,0,...) =

0	0	0	0	0
6	0	8	0	0
0	0	0	14	0
16	0	0	0	0
0	0	0	0	0

[torch.FloatTensor of size 1x1x5x5]

class torch.nn.MaxUnpool3d(kernel_size, stride=None, padding=0)

Maxpool3d 的逆过程，不过并不是完全的逆过程，因为在 maxpool3d 的过程中，一些最大值的已经丢失。MaxUnpool3d 的输入就是 MaxPool3d 的输出，包括最大值的索引，并计算所有 maxpool3d 过程中非最大值被设置为零的部分的反向。

注意：

MaxPool3d 可以将多个输入大小映射到相同的输出大小。因此，反演过程可能会变得模棱两可。为了适应这一点，可以在调用中将输出大小（output_size）作为额外的参数传入。具体用法，请参阅下面的输入和示例

参数：

- kernel_size(int or tuple) - Maxpooling 窗口大小
- stride(int or tuple, optional) - max pooling 的窗口移动的步长。默认值是 kernel_size
- padding(int or tuple, optional) - 输入的每一条边补充 0 的层数

输入：

input: 需要转换的 tensor

indices: Maxpool1d 的索引序数

output_size: 一个指定输出大小的 torch.Size

大小：

input: (N,C,D_in,H_in,W_in)

output: (N,C,D_out,H_out,W_out)

$$D_{out} = (D_{in} - 1) \cdot stride[0] - 2 \cdot padding[0] + kernel_size[0]$$

$$H_{out} = (H_{in} - 1) \cdot stride[1] - 2 \cdot padding[1] + kernel_size[1]$$

$$W_{out} = (W_{in} - 1) \cdot stride[2] - 2 \cdot padding[2] + kernel_size[2]$$

$$\end{aligned}$$

也可以使用 output_size 指定输出的大小

Example:

```
>>> # pool of square window of size=3, stride=2
>>> pool = nn.MaxPool3d(3, stride=2, return_indices=True)
>>> unpool = nn.MaxUnpool3d(3, stride=2)
>>> output, indices = pool(Variable(torch.randn(20, 16, 51, 33, 15)))
```

```
>>> unpooled_output = unpool(output, indices)
```

```
>>> unpooled_output.size()
```

```
torch.Size([20, 16, 51, 33, 15])
```

```
class torch.nn.AvgPool1d(kernel_size, stride=None, padding=0, ceil_mode=False, count_include_pad=True)
```

对信号的输入通道，提供 1 维平均池化（average pooling）输入信号的大小(N,C,L)，输出大小(N,C,L_out)和池化窗口大小 k 的关系是：

$$\text{out}(N_i, C_j, l) = \frac{1}{k} \sum_{m=0}^{k-1} \text{input}(N_i, C_j, \text{stride}l + m)$$

如果 padding 不是 0，会在输入的每一边添加相应数目 0

参数：

- kernel_size(int or tuple) - 池化窗口大小
- stride(int or tuple, optional) - max pooling 的窗口移动的步长。默认值是 kernel_size
- padding(int or tuple, optional) - 输入的每一条边补充 0 的层数
- dilation(int or tuple, optional) - 一个控制窗口中元素步幅的参数
- return_indices - 如果等于 True，会返回输出最大值的序号，对于上采样操作会有帮助
- ceil_mode - 如果等于 True，计算输出信号大小的时候，会使用向上取整，代替默认的向下取整的操作

大小：

input:(N,C,L_in)

output:(N,C,L_out)

$$L_{out} = \text{floor}((L_{in} + 2 * \text{padding} - \text{kernel_size}) / \text{stride} + 1)$$

Example:

```
>>> # pool with window of size=3, stride=2
```

```
>>> m = nn.AvgPool1d(3, stride=2)
```

```
>>> m(Variable(torch.Tensor([[[[1,2,3,4,5,6,7]]]])))
```

Variable containing:

```
(0 ,...)=
```

```
2  4  6
```

```
[torch.FloatTensor of size 1x1x3]
```

```
class torch.nn.AvgPool2d(kernel_size, stride=None, padding=0, ceil_mode=False, count_include_pad=True)
```

对信号的输入通道，提供 2 维的平均池化（average pooling）

输入信号的大小(N,C,H,W)，输出大小(N,C,H_out,W_out)和池化窗口大小(kH,kW)的关系是：

$$\text{out}(N_i, C_j, h, w) = \frac{1}{(kH kW)} \sum_{m=0}^{kH-1} \sum_{n=0}^{kW-1} \text{input}(N_i, C_j, \text{stride}[0]h + m, \text{stride}[1]w + n)$$

如果 padding 不是 0，会在输入的每一边添加相应数目 0

参数：

- kernel_size(int or tuple) - 池化窗口大小
- stride(int or tuple, optional) - max pooling 的窗口移动的步长。默认值是 kernel_size
- padding(int or tuple, optional) - 输入的每一条边补充 0 的层数
- dilation(int or tuple, optional) - 一个控制窗口中元素步幅的参数
- ceil_mode - 如果等于 True，计算输出信号大小的时候，会使用向上取整，代替默认的向下取整的操作
- count_include_pad - 如果等于 True，计算平均池化时，将包括 padding 填充的 0

shape:

input: (N,C,H_in,W_in)

output: (N,C,H_out,W_out)

$$\begin{aligned} H_{out} &= \text{floor}((H_{in} + 2\text{padding}[0] - \text{kernel_size}[0]) / \text{stride}[0] + 1) \\ W_{out} &= \text{floor}((W_{in} + 2\text{padding}[1] - \text{kernel_size}[1]) / \text{stride}[1] + 1) \end{aligned}$$

Example:

```
>>> # pool of square window of size=3, stride=2
>>> m = nn.AvgPool2d(3, stride=2)
>>> # pool of non-square window
>>> m = nn.AvgPool2d((3, 2), stride=(2, 1))
>>> input = autograd.Variable(torch.randn(20, 16, 50, 32))
>>> output = m(input)
```

class torch.nn.AvgPool3d(kernel_size, stride=None)

对信号的输入通道，提供 3 维的平均池化（average pooling）输入信号的大小(N,C,D,H,W)，输出大小(N,C,D_out,H_out,W_out)和池化窗口大小(kD,kH,kW)的关系是：

$$\begin{aligned} \text{out}(N_i, C_j, d, h, w) &= \frac{1}{(kDkHkW)} \sum_{k=0}^{kD-1} \sum_{h=0}^{kH-1} \sum_{w=0}^{kW-1} \text{input}(N\{i\}, C\{j\}, \text{stride}[0]d+k, \text{stride}[1]h+m, \text{stride}[2]w+n) \end{aligned}$$

如果 padding 不是 0，会在输入的每一边添加相应数目 0

参数:

- kernel_size(int or tuple) - 池化窗口大小
- stride(int or tuple, optional) - max pooling 的窗口移动的步长。默认值是 kernel_size

shape:

输入大小:(N,C,D_in,H_in,W_in)

输出大小:(N,C,D_out,H_out,W_out)

$$\begin{aligned} D_{out} &= \text{floor}((D_{in} + 2\text{padding}[0] - \text{kernel_size}[0]) / \text{stride}[0] + 1) \\ H_{out} &= \text{floor}((H_{in} + 2\text{padding}[1] - \text{kernel_size}[1]) / \text{stride}[1] + 1) \\ W_{out} &= \text{floor}((W_{in} + 2\text{padding}[2] - \text{kernel_size}[2]) / \text{stride}[2] + 1) \end{aligned}$$

Example:

```
>>> # pool of square window of size=3, stride=2
>>> m = nn.AvgPool3d(3, stride=2)
>>> # pool of non-square window
>>> m = nn.AvgPool3d((3, 2, 2), stride=(2, 1, 2))
>>> input = autograd.Variable(torch.randn(20, 16, 50, 44, 31))
>>> output = m(input)
```

class torch.nn.FractionalMaxPool2d(kernel_size, output_size=None, output_ratio=None, return_indices=False, _random_samples=None)

对输入的信号，提供 2 维的分数最大化池化操作 分数最大化池化的细节请阅读[论文](#) 由目标输出大小确定的随机步长,在\$kh*kw\$区域进行最大池化操作。输出特征和输入特征的数量相同。

参数:

- kernel_size(int or tuple) - 最大池化操作时的窗口大小。可以是一个数字（表示 K*K 的窗口），也可以是一个元组 (kh*kw)
- output_size - 输出图像的尺寸。可以使用一个 tuple 指定(oH,oW)，也可以使用一个数字 oH 指定一

个 $oH \times oH$ 的输出。

- `output_ratio` – 将输入图像的大小的百分比指定为输出图片的大小，使用一个范围在(0,1)之间的数字指定
- `return_indices` - 默认值 `False`，如果设置为 `True`，会返回输出的索引，索引对 `nn.MaxUnpool2d` 有用。

Example:

```
>>> # pool of square window of size=3, and target output size 13x12
>>> m = nn.FractionalMaxPool2d(3, output_size=(13, 12))
>>> # pool of square window and target output size being half of input image size
>>> m = nn.FractionalMaxPool2d(3, output_ratio=(0.5, 0.5))
>>> input = autograd.Variable(torch.randn(20, 16, 50, 32))
>>> output = m(input)
```

`class torch.nn.LPPool2d(norm_type, kernel_size, stride=None, ceil_mode=False)`

对输入信号提供 2 维的幂平均池化操作。 输出的计算方式：

$f(x) = \text{pow}(\text{sum}(X, p), 1/p)$

- 当 p 为无穷大的时候时，等价于最大池化操作
- 当 $p=1$ 时，等价于平均池化操作

参数 `kernel_size`, `stride` 的数据类型：

- `int`，池化窗口的宽和高相等
- `tuple` 数组（两个数字的），一个元素是池化窗口的高，另一个是宽

参数

- `kernel_size`: 池化窗口的大小
- `stride`: 池化窗口移动的步长。 `kernel_size` 是默认值
- `ceil_mode`: `ceil_mode=True` 时，将使用向下取整代替向上取整

shape

- 输入: (N, C, H_{in}, W_{in})
- 输出: (N, C, H_{out}, W_{out})

$$\begin{aligned} H_{out} &= \text{floor}((H_{in} + 2\text{padding}[0] - \text{dilation}[0](\text{kernel_size}[0] - 1) - 1) / \text{stride}[0] + 1) \\ W_{out} &= \text{floor}((W_{in} + 2\text{padding}[1] - \text{dilation}[1](\text{kernel_size}[1] - 1) - 1) / \text{stride}[1] + 1) \end{aligned}$$

Example:

```
>>> # power-2 pool of square window of size=3, stride=2
>>> m = nn.LPPool2d(2, 3, stride=2)
>>> # pool of non-square window of power 1.2
>>> m = nn.LPPool2d(1.2, (3, 2), stride=(2, 1))
>>> input = autograd.Variable(torch.randn(20, 16, 50, 32))
>>> output = m(input)
```

`class torch.nn.AdaptiveMaxPool1d(output_size, return_indices=False)`

对输入信号，提供 1 维的自适应最大池化操作 对于任何输入大小的输入，可以将输出尺寸指定为 `H`，但是输入和输出特征的数目不会变化。

参数:

- `output_size`: 输出信号的尺寸
- `return_indices`: 如果设置为 `True`，会返回输出的索引。对 `nn.MaxUnpool1d` 有用，默认值是 `False`

Example:

```
>>> # target output size of 5
>>> m = nn.AdaptiveMaxPool1d(5)
```

```
>>> input = autograd.Variable(torch.randn(1, 64, 8))
```

```
>>> output = m(input)
```

`class torch.nn.AdaptiveMaxPool2d(output_size, return_indices=False)`

对输入信号，提供 2 维的自适应最大池化操作 对于任何输入大小的输入，可以将输出尺寸指定为 $H \times W$ ，但是输入和输出特征的数目不会变化。

参数：

- `output_size`: 输出信号的尺寸,可以用 (H, W) 表示 $H \times W$ 的输出，也可以使用数字 H 表示 $H \times H$ 大小的输出
- `return_indices`: 如果设置为 `True`，会返回输出的索引。对 `nn.MaxUnpool2d` 有用，默认值是 `False`

Example:

```
>>> # target output size of 5x7
```

```
>>> m = nn.AdaptiveMaxPool2d((5,7))
```

```
>>> input = autograd.Variable(torch.randn(1, 64, 8, 9))
```

```
>>> # target output size of 7x7 (square)
```

```
>>> m = nn.AdaptiveMaxPool2d(7)
```

```
>>> input = autograd.Variable(torch.randn(1, 64, 10, 9))
```

```
>>> output = m(input)
```

`class torch.nn.AdaptiveAvgPool1d(output_size)`

对输入信号，提供 1 维的自适应平均池化操作 对于任何输入大小的输入，可以将输出尺寸指定为 $H \times W$ ，但是输入和输出特征的数目不会变化。

参数：

- `output_size`: 输出信号的尺寸

Example:

```
>>> # target output size of 5
```

```
>>> m = nn.AdaptiveAvgPool1d(5)
```

```
>>> input = autograd.Variable(torch.randn(1, 64, 8))
```

```
>>> output = m(input)
```

`class torch.nn.AdaptiveAvgPool2d(output_size)`

对输入信号，提供 2 维的自适应平均池化操作 对于任何输入大小的输入，可以将输出尺寸指定为 $H \times W$ ，但是输入和输出特征的数目不会变化。

参数：

- `output_size`: 输出信号的尺寸,可以用 (H, W) 表示 $H \times W$ 的输出，也可以使用耽搁数字 H 表示 $H \times H$ 大小的输出

Example:

```
>>> # target output size of 5x7
```

```
>>> m = nn.AdaptiveAvgPool2d((5,7))
```

```
>>> input = autograd.Variable(torch.randn(1, 64, 8, 9))
```

```
>>> # target output size of 7x7 (square)
```

```
>>> m = nn.AdaptiveAvgPool2d(7)
```

```
>>> input = autograd.Variable(torch.randn(1, 64, 10, 9))
```

```
>>> output = m(input)
```

Non-Linear Activations [\[source\]](#)

`class torch.nn.ReLU(inplace=False) \[source\]`

对输入运用修正线性单元函数 $\text{ReLU}(x) = \max(0, x)$ ，

参数: inplace-选择是否进行覆盖运算

shape:

- 输入: $(N,)$, 代表任意数目附加维度
- 输出: $(N, *)$, 与输入拥有同样的 shape 属性

例子:

```
>>> m = nn.ReLU()
>>> input = autograd.Variable(torch.randn(2))
>>> print(input)
>>> print(m(input))
```

class torch.nn.ReLU6(inplace=False) [\[source\]](#)

对输入的每一个元素运用函数 $\text{ReLU6}(x) = \min(\max(0, x), 6)$,

参数: inplace-选择是否进行覆盖运算

shape:

- 输入: $(N,)$, 代表任意数目附加维度
- 输出: $(N, *)$, 与输入拥有同样的 shape 属性

例子:

```
>>> m = nn.ReLU6()
>>> input = autograd.Variable(torch.randn(2))
>>> print(input)
>>> print(m(input))
```

class torch.nn.ELU(alpha=1.0, inplace=False) [\[source\]](#)

对输入的每一个元素运用函数 $f(x) = \max(0, x) + \min(0, \alpha * (e^x - 1))$,

shape:

- 输入: $(N, *)$, 星号代表任意数目附加维度
- 输出: $(N, *)$ 与输入拥有同样的 shape 属性

例子:

```
>>> m = nn.ELU()
>>> input = autograd.Variable(torch.randn(2))
>>> print(input)
>>> print(m(input))
```

class torch.nn.PReLU(num_parameters=1, init=0.25) [\[source\]](#)

对输入的每一个元素运用函数 $\text{PReLU}(x) = \max(0, x) + a * \min(0, x)$, a 是一个可学习参数。当没有声明时, `nn.PReLU()` 在所有的输入中只有一个参数 a : 如果是 `nn.PReLU(nChannels)`, a 将应用到每个输入。

注意: 当为了表现更佳模型而学习参数 a 时不要使用权重衰减 (weight decay)

参数:

- num_parameters: 需要学习的 a 的个数, 默认等于 1
- init: a 的初始值, 默认等于 0.25

shape:

- 输入: $(N,)$, 代表任意数目附加维度
- 输出: $(N, *)$, 与输入拥有同样的 shape 属性

例子:

```
>>> m = nn.PReLU()
>>> input = autograd.Variable(torch.randn(2))
```

```
>>> print(input)
```

```
>>> print(m(input))
```

class torch.nn.LeakyReLU(negative_slope=0.01, inplace=False) [\[source\]](#)

对输入的每一个元素运用 $f(x) = \max(0, x) + \{\text{negative_slope}\} * \min(0, x)$

参数:

- negative_slope: 控制负斜率的角度, 默认等于 0.01
- inplace-选择是否进行覆盖运算

shape:

- 输入: $(N, *)$, 代表任意数目附加维度
- 输出: $(N, *)$, 与输入拥有同样的 shape 属性

例子:

```
>>> m = nn.LeakyReLU(0.1)
```

```
>>> input = autograd.Variable(torch.randn(2))
```

```
>>> print(input)
```

```
>>> print(m(input))
```

class torch.nn.Threshold(threshold, value, inplace=False) [\[source\]](#)

Threshold 定义:

$y=x, \text{if } x \geq \text{threshold}$; $y=\text{value}, \text{if } x < \text{threshold}$

参数:

- threshold: 阈值
- value: 输入值小于阈值则会被 value 代替
- inplace: 选择是否进行覆盖运算

shape:

- 输入: $(N, *)$, 代表任意数目附加维度
- 输出: $(N, *)$, 与输入拥有同样的 shape 属性

例子:

```
>>> m = nn.Threshold(0.1, 20)
```

```
>>> input = Variable(torch.randn(2))
```

```
>>> print(input)
```

```
>>> print(m(input))
```

class torch.nn.Hardtanh(min_value=-1, max_value=1, inplace=False) [\[source\]](#)

对每个元素,

$f(x)=+1, \text{if } x > 1$; $f(x)=-1, \text{if } x < -1$; $f(x)=x, \text{otherwise}$

线性区域的范围 $[-1, 1]$ 可以被调整

参数:

- min_value: 线性区域范围最小值
- max_value: 线性区域范围最大值
- inplace: 选择是否进行覆盖运算

shape:

- 输入: $(N, *)$, *表示任意维度组合
- 输出: $(N, *)$, 与输入有相同的 shape 属性

例子:

```
>>> m = nn.Hardtanh()
```

```
>>> input = autograd.Variable(torch.randn(2))
```

```
>>> print(input)
```

```
>>> print(m(input))
```

class torch.nn.Sigmoid [\[source\]](#)

对每个元素运用 Sigmoid 函数，Sigmoid 定义如下：

$$f(x)=1/(1+e^{-x})$$

shape:

- 输入: (N, *), *表示任意维度组合
- 输出: (N, *), 与输入有相同的 shape 属性

例子:

```
>>> m = nn.Sigmoid()
```

```
>>> input = autograd.Variable(torch.randn(2))
```

```
>>> print(input)
```

```
>>> print(m(input))
```

class torch.nn.Tanh [\[source\]](#)

对输入的元素，

$$f(x)=e^x-e^{-x}e^x+e^x$$

shape:

- 输入: (N, *), *表示任意维度组合
- 输出: (N, *), 与输入有相同的 shape 属性

例子:

```
>>> m = nn.Tanh()
```

```
>>> input = autograd.Variable(torch.randn(2))
```

```
>>> print(input)
```

```
>>> print(m(input))
```

class torch.nn.LogSigmoid [\[source\]](#)

对输入的元素， $\text{LogSigmoid}(x) = \log(1 / (1 + e^{-x}))$

shape:

- 输入: (N, *), *表示任意维度组合
- 输出: (N, *), 与输入有相同的 shape 属性

例子:

```
>>> m = nn.LogSigmoid()
```

```
>>> input = autograd.Variable(torch.randn(2))
```

```
>>> print(input)
```

```
>>> print(m(input))
```

class torch.nn.Softplus(beta=1, threshold=20) [\[source\]](#)

对每个元素运用 Softplus 函数，Softplus 定义如下：

$$f(x)=1\beta\log(1+e^{\beta x})$$

Softplus 函数是 ReLU 函数的平滑逼近，Softplus 函数可以使得输出值限定为正数。

为了保证数值稳定性，线性函数的转换可以使输出大于某个值。

参数:

- beta: Softplus 函数的 beta 值
- threshold: 阈值

shape:

- 输入: (N, *), *表示任意维度组合

- 输出: (N, *), 与输入有相同的 shape 属性

例子:

```
>>> m = nn.Softplus()
>>> input = autograd.Variable(torch.randn(2))
>>> print(input)
>>> print(m(input))
```

class torch.nn.Softshrink(lambd=0.5)[source]

对每个元素运用 Softshrink 函数, Softshrink 函数定义如下:

$f(x)=x-\lambda, \text{if } x>\lambda$ $f(x)=x+\lambda, \text{if } x<-\lambda$ $f(x)=0, \text{otherwise}$

参数:

lambd: Softshrink 函数的 lambda 值, 默认为 0.5

shape:

- 输入: (N, *), *表示任意维度组合
- 输出: (N, *), 与输入有相同的 shape 属性

例子:

```
>>> m = nn.Softshrink()
>>> input = autograd.Variable(torch.randn(2))
>>> print(input)
>>> print(m(input))
```

class torch.nn.Softsign [source]

$f(x) = x / (1 + |x|)$

shape:

- 输入: (N, *), *表示任意维度组合
- 输出: (N, *), 与输入有相同的 shape 属性

例子:

```
>>> m = nn.Softsign()
>>> input = autograd.Variable(torch.randn(2))
>>> print(input)
>>> print(m(input))
```

class torch.nn.Softshrink(lambd=0.5)[source]

对每个元素运用 Tanhshrink 函数, Tanhshrink 函数定义如下:

$\text{Tanhshrink}(x)=x-\tanh(x)$

shape:

- 输入: (N, *), *表示任意维度组合
- 输出: (N, *), 与输入有相同的 shape 属性

例子:

```
>>> m = nn.Tanhshrink()
>>> input = autograd.Variable(torch.randn(2))
>>> print(input)
>>> print(m(input))
```

class torch.nn.Softmin [source]

对 n 维输入张量运用 Softmin 函数, 将张量的每个元素缩放到 (0,1) 区间且和为 1。Softmin 函数定义如下:

$f_i(x)=\frac{e^{(-x_i-\text{shift})}}{\sum_j e^{(-x_j-\text{shift})}}, \text{shift}=\max(x_i)$

shape:

- 输入: (N, L)
- 输出: (N, L)

例子:

```
>>> m = nn.Softmin()
>>> input = autograd.Variable(torch.randn(2, 3))
>>> print(input)
>>> print(m(input))
```

class torch.nn.Softmax [\[source\]](#)

对 n 维输入张量运用 Softmax 函数, 将张量的每个元素缩放到 (0,1) 区间且和为 1。Softmax 函数定义如下:

$$f_i(x) = \frac{e^{(x_i - \text{shift})}}{\sum_j e^{(x_j - \text{shift})}}, \text{shift} = \max(x_i)$$

shape:

- 输入: (N, L)
- 输出: (N, L)

返回结果是一个与输入维度相同的张量, 每个元素的取值范围在 (0,1) 区间。

例子:

```
>>> m = nn.Softmax()
>>> input = autograd.Variable(torch.randn(2, 3))
>>> print(input)
>>> print(m(input))
```

class torch.nn.LogSoftmax [\[source\]](#)

对 n 维输入张量运用 LogSoftmax 函数, LogSoftmax 函数定义如下:

$$f_i(x) = \log e^{(x_i)} a, a = \sum_j e^{(x_j)}$$

shape:

- 输入: (N, L)
- 输出: (N, L)

例子:

```
>>> m = nn.LogSoftmax()
>>> input = autograd.Variable(torch.randn(2, 3))
>>> print(input)
>>> print(m(input))
```

Normalization layers [\[source\]](#)

class torch.nn.BatchNorm1d(num_features, eps=1e-05, momentum=0.1, affine=True) [\[source\]](#)

对小批量(mini-batch)的 2d 或 3d 输入进行批标准化(Batch Normalization)操作

$$y = \frac{x - \text{mean}[x]}{\sqrt{\text{Var}[x] + \epsilon}} \gamma + \beta$$

在每一个小批量 (mini-batch) 数据中, 计算输入各个维度的均值和标准差。gamma 与 beta 是可学习的大小为 C 的参数向量 (C 为输入大小)

在训练时, 该层计算每次输入的均值与方差, 并进行移动平均。移动平均默认的动量值为 0.1。

在验证时, 训练求得的均值/方差将用于标准化验证数据。

参数:

- **num_features:** 来自期望输入的特征数, 该期望输入的大小为 'batch_size x num_features [x width]'
- **eps:** 为保证数值稳定性 (分母不能趋近或取 0), 给分母加上的值。默认为 1e-5。

- **momentum:** 动态均值和动态方差所使用的动量。默认为 0.1。
- **affine:** 一个布尔值，当设为 true，给该层添加可学习的仿射变换参数。

Shape: - 输入: (N, C) 或者 (N, C, L) - 输出: (N, C) 或者 (N, C, L) (输入输出相同)

例子

```
>>> # With Learnable Parameters
>>> m = nn.BatchNorm1d(100)
>>> # Without Learnable Parameters
>>> m = nn.BatchNorm1d(100, affine=False)
>>> input = autograd.Variable(torch.randn(20, 100))
>>> output = m(input)
```

`class torch.nn.BatchNorm2d(num_features, eps=1e-05, momentum=0.1, affine=True)`[\[source\]](#)

对小批量(mini-batch)3d 数据组成的 4d 输入进行批标准化(Batch Normalization)操作

$$y = x - \text{mean}[x] \text{Var}[x] \text{---} \sqrt{\epsilon + \gamma + \beta}$$

在每一个小批量 (mini-batch) 数据中，计算输入各个维度的均值和标准差。 γ 与 β 是可学习的大小为 C 的参数向量 (C 为输入大小)

在训练时，该层计算每次输入的均值与方差，并进行移动平均。移动平均默认的动量值为 0.1。

在验证时，训练求得的均值/方差将用于标准化验证数据。

参数:

- **num_features:** 来自期望输入的特征数，该期望输入的大小为 'batch_size x num_features x height x width'
- **eps:** 为保证数值稳定性 (分母不能趋近或取 0), 给分母加上的值。默认为 1e-5。
- **momentum:** 动态均值和动态方差所使用的动量。默认为 0.1。
- **affine:** 一个布尔值，当设为 true，给该层添加可学习的仿射变换参数。

Shape: - 输入: (N, C, H, W) - 输出: (N, C, H, W) (输入输出相同)

例子

```
>>> # With Learnable Parameters
>>> m = nn.BatchNorm2d(100)
>>> # Without Learnable Parameters
>>> m = nn.BatchNorm2d(100, affine=False)
>>> input = autograd.Variable(torch.randn(20, 100, 35, 45))
>>> output = m(input)
```

`class torch.nn.BatchNorm3d(num_features, eps=1e-05, momentum=0.1, affine=True)`[\[source\]](#)

对小批量(mini-batch)4d 数据组成的 5d 输入进行批标准化(Batch Normalization)操作

$$y = x - \text{mean}[x] \text{Var}[x] \text{---} \sqrt{\epsilon + \gamma + \beta}$$

在每一个小批量 (mini-batch) 数据中，计算输入各个维度的均值和标准差。 γ 与 β 是可学习的大小为 C 的参数向量 (C 为输入大小)

在训练时，该层计算每次输入的均值与方差，并进行移动平均。移动平均默认的动量值为 0.1。

在验证时，训练求得的均值/方差将用于标准化验证数据。

参数:

- **num_features:** 来自期望输入的特征数，该期望输入的大小为 'batch_size x num_features depth x height x width'
- **eps:** 为保证数值稳定性 (分母不能趋近或取 0), 给分母加上的值。默认为 1e-5。

- **momentum**: 动态均值和动态方差所使用的动量。默认为 0.1。
 - **affine**: 一个布尔值，当设为 true，给该层添加可学习的仿射变换参数。
- Shape**: - 输入: (N, C, D, H, W) - 输出: (N, C, D, H, W) (输入输出相同)

例子

```
>>> # With Learnable Parameters
>>> m = nn.BatchNorm3d(100)
>>> # Without Learnable Parameters
>>> m = nn.BatchNorm3d(100, affine=False)
>>> input = autograd.Variable(torch.randn(20, 100, 35, 45, 10))
>>> output = m(input)
```

Recurrent layers

`class torch.nn.RNN( args, * kwargs)[source]`

将一个多层的 Elman RNN，激活函数为 `tanh` 或者 `ReLU`，用于输入序列。

对输入序列中每个元素，RNN 每层的计算公式为 $h_t = \tanh(w_{ih} x_t + b_{ih} + w_{hh} h_{t-1} + b_{hh})$ 。 h_t 是时刻 t 的隐状态。 x_t 是上一层时刻 t 的隐状态，或者是第一层在时刻 t 的输入。如果 `nonlinearity='relu'`，那么将使用 `relu` 代替 `tanh` 作为激活函数。

参数说明:

- `input_size` – 输入 x 的特征数量。
- `hidden_size` – 隐层的特征数量。
- `num_layers` – RNN 的层数。
- `nonlinearity` – 指定非线性函数使用 `tanh` 还是 `relu`。默认是 `tanh`。
- `bias` – 如果是 `False`，那么 RNN 层就不会使用偏置权重 `bih` 和 `bhh`，默认是 `True`。
- `batch_first` – 如果 `True` 的话，那么输入 `Tensor` 的 shape 应该是 `[batch_size, time_step, feature]`，输出也是这样。
- `dropout` – 如果值非零，那么除了最后一层外，其它层的输出都会套上一个 `dropout` 层。
- `bidirectional` – 如果 `True`，将会变成一个双向 RNN，默认是 `False`。

RNN 的输入: **(input, h₀)** - `input` (seq_len, batch, input_size): 保存输入序列特征的 `tensor`。 `input` 可以被填充的变长的序列。细节请看 `torch.nn.utils.rnn.pack_padded_sequence()`

• `h0` (num_layers * num_directions, batch, hidden_size): 保存着初始隐状态的 `tensor`

RNN 的输出: **(output, h_n)**

• `output` (seq_len, batch, hidden_size * num_directions): 保存着 RNN 最后一层的输出特征。如果输入是被填充过的序列，那么输出也是被填充的序列。

• `hn` (num_layers * num_directions, batch, hidden_size): 保存着最后一个时刻隐状态。

RNN 模型参数:

- `weightihl[k]` – 第 k 层的 `input-hidden` 权重，可学习，形状是 `(input_size x hidden_size)`。
- `weighthhl[k]` – 第 k 层的 `hidden-hidden` 权重，可学习，形状是 `(hidden_size x hidden_size)`。
- `biasihl[k]` – 第 k 层的 `input-hidden` 偏置，可学习，形状是 `(hidden_size)`。
- `biashhl[k]` – 第 k 层的 `hidden-hidden` 偏置，可学习，形状是 `(hidden_size)`。

示例:

```
rnn = nn.RNN(10, 20, 2)
input = Variable(torch.randn(5, 3, 10))
h0 = Variable(torch.randn(2, 3, 20))
```

```
output, hn = rnn(input, h0)
```

```
class torch.nn.LSTM( args, * kwargs)[source]
```

将一个多层的 (LSTM) 应用到输入序列。

对输入序列的每个元素，LSTM 的每层都会执行以下计算：

$$\begin{aligned} i_t &= \text{sigmoid}(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi}) \\ f_t &= \text{sigmoid}(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf}) \\ g_t &= \text{sigmoid}(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho}) \\ c_t &= \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg}) \\ h_t &= o_t * \tanh(c_t) \end{aligned}$$

h_t 是时刻 t 的隐状态, c_t 是时刻 t 的细胞状态, x_t 是上一层的在时刻 t 的隐状态或者是第一层在时刻 t 的输入。 i_t, f_t, g_t, o_t 分别代表 输入门, 遗忘门, 细胞和输出门。

参数说明:

- input_size – 输入的特征维度
- hidden_size – 隐状态的特征维度
- num_layers – 层数 (和时序展开要区分开)
- bias – 如果为 False, 那么 LSTM 将不会使用 b_{ih}, b_{hh} , 默认为 True。
- batch_first – 如果为 True, 那么输入和输出 Tensor 的形状为 (batch, seq, feature)
- dropout – 如果非零的话, 将会在 RNN 的输出上加个 dropout, 最后一层除外。
- bidirectional – 如果为 True, 将会变成一个双向 RNN, 默认为 False。

LSTM 输入: input, (h_0, c_0)

- input (seq_len, batch, input_size): 包含输入序列特征的 Tensor。也可以是 packed variable, 详见 [pack_padded_sequence](#torch.nn.utils.rnn.pack_padded_sequence(input, lengths, batch_first=False)[source])

- h_0 (num_layers * num_directions, batch, hidden_size): 保存着 batch 中每个元素的初始化隐状态的 Tensor

- c_0 (num_layers * num_directions, batch, hidden_size): 保存着 batch 中每个元素的初始化细胞状态的 Tensor

LSTM 输出 output, (h_n, c_n)

- output (seq_len, batch, hidden_size * num_directions): 保存 RNN 最后一层的输出的 Tensor。如果输入是 torch.nn.utils.rnn.PackedSequence, 那么输出也是 torch.nn.utils.rnn.PackedSequence。

- h_n (num_layers * num_directions, batch, hidden_size): Tensor, 保存着 RNN 最后一个时间步的隐状态。

- c_n (num_layers * num_directions, batch, hidden_size): Tensor, 保存着 RNN 最后一个时间步的细胞状态。

LSTM 模型参数:

- weight_ih_l[k] – 第 k 层可学习的 input-hidden 权重 ($W_{ii}|W_{if}|W_{ig}|W_{io}$), 形状为 (input_size x 4*hidden_size)

- weight_hh_l[k] – 第 k 层可学习的 hidden-hidden 权重 ($W_{hi}|W_{hf}|W_{hg}|W_{ho}$), 形状为 (hidden_size x 4*hidden_size)。

- bias_ih_l[k] – 第 k 层可学习的 input-hidden 偏置 ($b_{ii}|b_{if}|b_{ig}|b_{io}$), 形状为 (4*hidden_size)

- bias_hh_l[k] – 第 k 层可学习的 hidden-hidden 偏置 ($b_{hi}|b_{hf}|b_{hg}|b_{ho}$), 形状为 (4*hidden_size)。示例:

```
lstm = nn.LSTM(10, 20, 2)
```

```
input = Variable(torch.randn(5, 3, 10))
```

```
h0 = Variable(torch.randn(2, 3, 20))
```

```
c0 = Variable(torch.randn(2, 3, 20))
```

```
output, hn = lstm(input, (h0, c0))
```

```
class torch.nn.GRU( args, * kwargs)[source]
```

将一个多层的 `GRU` 用于输入序列。

对输入序列中的每个元素，每层进行了一下计算：

```
$$ \begin{aligned} r_t &= \text{sigmoid}(W_{\{ir\}}x_t + b_{\{ir\}} + W_{\{hr\}}h_{\{(t-1)\}} + b_{\{hr\}}) \\ i_t &= \text{sigmoid}(W_{\{ii\}}x_t + b_{\{ii\}} + W_{\{hi\}}h_{\{(t-1)\}} + b_{\{hi\}}) \\ n_t &= \text{tanh}(W_{\{in\}}x_t + b_{\{in\}} + r_t(W_{\{hn\}}h_{\{(t-1)\}} + b_{\{hn\}})) \\ h_t &= (1 - i_t)n_t + i_t h_{(t-1)} \end{aligned} $$
```

h_t 是时间 t 上的隐状态， x_t 是前一层 t 时刻的隐状态或者是第一层的 t 时刻的输入， r_t, i_t, n_t 分别是重置门，输入门和新门。

参数说明： - `input_size` – 期望的输入 x 的特征值的维度 - `hidden_size` – 隐状态的维度 - `num_layers` – `RNN` 的层数。 - `bias` – 如果为 `False`，那么 `RNN` 层将不会使用 `bias`，默认为 `True`。 - `batch_first` – 如果为 `True` 的话，那么输入和输出的 `tensor` 的形状是 `(batch, seq, feature)`。 - `dropout` – 如果非零的话，将会在 `RNN` 的输出上加个 `dropout`，最后一层除外。 - `bidirectional` – 如果为 `True`，将会变成一个双向 `RNN`，默认为 `False`。

输入： `input, h_0`

- `input` (`seq_len, batch, input_size`): 包含输入序列特征的 `Tensor`。也可以是 `packed variable`，详见 `[pack_padded_sequence](#torch.nn.utils.rnn.pack_padded_sequence(input, lengths, batch_first=False)[source])`。

- `h_0` (`num_layers * num_directions, batch, hidden_size`): 保存着 `batch` 中每个元素的初始化隐状态的 `Tensor`

输出： `output, h_n`

- `output` (`seq_len, batch, hidden_size * num_directions`): 保存 `RNN` 最后一层的输出的 `Tensor`。如果输入是 `torch.nn.utils.rnn.PackedSequence`，那么输出也是 `torch.nn.utils.rnn.PackedSequence`。

- `h_n` (`num_layers * num_directions, batch, hidden_size`): `Tensor`，保存着 `RNN` 最后一个时间步的隐状态。

变量：

- `weight_ih_l[k]` – 第 k 层可学习的 `input-hidden` 权重 ($W_{\{ir\}}|W_{\{ii\}}|W_{\{in\}}$)，形状为 `(input_size x 3*hidden_size)`

- `weight_hh_l[k]` – 第 k 层可学习的 `hidden-hidden` 权重 ($W_{\{hr\}}|W_{\{hi\}}|W_{\{hn\}}$)，形状为 `(hidden_size x 3*hidden_size)`。

- `bias_ih_l[k]` – 第 k 层可学习的 `input-hidden` 偏置 ($b_{\{ir\}}|b_{\{ii\}}|b_{\{in\}}$)，形状为 `(3*hidden_size)`

- `bias_hh_l[k]` – 第 k 层可学习的 `hidden-hidden` 偏置 ($b_{\{hr\}}|b_{\{hi\}}|b_{\{hn\}}$)，形状为 `(3*hidden_size)`。

例子：

```
rnn = nn.GRU(10, 20, 2)
```

```
input = Variable(torch.randn(5, 3, 10))
```

```
h0 = Variable(torch.randn(2, 3, 20))
```

```
output, hn = rnn(input, h0)
```

```
class torch.nn.RNNCell(input_size, hidden_size, bias=True, nonlinearity='tanh')[source]
```

一个 `Elman RNN cell`，激活函数是 `tanh` 或 `ReLU`，用于输入序列。 将一个多层的 `Elman RNNCell`，激活函数为 `tanh` 或者 `ReLU`，用于输入序列。 $h' = \text{tanh}(w_{\{ih\}}x + b_{\{ih\}} + w_{\{hh\}}h + b_{\{hh\}})$ 如果 `nonlinearity=relu`，那么将会使用 `ReLU` 来代替 `tanh`。

参数:

- `input_size` – 输入 x 的维度。
- `hidden_size` – 隐状态特征的维度。
- `bias` – 如果为 `False`, `RNN cell` 中将不会加入 `bias`, 默认为 `True`。
- `nonlinearity` – 用于选择非线性激活函数 [`tanh`|`relu`]. 默认值为: `tanh`

输入: `input, hidden`

- `input (batch, input_size)`: 包含输入特征的 `tensor`。
- `hidden (batch, hidden_size)`: 保存着初始隐状态值的 `tensor`。

输出: `h'`

- `h' (batch, hidden_size)`: 下一个时刻的隐状态。

变量:

- `weight_ih` – `input-hidden` 权重, 可学习, 形状是 `(input_size x hidden_size)`。
- `weight_hh` – `hidden-hidden` 权重, 可学习, 形状是 `(hidden_size x hidden_size)`
- `bias_ih` – `input-hidden` 偏置, 可学习, 形状是 `(hidden_size)`
- `bias_hh` – `hidden-hidden` 偏置, 可学习, 形状是 `(hidden_size)`

例子:

```
rnn = nn.RNNCell(10, 20)
input = Variable(torch.randn(6, 3, 10))
hx = Variable(torch.randn(3, 20))
output = []
for i in range(6):
    hx = rnn(input[i], hx)
    output.append(hx)
```

`class torch.nn.LSTMCell(input_size, hidden_size, bias=True)[source]`

`LSTM cell`。
$$\begin{aligned} i &= \text{sigmoid}(W_{ii}x + b_{ii} + W_{hi}h + b_{hi}) \quad f = \text{sigmoid}(W_{if}x + b_{if} + W_{hf}h + b_{hf}) \\ o &= \text{sigmoid}(W_{io}x + b_{io} + W_{ho}h + b_{ho}) \quad g = \tanh(W_{ig}x + b_{ig} + W_{hg}h + b_{hg}) \\ c' &= f_{tc_t-1} + i_{tg_t} \quad h' = o_t * \tanh(c') \end{aligned}$$

参数:

- `input_size` – 输入的特征维度。
- `hidden_size` – 隐状态的维度。
- `bias` – 如果为 `False`, 那么将不会使用 `bias`。默认为 `True`。

`LSTM` 输入: `input, (h_0, c_0)`

- `input (seq_len, batch, input_size)`: 包含输入序列特征的 `Tensor`。也可以是 `packed variable`, 详见 `[pack_padded_sequence](#torch.nn.utils.rnn.pack_padded_sequence(input, lengths, batch_first=False[source]))`

- `h_0 (batch, hidden_size)`: 保存着 `batch` 中每个元素的初始化隐状态的 `Tensor`
- `c_0 (batch, hidden_size)`: 保存着 `batch` 中每个元素的初始化细胞状态的 `Tensor`

输出: `h_1, c_1`

- `h_1 (batch, hidden_size)`: 下一个时刻的隐状态。
- `c_1 (batch, hidden_size)`: 下一个时刻的细胞状态。

`LSTM` 模型参数:

- `weight_ih` – `input-hidden` 权重 ($W_{ii}|W_{if}|W_{ig}|W_{io}$), 形状为 `(input_size x 4*hidden_size)`
- `weight_hh` – `hidden-hidden` 权重 ($W_{hi}|W_{hf}|W_{hg}|W_{ho}$), 形状为 `(hidden_size x 4*hidden_size)`。

- `bias_ih` – input-hidden 偏置($b_{ii}|b_{if}|b_{ig}|b_{io}$), 形状为 $(4*hidden_size)$
- `bias_hh` – hidden-hidden 偏置($b_{hi}|b_{hf}|b_{hg}|b_{ho}$), 形状为 $(4*hidden_size)$ 。

Examples:

```
rnn = nn.LSTMCell(10, 20)
input = Variable(torch.randn(6, 3, 10))
hx = Variable(torch.randn(3, 20))
cx = Variable(torch.randn(3, 20))
output = []
for i in range(6):
    hx, cx = rnn(input[i], (hx, cx))
    output.append(hx)
```

`class torch.nn.GRUCell(input_size, hidden_size, bias=True)[source]`

一个 GRU cell。
$$\begin{aligned} r &= \text{sigmoid}(W_{ir}x + b_{ir} + W_{hr}h + b_{hr}) \\ i &= \text{sigmoid}(W_{ii}x + b_{ii} + W_{hi}h + b_{hi}) \\ n &= \tanh(W_{in}x + b_{in} + r(W_{hn}h + b_{hn})) \\ h' &= (1-i) \cdot n + i \cdot h \end{aligned}$$

参数说明: - `input_size` – 期望的输入 x 的特征值的维度 - `hidden_size` – 隐状态的维度 - `bias` – 如果为 `False`, 那么 RNN 层将不会使用 `bias`, 默认为 `True`。

输入: `input, h_0`

- `input` (batch, input_size): 包含输入特征的 Tensor
- `h_0` (batch, hidden_size): 保存着 batch 中每个元素的初始化隐状态的 Tensor

输出: `h_1`

- `h_1` (batch, hidden_size): Tensor, 保存着 RNN 下一个时刻的隐状态。

变量:

- `weight_ih` – input-hidden 权重($W_{ir}|W_{ii}|W_{in}$), 形状为 $(input_size \times 3*hidden_size)$
- `weight_hh` – hidden-hidden 权重($W_{hr}|W_{hi}|W_{hn}$), 形状为 $(hidden_size \times 3*hidden_size)$ 。
- `bias_ih` – input-hidden 偏置($b_{ir}|b_{ii}|b_{in}$), 形状为 $(3*hidden_size)$
- `bias_hh` – hidden-hidden 偏置($b_{hr}|b_{hi}|b_{hn}$), 形状为 $(3*hidden_size)$ 。

例子:

```
rnn = nn.GRUCell(10, 20)
input = Variable(torch.randn(6, 3, 10))
hx = Variable(torch.randn(3, 20))
output = []
for i in range(6):
    hx = rnn(input[i], hx)
    output.append(hx)
```

Linear layers

`class torch.nn.Linear(in_features, out_features, bias=True)`

对输入数据做线性变换: $y = Ax + b$

参数:

- `in_features` - 每个输入样本的大小
- `out_features` - 每个输出样本的大小
- `bias` - 若设置为 `False`, 这层不会学习偏置。默认值: `True`

形状:

- 输入: $(N, in_features)$

- **输出:** (N,out_features)

变量:

- **weight** - 形状为(out_features x in_features)的模块中可学习的权值
- **bias** - 形状为(out_features)的模块中可学习的偏置

例子:

```
>>> m = nn.Linear(20, 30)
>>> input = autograd.Variable(torch.randn(128, 20))
>>> output = m(input)
>>> print(output.size())
```

Dropout layers

```
class torch.nn.Dropout(p=0.5, inplace=False)
```

随机将输入张量中部分元素设置为 0。对于每次前向调用，被置 0 的元素都是随机的。

参数:

- **p** - 将元素置 0 的概率。默认值: 0.5
- **in-place** - 若设置为 True，会在原地执行操作。默认值: False

形状:

- **输入:** 任意。输入可以为任意形状。
- **输出:** 相同。输出和输入形状相同。

例子:

```
>>> m = nn.Dropout(p=0.2)
>>> input = autograd.Variable(torch.randn(20, 16))
>>> output = m(input)
```

```
class torch.nn.Dropout2d(p=0.5, inplace=False)
```

随机将输入张量中整个通道设置为 0。对于每次前向调用，被置 0 的通道都是随机的。

通常输入来自 Conv2d 模块。

像在论文 [Efficient Object Localization Using Convolutional Networks](#)，如果特征图中相邻像素是强相关的（在前几层卷积层很常见），那么 iid dropout 不会归一化激活，而只会降低学习率。

在这种情形，`nn.Dropout2d()`可以提高特征图之间的独立程度，所以应该使用它。

参数:

- **p(float, optional)** - 将元素置 0 的概率。
- **in-place(bool, optional)** - 若设置为 True，会在原地执行操作。

形状:

- **输入:** (N,C,H,W)
- **输出:** (N,C,H,W)（与输入形状相同）

例子:

```
>>> m = nn.Dropout2d(p=0.2)
>>> input = autograd.Variable(torch.randn(20, 16, 32, 32))
>>> output = m(input)
```

```
class torch.nn.Dropout3d(p=0.5, inplace=False)
```

随机将输入张量中整个通道设置为 0。对于每次前向调用，被置 0 的通道都是随机的。

通常输入来自 Conv3d 模块。

像在论文 [Efficient Object Localization Using Convolutional Networks](#)，如果特征图中相邻像素是强相关的（在前几层卷积层很常见），那么 iid dropout 不会归一化激活，而只会降低学习率。

在这种情形，`nn.Dropout3d()`可以提高特征图之间的独立程度，所以应该使用它。

参数:

- **p**(float, optional) - 将元素置 0 的概率。
- **in-place**(bool, optional) - 若设置为 True, 会在原地执行操作。

形状:

- 输入: (N,C,D,H,W)
- 输出: (N,C,D,H,W) (与输入形状相同)

例子:

```
>>> m = nn.Dropout3d(p=0.2)
>>> input = autograd.Variable(torch.randn(20, 16, 4, 32, 32))
>>> output = m(input)
```

Sparse layers

```
class torch.nn.Embedding(num_embeddings, embedding_dim, padding_idx=None, max_norm=None,
norm_type=2, scale_grad_by_freq=False, sparse=False)
```

一个保存了固定字典和大小的简单查找表。

这个模块常用来保存词嵌入和用下标检索它们。模块的输入是一个下标的列表, 输出是对应的词嵌入。

参数:

- **num_embeddings** (int) - 嵌入字典的大小
- **embedding_dim** (int) - 每个嵌入向量的大小
- **padding_idx** (int, optional) - 如果提供的话, 输出遇到此下标时用零填充
- **max_norm** (float, optional) - 如果提供的话, 会重新归一化词嵌入, 使它们的范数小于提供的值
- **norm_type** (float, optional) - 对于 max_norm 选项计算 p 范数时的 p
- **scale_grad_by_freq** (boolean, optional) - 如果提供的话, 会根据字典中单词频率缩放梯度

变量:

- **weight** (Tensor) - 形状为(num_embeddings, embedding_dim)的模块中可学习的权重

形状:

- 输入: LongTensor (N, W), N = mini-batch, W = 每个 mini-batch 中提取的下标数
- 输出: (N, W, embedding_dim)

例子:

```
>>> # an Embedding module containing 10 tensors of size 3
>>> embedding = nn.Embedding(10, 3)
>>> # a batch of 2 samples of 4 indices each
>>> input = Variable(torch.LongTensor([[1,2,4,5],[4,3,2,9]]))
>>> embedding(input)
```

Variable containing:

```
(0 ,,,) =
-1.0822  1.2522  0.2434
 0.8393 -0.6062 -0.3348
 0.6597  0.0350  0.0837
 0.5521  0.9447  0.0498
```

```
(1 ,,,) =
 0.6597  0.0350  0.0837
-0.1527  0.0877  0.4260
```

```
0.8393 -0.6062 -0.3348
-0.8738 -0.9054  0.4281
[torch.FloatTensor of size 2x4x3]
```

```
>>> # example with padding_idx
>>> embedding = nn.Embedding(10, 3, padding_idx=0)
>>> input = Variable(torch.LongTensor([[0,2,0,5]]))
>>> embedding(input)
```

Variable containing:

```
(0 ,,,) =
 0.0000  0.0000  0.0000
 0.3452  0.4937 -0.9361
 0.0000  0.0000  0.0000
 0.0706 -2.1962 -0.6276
[torch.FloatTensor of size 1x4x3]
```

Distance functions

```
class torch.nn.PairwiseDistance(p=2, eps=1e-06)
```

按批计算向量 v_1, v_2 之间的距离:

// $x // p := (\sum_{i=1}^n |x_i|^p)^{1/p}$

参数:

- **x** (Tensor): 包含两个输入 batch 的张量
- **p** (real): 范数次数, 默认值: 2

形状:

- **输入:** (N,D), 其中 D=向量维数
- **输出:** (N,1)

```
>>> pdist = nn.PairwiseDistance(2)
>>> input1 = autograd.Variable(torch.randn(100, 128))
>>> input2 = autograd.Variable(torch.randn(100, 128))
>>> output = pdist(input1, input2)
```

Loss functions

基本用法:

```
criterion = LossCriterion() #构造函数有自己的参数
```

```
loss = criterion(x, y) #调用标准时也有参数
```

计算出来的结果已经对 **mini-batch** 取了平均。

```
class torch.nn.L1Loss(size_average=True)[source]
```

创建一个衡量输入 x (模型预测输出) 和目标 y 之间差的绝对值的平均值的标准。

$loss(x,y) = 1/n \sum |x_i - y_i|$

- x 和 y 可以是任意形状, 每个包含 n 个元素。
- 对 n 个元素对应的差值的绝对值求和, 得出来的结果除以 n 。
- 如果在创建 **L1Loss** 实例的时候在构造函数中传入 `size_average=False`, 那么求出来的绝对值的和将不会除以 n

```
class torch.nn.MSELoss(size_average=True)[source]
```

创建一个衡量输入 x (模型预测输出) 和目标 y 之间均方误差标准。

$\text{loss}(x,y)=1/n\sum(x_i-y_i)^2$

- x 和 y 可以是任意形状，每个包含 n 个元素。
- 对 n 个元素对应的差值的绝对值求和，得出来的结果除以 n 。
- 如果在创建 `MSELoss` 实例的时候在构造函数中传入 `size_average=False`，那么求出来的平方和将不会除以 n

`class torch.nn.CrossEntropyLoss(weight=None, size_average=True)`[\[source\]](#)

此标准将 `LogSoftMax` 和 `NLLLoss` 集成到一个类中。

当训练一个多类分类器的时候，这个方法是十分有用的。

- `weight(tensor)`: 1-D tensor, n 个元素，分别代表 n 类的权重，如果你的训练样本很不均衡的话，是非常有用的。默认值为 `None`。

调用时参数：

- `input`：包含每个类的得分，2-D tensor, shape 为 `batch*n`
- `target`: 大小为 n 的 1-D tensor, 包含类别的索引(0 到 $n-1$)。

Loss 可以表述为以下形式：

$\text{loss}(x, \text{class}) = -\log \exp(x[\text{class}]) / \sum_j \exp(x[j]) = -x[\text{class}] + \log(\sum_j \exp(x[j]))$

当 `weight` 参数被指定的时候，`loss` 的计算公式变为：

$\text{loss}(x, \text{class}) = \text{weights}[\text{class}] * (-x[\text{class}] + \log(\sum_j \exp(x[j])))$

计算出的 `loss` 对 `mini-batch` 的大小取了平均。

形状(shape):

- `Input: (N,C)` C 是类别的数量
- `Target: (N)` N 是 `mini-batch` 的大小, $0 \leq \text{targets}[i] \leq C-1$

`class torch.nn.NLLLoss(weight=None, size_average=True)`[\[source\]](#)

负的 `log likelihood loss` 损失。用于训练一个 n 类分类器。

如果提供的话，`weight` 参数应该是一个 1-D tensor，里面的值对应类别的权重。当你的训练集样本不均衡的话，使用这个参数是非常有用的。

输入是一个包含类别 `log-probabilities` 的 2-D tensor，形状是 `(mini-batch, n)`

可以通过在最后一层加 `LogSoftmax` 来获得类别的 `log-probabilities`。

如果您不想增加一个额外层的话，您可以使用 `CrossEntropyLoss`。

此 `loss` 期望的 `target` 是类别的索引 (0 to $N-1$, where N = number of classes)

此 `loss` 可以被表示如下：

$\text{loss}(x, \text{class}) = -x[\text{class}]$

如果 `weights` 参数被指定的话，`loss` 可以表示如下：

$\text{loss}(x, \text{class}) = -\text{weights}[\text{class}] * x[\text{class}]$

参数说明：

- `weight (Tensor, optional)` – 手动指定每个类别的权重。如果给定的话，必须是长度为 `nclasses`
- `size_average (bool, optional)` – 默认情况下，会计算 `mini-batch`loss` 的平均值。然而，如果 `size_average=False` 那么将会把 `mini-batch` 中所有样本的 `loss` 累加起来。

形状:

- `Input: (N,C)`, C 是类别的个数
- `Target: (N)`, `target` 中每个值的大小满足 $0 \leq \text{targets}[i] \leq C-1$

例子:

```
m = nn.LogSoftmax()
```

```
loss = nn.NLLLoss()
```

```
# input is of size nBatch x nClasses = 3 x 5
input = autograd.Variable(torch.randn(3, 5), requires_grad=True)
# each element in target has to have 0 <= value < nclasses
target = autograd.Variable(torch.LongTensor([1, 0, 4]))
output = loss(m(input), target)
output.backward()
```

`class torch.nn.NLLLoss2d(weight=None, size_average=True)`[\[source\]](#)

对于图片的 **negative log likelihood loss**。计算每个像素的 **NLL loss**。

参数说明：

- **weight** (Tensor, optional) – 用来作为每类的权重，如果提供的话，必须为 **1-D tensor**，大小为 **C**：类别的个数。
- **size_average** – 默认情况下，会计算 **mini-batch** loss 均值。如果设置为 **False** 的话，将会累加 **mini-batch** 中所有样本的 **loss** 值。默认值：**True**。

形状：

- **Input**: (N,C,H,W) **C** 类的数量
- **Target**: (N,H,W) where each value is $0 \leq \text{targets}[i] \leq C-1$

例子：

```
m = nn.Conv2d(16, 32, (3, 3)).float()
loss = nn.NLLLoss2d()
# input is of size nBatch x nClasses x height x width
input = autograd.Variable(torch.randn(3, 16, 10, 10))
# each element in target has to have 0 <= value < nclasses
target = autograd.Variable(torch.LongTensor(3, 8, 8).random_(0, 4))
output = loss(m(input), target)
output.backward()
```

`class torch.nn.KLDivLoss(weight=None, size_average=True)`[\[source\]](#)

计算 **KL 散度** 损失。

KL 散度 常用来描述两个分布的距离，并在输出分布的空间上执行直接回归是有用的。

与 **NLLLoss** 一样，给定的输入应该是 **log-probabilities**。然而，和 **NLLLoss** 不同的是，**input** 不限于 **2-D tensor**，因为此标准是基于 **element** 的。

target 应该和 **input** 的形状相同。

此 loss 可以表示为：

$$\text{loss}(x, \text{target}) = -\ln \sum_i (\text{target}_i * (\log(\text{target}_i) - x_i))$$

默认情况下，loss 会基于 **element** 求平均。如果 **size_average=False** loss 会被累加起来。

`class torch.nn.BCELoss(weight=None, size_average=True)`[\[source\]](#)

计算 **target** 与 **output** 之间的 **二进制交叉熵**。

$$\text{loss}(o, t) = -\frac{1}{n} \sum_i (t[i] \log(o[i]) + (1-t[i]) \log(1-o[i])) \quad \text{如果 } \text{weight} \text{ 被指定:}$$

$$\text{loss}(o, t) = -\frac{1}{n} \sum_i \text{weights}[i] (t[i] \log(o[i]) + (1-t[i]) * \log(1-o[i]))$$

这个用于计算 **auto-encoder** 的 **reconstruction error**。注意 $0 \leq \text{target}[i] \leq 1$ 。

默认情况下，loss 会基于 **element** 平均，如果 **size_average=False** 的话，loss 会被累加。

`class torch.nn.MarginRankingLoss(margin=0, size_average=True)`[\[source\]](#)

创建一个标准，给定输入 x_1, x_2 两个 **1-D mini-batch Tensor's**，和一个 y (**1-D mini-batch tensor**)， y 里面的值只能是 -1 或 1。

如果 $y=1$ ，代表第一个输入的值应该大于第二个输入的值，如果 $y=-1$ 的话，则相反。

`mini-batch` 中每个样本的 loss 的计算公式如下：

$$\text{loss}(x,y)=\max(0,-y*(x1-x2)+\text{margin})$$

如果 `size_average=True`, 那么求出的 `loss` 将会对 `mini-batch` 求平均, 反之, 求出的 `loss` 会累加。默认情况下, `size_average=True`。

`class torch.nn.HingeEmbeddingLoss(size_average=True)` [\[source\]](#)

给定一个输入 x (2-D mini-batch tensor) 和对应的 标签 y (1-D tensor, 1, -1), 此函数用来计算之间的损失值。这个 `loss` 通常用来测量两个输入是否相似, 即: 使用 L1 成对距离。典型是用于学习非线性 `embedding` 或者半监督学习中:

$$\text{loss}(x,y)=1n\sum_i\{x_i,\text{if } y_i==1 \max(0,\text{margin}-x_i),\text{if } y_i==-1$$

x 和 y 可以是任意形状, 且都有 n 的元素, `loss` 的求和操作作用在所有的元素上, 然后除以 n 。如果您不想除以 n 的话, 可以通过设置 `size_average=False`。

`margin` 的默认值为 1, 可以通过构造函数来设置。

`class torch.nn.MultiLabelMarginLoss(size_average=True)` [\[source\]](#)

计算多标签分类的 `hinge loss` (margin-based loss), 计算 `loss` 时需要两个输入: `input x` (2-D mini-batch Tensor), 和 `output y` (2-D tensor 表示 mini-batch 中样本类别的索引)。

$$\text{loss}(x,y)=1x.\text{size}(0)\sum_{i=0,J}(\max(0,1-(x[y[j]]-x[i])))$$

其中 $I=x.\text{size}(0), J=y.\text{size}(0)$ 。对于所有的 i 和 j , 满足 $y[j]\neq 0, i\neq y[j]$

x 和 y 必须具有同样的 `size`。

这个标准仅考虑了第一个非零 `y[j]` targets 此标准允许了, 对于每个样本来说, 可以有多个类别。

`class torch.nn.SmoothL1Loss(size_average=True)` [\[source\]](#)

平滑版 L1 loss。

`loss` 的公式如下:

$$\text{loss}(x,y)=1n\sum_i\{0.5*(x_i-y_i)^2,\text{if } |x_i-y_i|<1 |x_i-y_i|-0.5,\text{otherwise}$$

此 `loss` 对于异常点的敏感性不如 `MSELoss`, 而且, 在某些情况下防止了梯度爆炸, (参照 `Fast R-CNN`)。这个 `loss` 有时也被称为 `Huber loss`。

x 和 y 可以是任何包含 n 个元素的 `tensor`。默认情况下, 求出来的 `loss` 会除以 n , 可以通过设置 `size_average=True` 使 `loss` 累加。

`class torch.nn.SoftMarginLoss(size_average=True)` [\[source\]](#)

创建一个标准, 用来优化 2 分类的 `logistic loss`。输入为 x (一个 2-D mini-batch Tensor) 和 目标 y (一个包含 1 或 -1 的 Tensor)。

$$\text{loss}(x,y)=1x.\text{nelement}()\sum_i(\log(1+\exp(-y[i]*x[i])))$$

如果求出的 `loss` 不想被平均可以通过设置 `size_average=False`。

`class torch.nn.MultiLabelSoftMarginLoss(weight=None, size_average=True)` [\[source\]](#)

创建一个标准, 基于输入 x 和目标 y 的 `max-entropy`, 优化多标签 `one-versus-all` 的损失。 x : 2-D mini-batch Tensor; y : binary 2D Tensor。对每个 mini-batch 中的样本, 对应的 `loss` 为:

$$\text{loss}(x,y)=-1x.\text{nElement}()\sum_{i=0}y[i]\log\exp(x[i])(1+\exp(x[i]))+(1-y[i])\log 1+\exp(x[i])$$

其中 $I=x.\text{nElement}()-1, y[i]\in\{0,1\}$, y 和 x 必须要有同样 `size`。

`class torch.nn.CosineEmbeddingLoss(margin=0, size_average=True)` [\[source\]](#)

给定 输入 `Tensors`, $x1, x2$ 和一个标签 Tensor y (元素的值为 1 或 -1)。此标准使用 `cosine` 距离测量两个输入是否相似, 一般用来学习非线性 `embedding` 或者半监督学习。

`margin` 应该是 -1 到 1 之间的值, 建议使用 0 到 0.5。如果没有传入 `margin` 实参, 默认值为 0。

每个样本的 `loss` 是:

$$\text{loss}(x,y)=\{1-\cos(x1,x2),\text{if } y==1 \max(0,\cos(x1,x2)-\text{margin}),\text{if } y==-1$$

如果 `size_average=True` 求出的 `loss` 会对 `batch` 求均值, 如果 `size_average=False` 的话, 则

会累加 `loss`。默认情况 `size_average=True`。

`class torch.nn.MultiMarginLoss(p=1, margin=1, weight=None, size_average=True)[source]`

用来计算 multi-class classification 的 hinge loss (margin-based loss)。输入是 `x` (2D mini-batch Tensor), `y` (1D Tensor) 包含类别的索引, $0 \leq y \leq x.size(1)$ 。

对每个 mini-batch 样本:

$$\text{loss}(x, y) = \frac{1}{x.size(0)} \sum_{i=0}^p (\max(0, \text{margin} - x[y] + x[i]))^p$$

其中 $i = x.size(0) \dots x.size(1) - 1$ 。 可选的, 如果您不想所有的类拥有同样的权重的话, 您可以通过在构造函数中传入 `weights` 参数来解决这个问题, `weights` 是一个 1D 权重 Tensor。

传入 `weights` 后, `loss` 函数变为:

$$\text{loss}(x, y) = \frac{1}{x.size(0)} \sum_i \max(0, w[y] * (\text{margin} - x[y] - x[i]))^p$$

默认情况下, 求出的 `loss` 会对 mini-batch 取平均, 可以通过设置 `size_average=False` 来取消取平均操作。

Vision layers

`class torch.nn.PixelShuffle(upscale_factor)[source]`

将 shape 为 $[N, C, H, W]$ 的 Tensor 重新排列为 shape 为 $[N, C, H \cdot \text{upscale_factor}, W \cdot \text{upscale_factor}]$ 的 Tensor。当使用 `stride=1` 的 sub-pixel 卷积的时候, 这个方法是非常有用的。

请看 [paper Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network by Shi et. al \(2016\)](#) 获取详细信息。

参数说明:

- `upscale_factor` (int) – 增加空间分辨率的因子

Shape:

- Input: $[N, C \cdot \text{upscale_factor}^2, H, W]$
- Output: $[N, C, H \cdot \text{upscale_factor}, W \cdot \text{upscale_factor}]$

例子:

```
>>> ps = nn.PixelShuffle(3)
>>> input = autograd.Variable(torch.Tensor(1, 9, 4, 4))
>>> output = ps(input)
>>> print(output.size())
torch.Size([1, 1, 12, 12])
```

`class torch.nn.UpsamplingNearest2d(size=None, scale_factor=None)[source]`

对于多 channel 输入 进行 2-D 最近邻上采样。

可以通过 `size` 或者 `scale_factor` 来指定上采样后的图片大小。

当给定 `size` 时, `size` 的值将会是输出图片的大小。

参数:

- `size` (tuple, optional) – 一个包含两个整数的元组 (`H_out`, `W_out`) 指定了输出的长宽
- `scale_factor` (int, optional) – 长和宽的一个乘子

形状:

- Input: (N, C, H_{in}, W_{in})
- Output: (N, C, H_{out}, W_{out}) $H_{out} = \text{floor}(H_{in} * \text{scale_factor})$ $W_{out} = \text{floor}(W_{in} * \text{scale_factor})$

例子:

```
>>> inp
Variable containing:
(0, 0, ..., ) =
```



```
1 2
3 4
```

```
[torch.FloatTensor of size 1x1x2x2]
```

```
>>> m = nn.UpsamplingNearest2d(scale_factor=2)
```

```
>>> m(inp)
```

```
Variable containing:
```

```
(0,0,...) =
```

```
1 1 2 2
1 1 2 2
3 3 4 4
3 3 4 4
```

```
[torch.FloatTensor of size 1x1x4x4]
```

`class torch.nn.UpsamplingBilinear2d(size=None, scale_factor=None)`[\[source\]](#)

对于多 channel 输入 进行 2-D bilinear 上采样。

可以通过 `size` 或者 `scale_factor` 来指定上采样后的图片大小。

当给定 `size` 时，`size` 的值将会是输出图片的大小。

参数：

- `size` (tuple, optional) – 一个包含两个整数的元组 (`H_out`, `W_out`)指定了输出的长宽
- `scale_factor` (int, optional) – 长和宽的一个乘子

形状：

- Input: (N,C,H_in,W_in)
- Output: (N,C,H_out,W_out) $H_{out} = \text{floor}(H_{in} * \text{scale_factor})$ $W_{out} = \text{floor}(W_{in} * \text{scale_factor})$

例子：

```
>>> inp
```

```
Variable containing:
```

```
(0,0,...) =
```

```
1 2
3 4
```

```
[torch.FloatTensor of size 1x1x2x2]
```

```
>>> m = nn.UpsamplingBilinear2d(scale_factor=2)
```

```
>>> m(inp)
```

```
Variable containing:
```

```
(0,0,...) =
```

```
1.0000 1.3333 1.6667 2.0000
1.6667 2.0000 2.3333 2.6667
2.3333 2.6667 3.0000 3.3333
3.0000 3.3333 3.6667 4.0000
```

```
[torch.FloatTensor of size 1x1x4x4]
```

Multi-GPU layers

`class torch.nn.DataParallel(module, device_ids=None, output_device=None, dim=0)`[\[source\]](#)

在模块级别上实现数据并行。

此容器通过将 mini-batch 划分到不同的设备上来实现给定 `module` 的并行。在 `forward` 过程中，`module`

会在每个设备上都复制一遍，每个副本都会处理部分输入。在 `backward` 过程中，副本上的梯度会累加到原始 `module` 上。

`batch` 的大小应该大于所使用的 GPU 的数量。还应当是 GPU 个数的整数倍，这样划分出来的每一块都会有相同的样本数量。

请看: [Use nn.DataParallel instead of multiprocessing](#)

除了 `Tensor`，任何位置参数和关键字参数都可以传到 `DataParallel` 中。所有的变量会通过指定的 `dim` 来划分（默认值为 0）。原始类型将会被广播，但是所有的其它类型都会被浅复制。所以如果在模型的 `forward` 过程中写入的话，将会被损坏。

参数说明:

- `module` – 要被并行的 `module`
- `device_ids` – CUDA 设备，默认为所有设备。
- `output_device` – 输出设备（默认为 `device_ids[0]`）

例子:

```
net = torch.nn.DataParallel(model, device_ids=[0, 1, 2])
```

```
output = net(input_var)
```

Utilities

工具函数

`torch.nn.utils.clip_grad_norm(parameters, max_norm, norm_type=2)`[\[source\]](#)

Clips gradient norm of an iterable of parameters.

正则项的值由所有的梯度计算出来，就像他们连成一个向量一样。梯度被 `in-place operation` 修改。

参数说明: - `parameters` (`Iterable[Variable]`) – 可迭代的 `Variables`，它们的梯度即将被标准化。 - `max_norm` (`float` or `int`) – `clip` 后, `gradients` p-norm 值 - `norm_type` (`float` or `int`) – 标准化的类型, p-norm. 可以是 `inf` 代表 `infinity norm`.

关于 `norm`

返回值:

所有参数的 p-norm 值。

`torch.nn.utils.rnn.PackedSequence(_cls, data, batch_sizes)`[\[source\]](#)

Holds the data and list of `batch_sizes` of a packed sequence.

All RNN modules accept packed sequences as inputs. 所有的 `RNN` 模块都接收这种被包裹后的序列作为它们的输入。

NOTE: 这个类的实例不能手动创建。它们只能被 `pack_padded_sequence()` 实例化。

参数说明:

- `data` (`Variable`) – 包含打包后序列的 `Variable`。
- `batch_sizes` (`list[int]`) – 包含 `mini-batch` 中每个序列长度的列表。

`torch.nn.utils.rnn.pack_padded_sequence(input, lengths, batch_first=False)`[\[source\]](#)

这里的 `pack`，理解成压紧比较好。将一个 填充过的变长序列 压紧。（填充时候，会有冗余，所以压紧一下）

输入的形状可以是 $(T \times B \times *)$ 。 `T` 是最长序列长度， `B` 是 `batch size`， `*` 代表任意维度(可以是 0)。如果 `batch_first=True` 的话，那么相应的 `input size` 就是 $(B \times T \times *)$ 。

`Variable` 中保存的序列，应该按序列长度的长短排序，长的在前，短的在后。即 `input[:,0]` 代表的是最长的序列， `input[:, B-1]` 保存的是最短的序列。

NOTE: 只要是维度大于等于 2 的 `input` 都可以作为这个函数的参数。你可以用它来打包 `labels`，然后用 `RNN` 的输出和打包后的 `labels` 来计算 `loss`。通过 `PackedSequence` 对象的 `.data` 属性可以获取 `Variable`。

参数说明:

- input (Variable) – 变长序列 被填充后的 batch
- lengths (list[int]) – Variable 中 每个序列的长度。
- batch_first (bool, optional) – 如果是 True, input 的形状应该是 $B \times T \times \text{size}$ 。

返回值:

一个 PackedSequence 对象。

`torch.nn.utils.rnn.pad_packed_sequence(sequence, batch_first=False)`[\[source\]](#)

填充 packed_sequence。

上面提到的函数的功能是将一个填充后的变长序列压紧。 这个操作和 `pack_padded_sequence()` 是相反的。把压紧的序列再填充回来。

返回的 Variable 的值的 size 是 $T \times B \times *$, T 是最长序列的长度, B 是 batch_size, 如果 `batch_first=True`, 那么返回值是 $B \times T \times *$ 。

Batch 中的元素将会以它们长度的逆序排列。

参数说明:

- sequence (PackedSequence) – 将要被填充的 batch
- batch_first (bool, optional) – 如果为 True, 返回的数据的格式为 $B \times T \times *$ 。

返回值: 一个 tuple, 包含被填充后的序列, 和 batch 中序列的长度列表。

例子:

```
import torch
import torch.nn as nn
from torch.autograd import Variable
from torch.nn import utils as nn_utils

batch_size = 2
max_length = 3
hidden_size = 2
n_layers = 1

tensor_in = torch.FloatTensor([[1, 2, 3], [1, 0, 0]]).resize_(2,3,1)
tensor_in = Variable( tensor_in ) #[batch, seq, feature], [2, 3, 1]
seq_lengths = [3,1] # list of integers holding information about the batch size at each sequence step

# pack it
pack = nn_utils.rnn.pack_padded_sequence(tensor_in, seq_lengths, batch_first=True)

# initialize
rnn = nn.RNN(1, hidden_size, n_layers, batch_first=True)
h0 = Variable(torch.randn(n_layers, batch_size, hidden_size))

#forward
out, _ = rnn(pack, h0)

# unpack
unpacked = nn_utils.rnn.pad_packed_sequence(out)
print(unpacked)
```



关注【深度学习爱好者】

更多学习资源免费获取

第五章 torch.nn.functional

Convolution 函数

```
torch.nn.functional.conv1d(input, weight, bias=None, stride=1, padding=0, dilation=1, groups=1)
```

对几个输入平面组成的输入信号应用 1D 卷积。

有关详细信息和输出形状，请参见 [Conv1d](#)。

参数： - **input** – 输入张量的形状 (minibatch x in_channels x iW) - **weight** – 过滤器的形状 (out_channels, in_channels, kW) - **bias** – 可选偏置的形状 (out_channels) - **stride** – 卷积核的步长，默认为 1

例子：

```
>>> filters = autograd.Variable(torch.randn(33, 16, 3))
```

```
>>> inputs = autograd.Variable(torch.randn(20, 16, 50))
```

```
>>> F.conv1d(inputs, filters)
```

```
torch.nn.functional.conv2d(input, weight, bias=None, stride=1, padding=0, dilation=1, groups=1)
```

对几个输入平面组成的输入信号应用 2D 卷积。

有关详细信息和输出形状，请参见 [Conv2d](#)。

参数： - **input** – 输入张量 (minibatch x in_channels x iH x iW) - **weight** – 过滤器张量 (out_channels, in_channels/groups, kH, kW) - **bias** – 可选偏置张量 (out_channels) - **stride** – 卷积核的步长，可以是单个数字或一个元组 (sh x sw)。默认为 1 - **padding** – 输入上隐含零填充。可以是单个数字或元组。默认值：0 - **groups** – 将输入分成组，in_channels 应该被组数除尽

例子：

```
>>> # With square kernels and equal stride
```

```
>>> filters = autograd.Variable(torch.randn(8,4,3,3))
```

```
>>> inputs = autograd.Variable(torch.randn(1,4,5,5))
```

```
>>> F.conv2d(inputs, filters, padding=1)
```

```
torch.nn.functional.conv3d(input, weight, bias=None, stride=1, padding=0, dilation=1, groups=1)
```

对几个输入平面组成的输入信号应用 3D 卷积。

有关详细信息和输出形状，请参见 [Conv3d](#)。

参数： - **input** – 输入张量的形状 (minibatch x in_channels x iT x iH x iW) - **weight** – 过滤器张量的形状 (out_channels, in_channels, kT, kH, kW) - **bias** – 可选偏置张量的形状 (out_channels) - **stride** – 卷积核的步长，可以是单个数字或一个元组 (sh x sw)。默认为 1 - **padding** – 输入上隐含零填充。可以是单个数字或元组。默认值：0

例子：

```
>>> filters = autograd.Variable(torch.randn(33, 16, 3, 3, 3))
```

```
>>> inputs = autograd.Variable(torch.randn(20, 16, 50, 10, 20))
```

```
>>> F.conv3d(inputs, filters)
```

```
torch.nn.functional.conv_transpose1d(input, weight, bias=None, stride=1, padding=0, output_padding=0, groups=1)
```

```
torch.nn.functional.conv_transpose2d(input, weight, bias=None, stride=1, padding=0, output_padding=0, groups=1)
```

在由几个输入平面组成的输入图像上应用二维转置卷积，有时也称为“去卷积”。

有关详细信息和输出形状，请参阅 [ConvTranspose2d](#)。

参数： - **input** – 输入张量的形状 (minibatch x in_channels x iH x iW) - **weight** – 过滤器的形状

(in_channels x out_channels x kH x kW) - **bias** – 可选偏置的形状 (out_channels) - **stride** – 卷积核的步长，可以是单个数字或一个元组 (sh x sw)。默认: 1 - **padding** – 输入上隐含零填充。可以是单个数字或元组。(padh x padw)。默认: 0 - **groups** – 将输入分成组，in_channels 应该被组数除尽 - **output_padding** – 0 <= padding < stride 的零填充，应该添加到输出。可以是单个数字或元组。默认值: 0

```
torch.nn.functional.conv_transpose3d(input, weight, bias=None, stride=1, padding=0, output_padding=0, groups=1)
```

在由几个输入平面组成的输入图像上应用三维转置卷积，有时也称为“去卷积”。

有关详细信息和输出形状，请参阅 [ConvTranspose3d](#)。

参数: - **input** – 输入张量的形状 (minibatch x in_channels x iT x iH x iW) - **weight** – 过滤器的形状 (in_channels x out_channels x kH x kW) - **bias** – 可选偏置的形状 (out_channels) - **stride** – 卷积核的步长，可以是单个数字或一个元组 (sh x sw)。默认: 1 - **padding** – 输入上隐含零填充。可以是单个数字或元组。(padh x padw)。默认: 0

Pooling 函数

```
torch.nn.functional.avg_pool1d(input, kernel_size, stride=None, padding=0, ceil_mode=False, count_include_pad=True)
```

对由几个输入平面组成的输入信号进行一维平均池化。

有关详细信息和输出形状，请参阅 [AvgPool1d](#)。

参数: - **kernel_size** – 窗口的大小 - **stride** – 窗口的步长。默认值为 kernel_size - **padding** – 在两边添加隐式零填充 - **ceil_mode** – 当为 True 时，将使用 ceil 代替 floor 来计算输出形状 - **count_include_pad** – 当为 True 时，这将在平均计算时包括补零

例子:

```
>>> # pool of square window of size=3, stride=2
>>> input = Variable(torch.Tensor([[[[1,2,3,4,5,6,7]]]]))
>>> F.avg_pool1d(input, kernel_size=3, stride=2)
```

Variable containing:

```
(0,...) =
```

```
 2  4  6
```

```
[torch.FloatTensor of size 1x1x3]
```

```
torch.nn.functional.avg_pool2d(input, kernel_size, stride=None, padding=0, ceil_mode=False, count_include_pad=True)
```

在 kh x kw 区域中应用步长为 dh x dw 的二维平均池化操作。输出特征的数量等于输入平面的数量。

有关详细信息和输出形状，请参阅 [AvgPool2d](#)。

参数: - **input** – 输入的张量 (minibatch x in_channels x iH x iW) - **kernel_size** – 池化区域的大小，可以是单个数字或者元组 (kh x kw) - **stride** – 池化操作的步长，可以是单个数字或者元组 (sh x sw)。默认等于核的大小 - **padding** – 在输入上隐式的零填充，可以是单个数字或者一个元组 (padh x padw)，默认: 0 - **ceil_mode** – 定义空间输出形状的操作 - **count_include_pad** – 除以原始非填充图像内的元素数量或 kh * kw

```
torch.nn.functional.avg_pool3d(input, kernel_size, stride=None)
```

在 kt x kh x kw 区域中应用步长为 dt x dh x dw 的二维平均池化操作。输出特征的数量等于 input planes / dt。

```
torch.nn.functional.max_pool1d(input, kernel_size, stride=None, padding=0, dilation=1, ceil_mode=False, return_indices=False)
```

```
torch.nn.functional.max_pool2d(input, kernel_size, stride=None, padding=0, dilation=1, ceil_mode=False,
```

```
return_indices=False)
```

```
torch.nn.functional.max_pool3d(input, kernel_size, stride=None, padding=0, dilation=1, ceil_mode=False,
return_indices=False)
```

```
torch.nn.functional.max_unpool1d(input, indices, kernel_size, stride=None, padding=0, output_size=None)
```

```
torch.nn.functional.max_unpool2d(input, indices, kernel_size, stride=None, padding=0, output_size=None)
```

```
torch.nn.functional.max_unpool3d(input, indices, kernel_size, stride=None, padding=0, output_size=None)
```

```
torch.nn.functional.lp_pool2d(input, norm_type, kernel_size, stride=None, ceil_mode=False)
```

```
torch.nn.functional.adaptive_max_pool1d(input, output_size, return_indices=False)
```

在由几个输入平面组成的输入信号上应用 1D 自适应最大池化。

有关详细信息和输出形状，请参阅 [AdaptiveMaxPool1d](#)。

参数： - **output_size** – 目标输出大小（单个整数） - **return_indices** – 是否返回池化的指数

```
torch.nn.functional.adaptive_max_pool2d(input, output_size, return_indices=False)
```

在由几个输入平面组成的输入信号上应用 2D 自适应最大池化。

有关详细信息和输出形状，请参阅 [AdaptiveMaxPool2d](#)。

参数： - **output_size** – 目标输出大小（单整数或双整数元组） - **return_indices** – 是否返回池化的指数

```
torch.nn.functional.adaptive_avg_pool1d(input, output_size)
```

在由几个输入平面组成的输入信号上应用 1D 自适应平均池化。

有关详细信息和输出形状，请参阅 [AdaptiveAvgPool1d](#)。

参数： - **output_size** – 目标输出大小（单整数或双整数元组）

```
torch.nn.functional.adaptive_avg_pool2d(input, output_size)
```

在由几个输入平面组成的输入信号上应用 2D 自适应平均池化。

有关详细信息和输出形状，请参阅 [AdaptiveAvgPool2d](#)。

参数： - **output_size** – 目标输出大小（单整数或双整数元组）

非线性激活函数

```
torch.nn.functional.threshold(input, threshold, value, inplace=False)
```

```
torch.nn.functional.relu(input, inplace=False)
```

```
torch.nn.functional.hardtanh(input, min_val=-1.0, max_val=1.0, inplace=False)
```

```
torch.nn.functional.relu6(input, inplace=False)
```

```
torch.nn.functional.elu(input, alpha=1.0, inplace=False)
```

```
torch.nn.functional.leaky_relu(input, negative_slope=0.01, inplace=False)
```

```
torch.nn.functional.prelu(input, weight)
```

```
torch.nn.functional.rrelu(input, lower=0.125, upper=0.3333333333333333, training=False, inplace=False)
```

```
torch.nn.functional.logsigmoid(input)
```

```
torch.nn.functional.hardshrink(input, lambd=0.5)
```

```
torch.nn.functional.tanhshrink(input)
```

```
torch.nn.functional.softsign(input)
```

```
torch.nn.functional.softplus(input, beta=1, threshold=20)
```

```
torch.nn.functional.softmin(input)
```

```
torch.nn.functional.softmax(input)
```

```
torch.nn.functional.softshrink(input, lambd=0.5)
```

```
torch.nn.functional.log_softmax(input)
```

```
torch.nn.functional.tanh(input)
```

```
torch.nn.functional.sigmoid(input)
```

Normalization 函数

```
torch.nn.functional.batch_norm(input, running_mean, running_var, weight=None, bias=None, training=False, momentum=0.1, eps=1e-05)
```

线性函数

```
torch.nn.functional.linear(input, weight, bias=None)
```

Dropout 函数

```
torch.nn.functional.dropout(input, p=0.5, training=False, inplace=False)
```

距离函数 (Distance functions)

```
torch.nn.functional.pairwise_distance(x1, x2, p=2, eps=1e-06)
```

计算向量 v_1 、 v_2 之间的距离 (成对或者成对, 意思是可以计算多个, 可以参看后面的参数)
$$\|x\|_p = \left(\sum_{i=1}^N |x_i|^p \right)^{1/p}$$
 参数:

- x_1 : 第一个输入的张量
- x_2 : 第二个输入的张量
- p : 矩阵范数的维度。默认值是 2, 即二范数。

规格:

- 输入: (N, D) 其中 D 等于向量的维度
- 输出: $(N, 1)$

例子:

```
>>> input1 = autograd.Variable(torch.randn(100, 128))
>>> input2 = autograd.Variable(torch.randn(100, 128))
>>> output = F.pairwise_distance(input1, input2, p=2)
>>> output.backward()
```

损失函数 (Loss functions)

```
torch.nn.functional.nll_loss(input, target, weight=None, size_average=True)
```

负的 log likelihood 损失函数. 详细请看 [NLLLoss](#).

参数: - **input** - (N, C) C 是类别的个数 - **target** - (N) 其大小是 $0 \leq \text{targets}[i] \leq C-1$ - **weight** (Variable, optional) - 一个可手动指定每个类别的权重。如果给定的话, 必须是大小为 $n\text{classes}$ 的 Variable - **size_average** (bool, optional) - 默认情况下, 是 `mini-batchloss` 的平均值, 然而, 如果 `size_average=False`, 则是 `mini-batchloss` 的总和。

Variables: - **weight** - 对于 constructor 而言, 每一类的权重作为输入

```
torch.nn.functional.kl_div(input, target, size_average=True)
```

KL 散度损失函数, 详细请看 [KLDivLoss](#)

参数: - **input** - 任意形状的 Variable - **target** - 与输入相同形状的 Variable - **size_average** - 如果为 `TRUE`, `loss` 则是平均值, 需要除以输入 tensor 中 `element` 的数目

```
torch.nn.functional.cross_entropy(input, target, weight=None, size_average=True)
```

该函数使用了 `log_softmax` 和 `nll_loss`, 详细请看 [CrossEntropyLoss](#)

参数: - **input** - (N, C) 其中, C 是类别的个数 - **target** - (N) 其大小是 $0 \leq \text{targets}[i] \leq C-1$ - **weight** (Variable, optional) - 一个可手动指定每个类别的权重。如果给定的话, 必须是大小为 $n\text{classes}$ 的 Variable - **size_average** (bool, optional) - 默认情况下, 是 `mini-batchloss` 的平均值, 然而, 如果 `size_average=False`, 则是 `mini-batchloss` 的总和。

```
torch.nn.functional.binary_cross_entropy(input, target, weight=None, size_average=True)
```

该函数计算了输出与 `target` 之间的二进制交叉熵, 详细请看 [BCELoss](#)

参数: - **input** - 任意形状的 Variable - **target** - 与输入相同形状的 Variable - **weight** (Variable, optional) - 一个可手动指定每个类别的权重。如果给定的话, 必须是大小为 $n\text{classes}$ 的 Variable

- **size_average** (bool, optional) – 默认情况下，是 `mini-batchloss` 的平均值，然而，如果 `size_average=False`，则是 `mini-batchloss` 的总和。

```
torch.nn.functional.smooth_l1_loss(input, target, size_average=True)
```

Vision functions

```
torch.nn.functional.pixel_shuffle(input, upscale_factor)[source]
```

将形状为 `[*, C*r^2, H, W]` 的 `Tensor` 重新排列成形状为 `[C, H*r, W*r]` 的 `Tensor`。

详细请看 [PixelShuffle](#)。

形参说明: - `input` (Variable) – 输入 - `upscale_factor` (int) – 增加空间分辨率的因子。

例子:

```
ps = nn.PixelShuffle(3)
```

```
input = autograd.Variable(torch.Tensor(1, 9, 4, 4))
```

```
output = ps(input)
```

```
print(output.size())
```

```
torch.Size([1, 1, 12, 12])
```

```
torch.nn.functional.pad(input, pad, mode='constant', value=0)[source]
```

填充 `Tensor`。

目前为止,只支持 2D 和 3D 填充. Currently only 2D and 3D padding supported. 当输入为 4D `Tensor` 的时候,`pad` 应该是一个 4 元素的 `tuple` (`pad_l`, `pad_r`, `pad_t`, `pad_b`), 当输入为 5D `Tensor` 的时候,`pad` 应该是一个 6 元素的 `tuple` (`pleft`, `pright`, `ptop`, `pbottom`, `pfront`, `pback`).

形参说明:

- `input` (Variable) – 4D 或 5D `tensor`
- `pad` (`tuple`) – 4 元素 或 6-元素 `tuple`
- `mode` – ‘constant’, ‘reflect’ or ‘replicate’
- `value` – 用于 `constant padding` 的值.



关注【深度学习爱好者】

更多学习资源免费获取

第六章 torch.nn.init

`torch.nn.init.calculate_gain(nonlinearity,param=None)`

对于给定的非线性函数，返回推荐的增益值。这些值如下所示：

linear 1
conv{1,2,3}d 1
sigmoid 1
tanh 5/3
relu sqrt(2)
leaky_relu sqrt(2/(1+negative_slope^2))

参数：

- **nonlinearity** - 非线性函数（`nn.functional` 名称）
- **param** - 非线性函数的可选参数

例子：

```
>>> gain = nn.init.gain('leaky_relu')
```

```
torch.nn.init.uniform(tensor, a=0, b=1)
```

从均匀分布 $U(a, b)$ 中生成值，填充输入的张量或变量

参数：

- **tensor** - n 维的 `torch.Tensor`
- **a** - 均匀分布的下界
- **b** - 均匀分布的上界

例子

```
>>> w = torch.Tensor(3, 5)
```

```
>>> nn.init.uniform(w)
```

```
torch.nn.init.normal(tensor, mean=0, std=1)
```

从给定均值和标准差的正态分布 $N(\text{mean}, \text{std})$ 中生成值，填充输入的张量或变量

参数：

- **tensor** - n 维的 `torch.Tensor`
- **mean** - 正态分布的均值
- **std** - 正态分布的标准差

例子

```
>>> w = torch.Tensor(3, 5)
```

```
>>> nn.init.normal(w)
```

```
torch.nn.init.constant(tensor, val)
```

用 `val` 的值填充输入的张量或变量

参数：

- **tensor** - n 维的 `torch.Tensor` 或 `autograd.Variable`
- **val** - 用来填充张量的值

例子：

```
>>> w = torch.Tensor(3, 5)
```

```
>>> nn.init.constant(w)
```

```
torch.nn.init.eye(tensor)
```

用单位矩阵来填充 2 维输入张量或变量。在线性层尽可能多的保存输入特性。

参数:

- **tensor** – 2 维的 torch.Tensor 或 autograd.Variable

例子:

```
>>> w = torch.Tensor(3, 5)
```

```
>>> nn.init.eye(w)
```

```
torch.nn.init.dirac(tensor)
```

用 Dirac δ 函数来填充 {3, 4, 5} 维输入张量或变量。在卷积层尽可能多的保存输入通道特性。

参数:

- **tensor** – {3, 4, 5} 维的 torch.Tensor 或 autograd.Variable

例子:

```
>>> w = torch.Tensor(3, 16, 5, 5)
```

```
>>> nn.init.dirac(w)
```

```
torch.nn.init.xavier_uniform(tensor, gain=1)
```

根据 Glorot, X. 和 Bengio, Y. 在“Understanding the difficulty of training deep feedforward neural networks”中描述的方法, 用一个均匀分布生成值, 填充输入的张量或变量。结果张量中的值采样自 $U(-a, a)$, 其中 $a = \text{gain} * \sqrt{2/(\text{fan_in} + \text{fan_out})} * \sqrt{3}$ 。该方法也被称为 Glorot initialisation

参数:

- **tensor** – n 维的 torch.Tensor
- **gain** - 可选的缩放因子

例子:

```
>>> w = torch.Tensor(3, 5)
```

```
>>> nn.init.xavier_uniform(w, gain=math.sqrt(2.0))
```

```
torch.nn.init.xavier_normal(tensor, gain=1)
```

根据 Glorot, X. 和 Bengio, Y. 于 2010 年在“Understanding the difficulty of training deep feedforward neural networks”中描述的方法, 用一个正态分布生成值, 填充输入的张量或变量。结果张量中的值采样自均值为 0, 标准差为 $\text{gain} * \sqrt{2/(\text{fan_in} + \text{fan_out})}$ 的正态分布。也被称为 Glorot initialisation.

参数:

- **tensor** – n 维的 torch.Tensor
- **gain** - 可选的缩放因子

例子:

```
>>> w = torch.Tensor(3, 5)
```

```
>>> nn.init.xavier_normal(w)
```

```
torch.nn.init.kaiming_uniform(tensor, a=0, mode='fan_in')
```

根据 He, K 等人于 2015 年在“Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification”中描述的方法, 用一个均匀分布生成值, 填充输入的张量或变量。结果张量中的值采样自 $U(-\text{bound}, \text{bound})$, 其中 $\text{bound} = \sqrt{2/((1 + a^2) * \text{fan_in})} * \sqrt{3}$ 。也被称为 He initialisation.

参数:

- **tensor** – n 维的 torch.Tensor 或 autograd.Variable
- **a** - 这层之后使用的 rectifier 的斜率系数 (ReLU 的默认值为 0)
- **mode** - 可以为“fan_in” (默认) 或“fan_out”。“fan_in”保留前向传播时权值方差的量级, “fan_out”保留反向传播时的量级。

例子:

```
>>> w = torch.Tensor(3, 5)
>>> nn.init.kaiming_uniform(w, mode='fan_in')
torch.nn.init.kaiming_normal(tensor, a=0, mode='fan_in')
```

根据 He, K 等人在“Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification”中描述的方法，用一个正态分布生成值，填充输入的张量或变量。结果张量中的值采样自均值为 0，标准差为 $\sqrt{2/((1 + a^2) * fan_in)}$ 的正态分布。

参数:

- **tensor** – n 维的 torch.Tensor 或 autograd.Variable
- **a** -这层之后使用的 rectifier 的斜率系数（ReLU 的默认值为 0）
- **mode** -可以为“fan_in”（默认）或“fan_out”。“fan_in”保留前向传播时权值方差的量级，“fan_out”保留反向传播时的量级。

例子:

```
>>> w = torch.Tensor(3, 5)
>>> nn.init.kaiming_normal(w, mode='fan_out')
torch.nn.init.orthogonal(tensor, gain=1)
```

用（半）正交矩阵填充输入的张量或变量。输入张量必须至少是 2 维的，对于更高维度的张量，超出的维度会被展平，视作行等于第一个维度，列等于稀疏矩阵乘积的 2 维表示。其中非零元素生成自均值为 0，标准差为 std 的正态分布。

参考: Saxe, A 等人(2013)的“Exact solutions to the nonlinear dynamics of learning in deep linear neural networks”

参数:

- **tensor** – n 维的 torch.Tensor 或 autograd.Variable，其中 $n \geq 2$
- **gain** -可选

例子:

```
>>> w = torch.Tensor(3, 5)
>>> nn.init.orthogonal(w)
torch.nn.init.sparse(tensor, sparsity, std=0.01)
```

将 2 维的输入张量或变量当做稀疏矩阵填充，其中非零元素根据一个均值为 0，标准差为 std 的正态分布生成。参考 Martens, J.(2010)的“Deep learning via Hessian-free optimization”。

参数:

- **tensor** – n 维的 torch.Tensor 或 autograd.Variable
- **sparsity** - 每列中需要被设置成零的元素比例
- **std** - 用于生成非零值的正态分布的标准差

例子:

```
>>> w = torch.Tensor(3, 5)
>>> nn.init.sparse(w, sparsity=0.1)
```



关注【深度学习爱好者】

更多学习资源免费获取

公众号：深度学习爱好者

第七章 torch.optim

`torch.optim` 是一个实现了各种优化算法的库。大部分常用的方法得到支持，并且接口具备足够的通用性，使得未来能够集成更加复杂的方法。

如何使用 optimizer

为了使用 `torch.optim`，你需要构建一个 `optimizer` 对象。这个对象能够保持当前参数状态并基于计算得到的梯度进行参数更新。

构建

为了构建一个 `Optimizer`，你需要给它一个包含了需要优化的参数（必须都是 `Variable` 对象）的 `iterable`。然后，你可以设置 `optimizer` 的参数选项，比如学习率，权重衰减，等等。

例子：

```
optimizer = optim.SGD(model.parameters(), lr = 0.01, momentum=0.9)
```

```
optimizer = optim.Adam([var1, var2], lr = 0.0001)
```

为每个参数单独设置选项

`Optimizer` 也支持为每个参数单独设置选项。若想这么做，不要直接传入 `Variable` 的 `iterable`，而是传入 `dict` 的 `iterable`。每一个 `dict` 都分别定义了一组参数，并且包含一个 `param` 键，这个键对应参数的列表。其他的键应该 `optimizer` 所接受的其他参数的关键字相匹配，并且会被用于对这组参数的优化。

注意：

你仍然能够传递选项作为关键字参数。在未重写这些选项的组中，它们会被用作默认值。当你只想改动一个参数组的选项，但其他参数组的选项不变时，这是非常有用的。

例如，当我们想指定每一层的学习率时，这是非常有用的：

```
optim.SGD([
    {'params': model.base.parameters()},
    {'params': model.classifier.parameters(), 'lr': 1e-3}
], lr=1e-2, momentum=0.9)
```

这意味着 `model.base` 的参数将会使用 `1e-2` 的学习率，`model.classifier` 的参数将会使用 `1e-3` 的学习率，并且 `0.9` 的 `momentum` 将会被用于所有的参数。

进行单次优化

所有的 `optimizer` 都实现了 `step()` 方法，这个方法会更新所有的参数。它能按两种方式来使用：

```
optimizer.step()
```

这是大多数 `optimizer` 所支持的简化版本。一旦梯度被如 `backward()` 之类的函数计算好后，我们就可以调用这个函数。

例子

```
for input, target in dataset:
    optimizer.zero_grad()
    output = model(input)
    loss = loss_fn(output, target)
    loss.backward()
    optimizer.step()
optimizer.step(closure)
```

一些优化算法例如 `Conjugate Gradient` 和 `LBFGS` 需要重复多次计算函数，因此你需要传入一个闭包去允许它们重新计算你的模型。这个闭包应当清空梯度，计算损失，然后返回。

例子：

```

for input, target in dataset:
    def closure():
        optimizer.zero_grad()
        output = model(input)
        loss = loss_fn(output, target)
        loss.backward()
        return loss
    optimizer.step(closure)

```

算法

`class torch.optim.Optimizer(params, defaults) [source]`

Base class for all optimizers.

参数:

- `params` (iterable) —— `Variable` 或者 `dict` 的 iterable。指定了什么参数应当被优化。
- `defaults` —— (dict): 包含了优化选项默认值的字典（一个参数组没有指定的参数选项将会使用默认值）。

`load_state_dict(state_dict) [source]`

加载 optimizer 状态

参数:

`state_dict` (dict) —— optimizer 的状态。应当是一个调用 `state_dict()` 所返回的对象。

`state_dict() [source]`

以 dict 返回 optimizer 的状态。

它包含两项。

- `state` - 一个保存了当前优化状态的 dict。optimizer 的类别不同，state 的内容也会不同。
- `param_groups` - 一个包含了全部参数组的 dict。

`step(closure) [source]`

进行单次优化 (参数更新)。

参数:

- `closure` (callable) – 一个重新评价模型并返回 loss 的闭包，对于大多数参数来说是可选的。

`zero_grad() [source]`

清空所有被优化过的 Variable 的梯度。

`class torch.optim.Adadelta(params, lr=1.0, rho=0.9, eps=1e-06, weight_decay=0) [source]`

实现 Adadelta 算法。

它在 [ADDELTA: An Adaptive Learning Rate Method](#) 中被提出。

参数:

- `params` (iterable) – 待优化参数的 iterable 或者是定义了参数组的 dict
- `rho` (float, 可选) – 用于计算平方梯度的运行平均值的系数（默认：0.9）
- `eps` (float, 可选) – 为了增加数值计算的稳定性而加到分母里的项（默认：1e-6）
- `lr` (float, 可选) – 在 delta 被应用到参数更新之前对它缩放的系数（默认：1.0）
- `weight_decay` (float, 可选) – 权重衰减（L2 惩罚）（默认：0）

`step(closure) [source]`

进行单次优化 (参数更新)。

参数:

- `closure` (callable) – 一个重新评价模型并返回 loss 的闭包，对于大多数参数来说是可选的。

```
class torch.optim.Adagrad(params, lr=0.01, lr_decay=0, weight_decay=0)[source]
```

实现 Adagrad 算法。

它在 [Adaptive Subgradient Methods for Online Learning and Stochastic Optimization](#) 中被提出。

参数:

- `params` (iterable) – 待优化参数的 iterable 或者是定义了参数组的 dict
- `lr` (float, 可选) – 学习率 (默认: 1e-2)
- `lr_decay` (float, 可选) – 学习率衰减 (默认: 0)
- `weight_decay` (float, 可选) – 权重衰减 (L2 惩罚) (默认: 0)

```
step(closure) [source]
```

进行单次优化 (参数更新).

参数:

- `closure` (callable) – 一个重新评价模型并返回 loss 的闭包, 对于大多数参数来说是可选的。

```
class torch.optim.Adam(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0)[source]
```

实现 Adam 算法。

它在 [Adam: A Method for Stochastic Optimization](#) 中被提出。

参数:

- `params` (iterable) – 待优化参数的 iterable 或者是定义了参数组的 dict
- `lr` (float, 可选) – 学习率 (默认: 1e-3)
- `betas` (Tuple[float, float], 可选) – 用于计算梯度以及梯度平方的运行平均值的系数 (默认: 0.9, 0.999)
- `eps` (float, 可选) – 为了增加数值计算的稳定性而加到分母里的项 (默认: 1e-8)
- `weight_decay` (float, 可选) – 权重衰减 (L2 惩罚) (默认: 0)

```
step(closure) [source]
```

进行单次优化 (参数更新).

参数:

- `closure` (callable) – 一个重新评价模型并返回 loss 的闭包, 对于大多数参数来说是可选的。

```
class torch.optim.Adamax(params, lr=0.002, betas=(0.9, 0.999), eps=1e-08, weight_decay=0)[source]
```

实现 Adamax 算法 (Adam 的一种基于无穷范数的变种)。

它在 [Adam: A Method for Stochastic Optimization](#) 中被提出。

参数:

- `params` (iterable) – 待优化参数的 iterable 或者是定义了参数组的 dict
- `lr` (float, 可选) – 学习率 (默认: 2e-3)
- `betas` (Tuple[float, float], 可选) – 用于计算梯度以及梯度平方的运行平均值的系数
- `eps` (float, 可选) – 为了增加数值计算的稳定性而加到分母里的项 (默认: 1e-8)
- `weight_decay` (float, 可选) – 权重衰减 (L2 惩罚) (默认: 0)

```
step(closure) [source]
```

进行单次优化 (参数更新).

参数:

- `closure` (callable) – 一个重新评价模型并返回 loss 的闭包, 对于大多数参数来说是可选的。

```
class torch.optim.ASGD(params, lr=0.01, lambd=0.0001, alpha=0.75, t0=1000000.0, weight_decay=0)[source]
```

实现平均随机梯度下降算法。

它在 [Acceleration of stochastic approximation by averaging](#) 中被提出。

参数:

- `params` (iterable) – 待优化参数的 iterable 或者是定义了参数组的 dict
- `lr` (float, 可选) – 学习率 (默认: $1e-2$)
- `lamdb` (float, 可选) – 衰减项 (默认: $1e-4$)
- `alpha` (float, 可选) – eta 更新的指数 (默认: 0.75)
- `t0` (float, 可选) – 指明在哪一次开始平均化 (默认: $1e6$)
- `weight_decay` (float, 可选) – 权重衰减 (L2 惩罚) (默认: 0)

`step(closure) [source]`

进行单次优化 (参数更新).

参数:

- `closure` (callable) – 一个重新评价模型并返回 loss 的闭包, 对于大多数参数来说是可选的。

```
class torch.optim.LBFGS(params, lr=1, max_iter=20, max_eval=None,
tolerance_grad=1e-05, tolerance_change=1e-09, history_size=100,
line_search_fn=None)[source]
```

实现 L-BFGS 算法。

警告

这个 optimizer 不支持为每个参数单独设置选项以及不支持参数组 (只能有一个)

警告

目前所有的参数不得不都在同一设备上。在将来这会得到改进。

注意

这是一个内存高度密集的 optimizer (它要求额外的 $\text{param_bytes} * (\text{history_size} + 1)$ 个字节)。如果它不适应内存, 尝试减小 history size, 或者使用不同的算法。

参数:

- `lr` (float) – 学习率 (默认: 1)
- `max_iter` (int) – 每一步优化的最大迭代次数 (默认: 20)
- `max_eval` (int) – 每一步优化的最大函数评价次数 (默认: $\text{max} * 1.25$)
- `tolerance_grad` (float) – 一阶最优的终止容忍度 (默认: $1e-5$)
- `tolerance_change` (float) – 在函数值/参数变化量上的终止容忍度 (默认: $1e-9$)
- `history_size` (int) – 更新历史的大小 (默认: 100)

`step(closure) [source]`

进行单次优化 (参数更新).

参数:

- `closure` (callable) – 一个重新评价模型并返回 loss 的闭包, 对于大多数参数来说是可选的。

```
class torch.optim.RMSprop(params, lr=0.01, alpha=0.99, eps=1e-08, weight_decay=0,
momentum=0, centered=False)[source]
```

实现 RMSprop 算法。

由 G. Hinton 在他的课程中提出。

中心版本首次出现在 [Generating Sequences With Recurrent Neural Networks](#).

参数:

- `params` (iterable) – 待优化参数的 iterable 或者是定义了参数组的 dict
- `lr` (float, 可选) – 学习率 (默认: $1e-2$)
- `momentum` (float, 可选) – 动量因子 (默认: 0)
- `alpha` (float, 可选) – 平滑常数 (默认: 0.99)
- `eps` (float, 可选) – 为了增加数值计算的稳定性而加到分母里的项 (默认: $1e-8$)

• `centered` (bool, 可选) – 如果为 True, 计算中心化的 RMSProp, 并且用它的方差预测值对梯度进行归一化

• `weight_decay` (float, 可选) – 权重衰减 (L2 惩罚) (默认: 0)

`step(closure) [source]`

进行单次优化 (参数更新).

参数:

• `closure` (callable) – 一个重新评价模型并返回 loss 的闭包, 对于大多数参数来说是可选的。

`class torch.optim.Rprop(params, lr=0.01, etas=(0.5, 1.2), step_sizes=(1e-06, 50))[source]`

实现弹性反向传播算法。

参数:

• `params` (iterable) – 待优化参数的 iterable 或者是定义了参数组的 dict

• `lr` (float, 可选) – 学习率 (默认: 1e-2)

• `etas` (Tuple[float, float], 可选) – 一对 (etaminus, etaplus), 它们分别是乘法的增加和减小的因子 (默认: 0.5, 1.2)

• `step_sizes` (Tuple[float, float], 可选) – 允许的一对最小和最大的步长 (默认: 1e-6, 50)

`step(closure) [source]`

进行单次优化 (参数更新).

参数:

• `closure` (callable) – 一个重新评价模型并返回 loss 的闭包, 对于大多数参数来说是可选的。

`class torch.optim.SGD(params, lr=, momentum=0, dampening=0, weight_decay=0, nesterov=False)[source]`

实现随机梯度下降算法 (`momentum` 可选)。

Nesterov 动量基于 [On the importance of initialization and momentum in deep learning](#) 中的公式。

参数:

• `params` (iterable) – 待优化参数的 iterable 或者是定义了参数组的 dict

• `lr` (float) – 学习率

• `momentum` (float, 可选) – 动量因子 (默认: 0)

• `weight_decay` (float, 可选) – 权重衰减 (L2 惩罚) (默认: 0)

• `dampening` (float, 可选) – 动量的抑制因子 (默认: 0)

• `nesterov` (bool, 可选) – 使用 Nesterov 动量 (默认: False)

例子:

```
>>> optimizer = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
```

```
>>> optimizer.zero_grad()
```

```
>>> loss_fn(model(input), target).backward()
```

```
>>> optimizer.step()
```

Note

带有动量/Nesterov 的 SGD 的实现稍微不同于 Sutskever 等人以及其他框架中的实现。

考虑动量的具体情况, 更新可以写成

$$v = \rho * v + g$$
$$p = p - lr * v$$

其中, p 、 g 、 v 和 ρ 分别是参数、梯度、速度和动量。

这跟 Sutskever 等人以及其他框架的实现是相反的, 它们采用这样的更新

$$v = \rho * v + lr * g$$

$p = p - v$

Nesterov 的版本也类似地被修改了。

`step(closure) [source]`

进行单次优化 (参数更新).

参数:

- `closure (callable)` – 一个重新评价模型并返回 `loss` 的闭包, 对于大多数参数来说是可选的。



关注【深度学习爱好者】

更多学习资源免费获取

第八章 Automatic differentiation package – torch.autograd

`torch.autograd` 提供了类和函数用来对任意标量函数进行求导。要想使用自动求导，只需要对已有的代码进行微小的改变。只需要将所有的 `tensor` 包含进 `Variable` 对象中即可。

`torch.autograd.backward(variables, grad_variables, retain_variables=False)`

Computes the sum of gradients of given variables w.r.t. graph leaves. 给定图的叶子节点 `variables`，计算图中变量的梯度和。计算图可以通过链式法则求导。如果 `variables` 中的任何一个 `variable` 是非标量 (non-scalar) 的，且 `requires_grad=True`。那么此函数需要指定 `grad_variables`，它的长度应该和 `variables` 的长度匹配，里面保存了相关 `variable` 的梯度(对于不需要 `gradient tensor` 的 `variable`，`None` 是可取的)。此函数累积 `leaf variables` 计算的梯度。你可能需要在调用此函数之前将 `leaf variable` 的梯度置零。

参数说明:

- `variables` (`variable` 列表) – 被求微分的叶子节点，即 `ys`。
- `grad_variables` (`Tensor` 列表) – 对应 `variable` 的梯度。仅当 `variable` 不是标量且需要求梯度的时候使用。
- `retain_variables` (`bool`) – `True`, 计算梯度时所需要的 `buffer` 在计算完梯度后不会被释放。如果想对一个子图多次求微分的话，需要设置为 `True`。

`Variable`

API 兼容性

`Variable API` 几乎和 `Tensor API` 一致 (除了一些 `in-place` 方法，这些 `in-place` 方法会修改 `required_grad=True` 的 `input` 的值)。多数情况下，将 `Tensor` 替换为 `Variable`，代码一样会正常的工作。由于这个原因，我们不会列出 `Variable` 的所有方法，你可以通过 `torch.Tensor` 的文档来获取相关知识。

In-place operations on Variables

在 `autograd` 中支持 `in-place operations` 是非常困难的。同时在很多情况下，我们阻止使用 `in-place operations`。`Autograd` 的贪婪的释放 `buffer` 和 复用使得它效率非常高。只有在非常少的情况下，使用 `in-place operations` 可以降低内存的使用。除非你面临很大的内存压力，否则不要使用 `in-place operations`。

In-place 正确性检查

所有的 `Variable` 都会记录用在他们身上的 `in-place operations`。如果 `pytorch` 检测到 `variable` 在一个 `Function` 中已经被保存用来 `backward`，但是之后它又被 `in-place operations` 修改。当这种情况发生时，在 `backward` 的时候，`pytorch` 就会报错。这种机制保证了，如果你用了 `in-place operations`，但是在 `backward` 过程中没有报错，那么梯度的计算就是正确的。

`class torch.autograd.Variable [source]`

包装一个 `Tensor`，并记录用在它身上的 `operations`。

`Variable` 是 `Tensor` 对象的一个 `thin wrapper`，它同时保存着 `Variable` 的梯度和创建这个 `Variable` 的 `Function` 的引用。这个引用可以用来追溯创建这个 `Variable` 的整条链。如果 `Variable` 是被用户所创建的，那么它的 `creator` 是 `None`，我们称这种对象为 `leaf Variables`。

由于 `autograd` 只支持标量值的反向求导(即: `y` 是标量)，梯度的大小总是和数据的大小匹配。同时，仅仅给 `leaf variables` 分配梯度，其他 `Variable` 的梯度总是为 0。

变量:

- `data` – 包含的 `Tensor`
- `grad` – 保存着 `Variable` 的梯度。这个属性是懒分配的，且不能被重新分配。

- `requires_grad` – 布尔值，指示这个 `Variable` 是否是被一个包含 `Variable` 的子图创建的。更多细节请看 `Excluding subgraphs from backward`。只能改变 `leaf variable` 的这个标签。
- `volatile` – 布尔值，指示这个 `Variable` 是否被用于推断模式(即，不保存历史信息)。更多细节请看 `Excluding subgraphs from backward`。只能改变 `leaf variable` 的这个标签。
- `creator` – 创建这个 `Variable` 的 `Function`，对于 `leaf variable`，这个属性为 `None`。只读属性。

属性:

- `data` (any tensor class) – 被包含的 `Tensor`
- `requires_grad` (bool) – `requires_grad` 标记。只能通过 `keyword` 传入。
- `volatile` (bool) – `volatile` 标记。只能通过 `keyword` 传入。

`backward(gradient=None, retain_variables=False)[source]`

当前 `Variable` 对 `leaf variable` 求偏导。

计算图可以通过链式法则求导。如果 `Variable` 是非标量(non-scalar)的，且 `requires_grad=True`。那么此函数需要指定 `gradient`，它的形状应该和 `Variable` 的长度匹配，里面保存了 `Variable` 的梯度。

此函数累积 `leaf variable` 的梯度。你可能需要在调用此函数之前将 `Variable` 的梯度置零。

参数:

- `gradient` (`Tensor`) – 其他函数对于此 `Variable` 的导数。仅当 `Variable` 不是标量的时候使用，类型和位形状应该和 `self.data` 一致。
- `retain_variables` (bool) – `True`，计算梯度所必要的 `buffer` 在经历过一次 `backward` 过程后不会被释放。如果你想多次计算某个子图的梯度的时候，设置为 `True`。在某些情况下，使用 `autograd.backward()` 效率更高。

`detach()[source]`

Returns a new `Variable`, detached from the current graph. 返回一个新的 `Variable`，从当前图中分离下来的。

返回的 `Variable` `requires_grad=False`，如果输入 `volatile=True`，那么返回的 `Variable` `volatile=True`。

注意:

返回的 `Variable` 和原始的 `Variable` 公用同一个 `data tensor`。`in-place` 修改会在两个 `Variable` 上同时体现(因为它们共享 `data tensor`)，可能会导致错误。

`detach_()[source]`

将一个 `Variable` 从创建它的图中分离，并把它设置成 `leaf variable`。

`register_hook(hook)[source]`

注册一个 `backward` 钩子。

每次 `gradients` 被计算的时候，这个 `hook` 都被调用。`hook` 应该拥有以下签名:

`hook(grad) -> Variable or None`

`hook` 不应该修改它的输入，但是它可以选择性的返回一个替代当前梯度的新梯度。

这个函数返回一个句柄(`handle`)。它有一个方法 `handle.remove()`，可以用这个方法将 `hook` 从 `module` 移除。

Example

```
v = Variable(torch.Tensor([0, 0, 0]), requires_grad=True)
h = v.register_hook(lambda grad: grad * 2) # double the gradient
v.backward(torch.Tensor([1, 1, 1]))
#先计算原始梯度，再进 hook，获得一个新梯度。
print(v.grad.data)
```

```

2
2
[torch.FloatTensor of size 3]
>>> h.remove() # removes the hook
def w_hook(grad):
    print("hello")
    return None
w1 = Variable(torch.FloatTensor([1, 1, 1]),requires_grad=True)

```

w1.register_hook(w_hook) # 如果 hook 返回的是 None 的话，那么梯度还是原来计算的梯度。

```

w1.backward(gradient=torch.FloatTensor([1, 1, 1]))
print(w1.grad)
hello
Variable containing:
 1
 1
 1

```

```
[torch.FloatTensor of size 3]
```

```
reinforce(reward)[source]
```

注册一个奖励，这个奖励是由一个随机过程得到的。

微分一个随机节点需要提供一个奖励值。如果你的计算图中包含随机 `operations`，你需要在他们的输出上调用这个函数。否则的话，会报错。

参数:

- reward (Tensor) – 每个元素的 reward。必须和 `Variable` 形状相同，并在同一个设备上。

```
class torch.autograd.Function[source]
```

Records operation history and defines formulas for differentiating ops. 记录 `operation` 的历史，定义微分公式。每个执行在 `Variables` 上的 `operation` 都会创建一个 `Function` 对象，这个 `Function` 对象执行计算工作，同时记录下来。这个历史以有向无环图的形式保存下来，有向图的节点为 `functions`，有向图的边代表数据依赖关系(input->output)。之后，当 `backward` 被调用的时候，计算图以拓扑顺序处理，通过调用每个 `Function` 对象的 `backward()`，同时将返回的梯度传递给下一个 `Function`。

通常情况下，用户能和 `Functions` 交互的唯一方法就是创建 `Function` 的子类，定义新的 `operation`。这是扩展 `torch.autograd` 的推荐方法。

由于 `Function` 逻辑在很多脚本上都是热点，所有我们把几乎所有的 `Function` 都使用 `C` 实现，通过这种策略保证框架的开销是最小的。

每个 `Function` 只被使用一次(在 `forward` 过程中)。

变量:

- saved_tensors – 调用 `forward()`时需要被保存的 `Tensors` 的 `tuple`。
- needs_input_grad – 长度为 输入数量的 布尔值组成的 `tuple`。指示给定的 `input` 是否需要梯度。这个被用来优化用于 `backward` 过程中的 `buffer`，忽略 `backward` 中的梯度计算。
- num_inputs – `forward` 的输入参数数量。
- num_outputs – `forward` 返回的 `Tensor` 数量。
- requires_grad – 布尔值。指示 `backward` 以后会不会被调用。
- previous_functions – 长度为 `num_inputs` 的 `Tuple of (int, Function) pairs`。 `Tuple` 中的每单元保存着创

建 `input` 的 `Function` 的引用，和索引。

`backward(* grad_output)[source]`

定义了 `operation` 的微分公式。

所有的 `Function` 子类都应该重写这个方法。

所有的参数都是 `Tensor`。他必须接收和 `forward` 的输出 相同个数的参数。而且它需要返回和 `forward` 的输入参数相同个数的 `Tensor`。即：`backward` 的输入参数是 此 `operation` 的输出的值的梯度。`backward` 的返回值是此 `operation` 输入值的梯度。

`forward(* input)[source]`

执行 `operation`。

所有的 `Function` 子类都需要重写这个方法。

可以接收和返回任意个数 `tensors`

`mark_dirty(* args)[source]`

将输入的 `tensors` 标记为被 `in-place operation` 修改过。

这个方法应当至多调用一次，仅仅用在 `forward` 方法里，而且 `mark_dirty` 的实参只能是 `forward` 的实参。

每个在 `forward` 方法中被 `in-place operations` 修改的 `tensor` 都应该传递给这个方法。这样，可以保证检查的正确性。这个方法在 `tensor` 修改前后调用都可以。

`mark_non_differentiable(* args)[source]`

将输出标记为不可微。

这个方法至多只能被调用一次，只能在 `forward` 中调用，而且实参只能是 `forward` 的返回值。

这个方法会将输出标记成不可微，会增加 `backward` 过程中的效率。在 `backward` 中，你依旧需要接收 `forward` 输出值的梯度，但是这些梯度一直是 `None`。

This is used e.g. for indices returned from a max `Function`.

`mark_shared_storage(* pairs)[source]`

将给定的 `tensors pairs` 标记为共享存储空间。

这个方法至多只能被调用一次，只能在 `forward` 中调用，而且所有的实参必须是 `(input, output)` 对。

如果一些 `inputs` 和 `outputs` 是共享存储空间的，所有的这样的 `(input, output)` 对都应该传给这个函数，保证 `in-place operations` 检查的正确性。唯一的特例就是，当 `output` 和 `input` 是同一个 `tensor` (`in-place operations` 的输入和输出)。这种情况下，就没必要指定它们之间的依赖关系，因为这个很容易就能推断出来。

这个函数在很多时候都用不到。主要是用在 索引 和 转置 这类的 `op` 中。

`save_for_backward(* tensors)[source]`

将传入的 `tensor` 保存起来，留着 `backward` 的时候用。

这个方法至多只能被调用一次，只能在 `forward` 中调用。

之后，被保存的 `tensors` 可以通过 `saved_tensors` 属性获取。在返回这些 `tensors` 之前，`pytorch` 做了一些检查，保证这些 `tensor` 没有被 `in-place operations` 修改过。

实参可以是 `None`。



关注【深度学习爱好者】

更多学习资源免费获取

公众号：深度学习爱好者

第九章 torch.multiprocessing

封装了 `multiprocessing` 模块。用于在相同数据的不同进程中共享视图。

一旦张量或者存储被移动到共享单元(见 `share_memory_()`),它可以不需要任何其他复制操作的发送到其他的进程中。

这个 API 与原始模型完全兼容,为了让张量通过队列或者其他机制共享,移动到内存中,我们可以由原来的 `import multiprocessing` 改为 `import torch.multiprocessing`。

由于 API 的相似性,我们没有记录这个软件包的大部分内容,我们建议您参考原始模块的非常好的文档。

warning: 如果主要的进程突然退出(例如,因为输入信号),Python 中的 `multiprocessing` 有时会不能清理他的子节点。

这是一个已知的警告,所以如果您在中断解释器后看到任何资源泄漏,这可能意味着这刚刚发生在您身上。

Strategy management

`torch.multiprocessing.get_all_sharing_strategies()`

返回一组由当前系统所支持的共享策略

`torch.multiprocessing.get_sharing_strategy()`

返回当前策略共享 CPU 中的张量。

`torch.multiprocessing.set_sharing_strategy(new_strategy)`

设置共享 CPU 张量的策略

参数: `new_strategy(str)`-被选中策略的名字。应当是 `get_all_sharing_strategies()` 中值当中的一个。

Sharing CUDA tensors

共享 CUDA 张量进程只支持 Python3, 使用 `spawn` 或者 `forkserver` 开始方法。

Python2 中的 `multiprocessing` 只能使用 `fork` 创建子进程, 并且不被 CUDA 支持。

warning: CUDA API 要求导出到其他进程的分配一直保持有效, 只要它们被使用。

你应该小心, 确保您共享的 CUDA 张量不要超出范围。

这不应该是共享模型参数的问题, 但传递其他类型的数据应该小心。请注意, 此限制不适用于共享 CPU 内存。

Sharing strategies

本节简要概述了不同的共享策略如何工作。

请注意, 它仅适用于 CPU 张量 - CUDA 张量将始终使用 CUDA API, 因为它们是唯一的共享方式。

File descriptor-file_descriptor

NOTE: 这是默认策略(除了不支持的 MacOS 和 OS X)。

此策略将使用文件描述符作为共享内存句柄。当存储被移动到共享内存中, 一个由 `shm_open` 获得的文件描述符被缓存,

并且当它将被发送到其他进程时, 文件描述符将被传送(例如通过 UNIX 套接字)。

接收者也将缓存文件描述符, 并且 `mmap` 它, 以获得对存储数据的共享视图。

请注意, 如果要共享很多张量, 则此策略将保留大量文件描述符。

如果你的系统对打开的文件描述符数量有限制, 并且无法提高, 你应该使用 `file_system` 策略。

File system-file_system

这个策略将提供文件名称给 `shm_open` 去定义共享内存区域。

该策略不需要缓存从其获得的文件描述符的优点, 但是容易发生共享内存泄漏。

该文件创建后不能被删除, 因为其他进程需要访问它以打开其视图。

如果进程崩溃或死机，并且不能调用存储析构函数，则文件将保留在系统中。这是非常严重的，因为它们在系统重新启动之前不断使用内存，或者手动释放它们。为了记录共享内存文件泄露数量，`torch.multiprocessing` 将产生一个守护进程叫做 `torch_shm_manager` 将自己与当前进程组隔离，并且将跟踪所有共享内存分配。一旦连接到它的所有进程退出，它将等待一会儿，以确保不会有新的连接，并且将遍历该组分配的所有共享内存文件。如果发现它们中的任何一个仍然存在，它们将被释放。我们已经测试了这种方法，并且它已被证明对于各种故障都是稳健的。如果你的系统有足够高的限制，并且 `file_descriptor` 是被支持的策略，我们不建议切换到这个。



关注【深度学习爱好者】

更多学习资源免费获取

第十章 torch.legacy

此包中包含从 Lua Torch 移植来的代码。

为了可以使用现有的模型并且方便当前 Lua Torch 使用者过渡，我们创建了这个包。可以在 `torch.legacy.nn` 中找到 `nn` 代码，并在 `torch.legacy.optim` 中找到 `optim` 代码。API 应该完全匹配 Lua Torch。



关注【深度学习爱好者】

更多学习资源免费获取

第十一章 torch.cuda

该包增加了对 CUDA 张量类型的支持，实现了与 CPU 张量相同的功能，但使用 GPU 进行计算。它是懒惰的初始化，所以你可以随时导入它，并使用 `is_available()` 来确定系统是否支持 CUDA。

CUDA 语义中有关于使用 CUDA 的更多细节。

`torch.cuda.current_blas_handle()`

返回 `cublasHandle_t` 指针，指向当前 cuBLAS 句柄

`torch.cuda.current_device()`

返回当前所选设备的索引。

`torch.cuda.current_stream()`

返回一个当前所选的 `Stream`

`class torch.cuda.device(idx)`

上下文管理器，可以更改所选设备。

参数： - `idx` (int) – 设备索引选择。如果这个参数是负的，则是无效操作。

`torch.cuda.device_count()`

返回可得到的 GPU 数量。

`class torch.cuda.device_of(obj)`

将当前设备更改为给定对象的上下文管理器。

可以使用张量和存储作为参数。如果给定的对象不是在 GPU 上分配的，这是一个无效操作。

参数： - `obj` (Tensor or Storage) – 在选定设备上分配的对象。

`torch.cuda.is_available()`

返回一个 bool 值，指示 CUDA 当前是否可用。

`torch.cuda.set_device(device)`

设置当前设备。

不鼓励使用此函数来设置。在大多数情况下，最好使用 `CUDA_VISIBLE_DEVICES` 环境变量。

参数： - `device` (int) – 所选设备。如果此参数为负，则此函数是无效操作。

`torch.cuda.stream(stream)`

选择给定流的上下文管理器。

在其上下文中排队的所有 CUDA 核心将在所选流上入队。

参数： - `stream` (`Stream`) – 所选流。如果是 `None`，则这个管理器是无效的。

`torch.cuda.synchronize()`

等待当前设备上所有流中的所有核心完成。

交流集

`torch.cuda.comm.broadcast(tensor, devices)`

向一些 GPU 广播张量。

参数： - `tensor` (Tensor) – 将要广播的张量 - `devices` (Iterable) – 一个可以广播的设备的迭代。注意，它的形式应该像 `(src, dst1, dst2, ...)`，其第一个元素是广播来源的设备。

返回： 一个包含张量副本的元组，放置在与设备的索引相对应的设备上。

`torch.cuda.comm.reduce_add(inputs, destination=None)`

将来自多个 GPU 的张量相加。

所有输入应具有匹配的形状。

参数： - `inputs` (Iterable[Tensor]) – 要相加张量的迭代 - `destination` (int, optional) – 将放置输出的设备（默认值：当前设备）。

返回： 一个包含放置在 `destination` 设备上的所有输入的元素总和的张量。

`torch.cuda.comm.scatter(tensor, devices, chunk_sizes=None, dim=0, streams=None)`

打散横跨多个 GPU 的张量。

参数： - **tensor** (Tensor) – 要分散的张量 - **devices** (Iterable[int]) – int 的迭代，指定哪些设备应该分散张量。 - **chunk_sizes** (Iterable[int], optional) – 要放置在每个设备上的块大小。它应该匹配 `devices` 的长度并且总和为 `tensor.size(dim)`。如果没有指定，张量将被分成相等的块。 - **dim** (int, optional) – 沿着这个维度来 chunk 张量

返回： 包含 `tensor` 块的元组，分布在给定的 `devices` 上。

`torch.cuda.comm.gather(tensors, dim=0, destination=None)`

从多个 GPU 收集张量。

张量尺寸在不同 `dim` 的所有维度上都应该匹配。

参数： - **tensors** (Iterable[Tensor]) – 要收集的张量的迭代。 - **dim** (int) – 沿着此维度张量将被连接。 - **destination** (int, optional) – 输出设备 (-1 表示 CPU，默认值：当前设备)。

返回： 一个张量位于 `destination` 设备上，这是沿着 `dim` 连接 `tensors` 的结果。

流和事件

`class torch.cuda.Stream`

CUDA 流的包装。

参数： - **device** (int, optional) – 分配流的设备。 - **priority** (int, optional) – 流的优先级。较低的数字代表较高的优先级。

- `query()`

检查所有提交的工作是否已经完成。

返回： 一个布尔值，表示此流中的所有核心是否完成。

- `record_event(event=None)`

记录一个事件。

参数： **event** (Event, optional) – 要记录的事件。如果没有给出，将分配一个新的。 **返回：** 记录的事件。

- `synchronize()`

等待此流中的所有核心完成。

- `wait_event(event)`

将所有未来的工作提交到流等待事件。

参数： **event** (Event) – 等待的事件

- `wait_stream(stream)`

与另一个流同步。

提交到此流的所有未来工作将等待直到所有核心在调用完成时提交给给定的流。

`class torch.cuda.Event(enable_timing=False, blocking=False, interprocess=False, _handle=None)`

CUDA 事件的包装。

参数： - **enable_timing** (bool) – 指示事件是否应该测量时间（默认值：False） - **blocking** (bool) – 如果为 true，`wait()` 将被阻塞（默认值：False） - **interprocess** (bool) – 如果为 true，则可以在进程之间共享事件（默认值：False）

- `elapsed_time(end_event)`

返回事件记录之前经过的时间。

- `ipc_handle()`

返回此事件的 IPC 句柄。

- `query()`

检查事件是否已被记录。

返回： 一个布尔值，指示事件是否已被记录。

- `record(stream=None)`

记录给定流的事件。

- `synchronize()`

与事件同步。

- `wait(stream=None)`

使给定的流等待事件。



关注【深度学习爱好者】

更多学习资源免费获取

第十二章 torch.utils.ffi

```
torch.utils.ffi.create_extension(name, headers, sources, verbose=True, with_cuda=False, package=False,
                                relative_to='.', **kwargs)
```

创建并配置一个 cffi.FFI 对象,用于 PyTorch 的扩展。

参数:

- **name** (str) – 包名。可以是嵌套模块, 例如 `ext.my_lib`。
- **headers** (str or List[str]) – 只包含导出函数的头文件列表
- **sources** (List[str]) – 用于编译的 sources 列表
- **verbose** (bool, optional) – 如果设置为 False, 则不会打印输出 (默认值: `True`)。
- **with_cuda** (bool, optional) – 设置为 True 以使用 CUDA 头文件进行编译 (默认值: `False`)。
- **package** (bool, optional) – 设置为 True 以在程序包模式下构建 (对于要作为 pip 程序包安装的模块) (默认值: `False`)。
- **relative_to** (str, optional) – 构建文件的路径。package 为 True 时需要。最好使用 `__file__` 作为参数。
- **kwargs** – 传递给 ffi 以声明扩展的附加参数。有关详细信息, 请参阅 [Extension API reference](#)。



关注【深度学习爱好者】

更多学习资源免费获取

第十三章 torch.utils.data

`class torch.utils.data.Dataset`

表示 Dataset 的抽象类。

所有其他数据集都应该进行子类化。所有子类应该 override `__len__` 和 `__getitem__`，前者提供了数据集的大小，后者支持整数索引，范围从 0 到 `len(self)`。

`class torch.utils.data.TensorDataset(data_tensor, target_tensor)`

包装数据和目标张量的数据集。

通过沿着第一个维度索引两个张量来恢复每个样本。

参数：

- **data_tensor** (Tensor) — 包含样本数据
- **target_tensor** (Tensor) — 包含样本目标（标签）

`class torch.utils.data.DataLoader(dataset, batch_size=1, shuffle=False, sampler=None, num_workers=0, collate_fn=<function default_collate>, pin_memory=False, drop_last=False)`

数据加载器。组合数据集和采样器，并在数据集上提供单进程或多进程迭代器。

参数：

- **dataset** (Dataset) — 加载数据的数据集。
- **batch_size** (int, optional) — 每个 batch 加载多少个样本(默认: 1)。
- **shuffle** (bool, optional) — 设置为 `True` 时会在每个 epoch 重新打乱数据(默认: `False`)。
- **sampler** (Sampler, optional) — 定义从数据集中提取样本的策略。如果指定，则忽略 `shuffle` 参数。
- **num_workers** (int, optional) — 用多少个子进程加载数据。0 表示数据将在主进程中加载(默认: 0)
- **collate_fn** (callable, optional) —
- **pin_memory** (bool, optional) —
- **drop_last** (bool, optional) — 如果数据集大小不能被 batch size 整除，则设置为 `True` 后可删除最后一个不完整的 batch。如果设为 `False` 并且数据集的大小不能被 batch size 整除，则最后一个 batch 将更小。(默认: `False`)

`class torch.utils.data.sampler.Sampler(data_source)`

所有采样器的基础类。

每个采样器子类必须提供一个 `__iter__` 方法，提供一种迭代数据集元素的索引的方法，以及返回迭代器长度的 `__len__` 方法。

`class torch.utils.data.sampler.SequentialSampler(data_source)`

样本元素顺序排列，始终以相同的顺序。

参数： - **data_source** (Dataset) — 采样的数据集。

`class torch.utils.data.sampler.RandomSampler(data_source)`

样本元素随机，没有替换。

参数： - **data_source** (Dataset) — 采样的数据集。

`class torch.utils.data.sampler.SubsetRandomSampler(indices)`

样本元素从指定的索引列表中随机抽取，没有替换。

参数： - **indices** (list) — 索引的列表

`class torch.utils.data.sampler.WeightedRandomSampler(weights, num_samples, replacement=True)`

样本元素来自于 `[0,...,len(weights)-1]`，给定概率（weights）。

参数： - **weights** (list) — 权重列表。没必要加起来为 1 - **num_samples** (int) — 抽样数量



关注【深度学习爱好者】

更多学习资源免费获取

公众号：深度学习爱好者

第十四章 torch.utils.model_zoo

`torch.utils.model_zoo.load_url(url, model_dir=None)`

在给定 URL 上加载 Torch 序列化对象。

如果对象已经存在于 `model_dir` 中，则将被反序列化并返回。URL 的文件名部分应遵循命名约定 `filename-<sha256>.ext`，其中 `<sha256>` 是文件内容的 SHA256 哈希的前八位或更多位数字。哈希用于确保唯一的名称并验证文件的内容。

`model_dir` 的默认值为 `$TORCH_HOME/models`，其中 `$TORCH_HOME` 默认为 `~/.torch`。可以使用 `$TORCH_MODEL_ZOO` 环境变量来覆盖默认目录。

参数：

- **url** (string) - 要下载对象的 URL
- **model_dir** (string, optional) - 保存对象的目录

例子：

```
>>> state_dict = torch.utils.model_zoo.load_url('https://s3.amazonaws.com/pytorch/models/resnet18-5c106cde.pth')
```



关注【深度学习爱好者】

更多学习资源免费获取